大模型训练与推理 · 现场录音讲稿整理

录音转写 × 投影片对照（2026-10-16 下午场）

Transcript-to-Slide Alignment · NVIDIA Developer Day, LLM Training & Inference Track

项目	内容
音频来源	`录音机-20261016-1331.wav`（120:00）、`录音机-20261016-1531.wav`（64:00）；上午场另有 2 段录音与 **4 段 4K 现场视频**（见附录 A）
转写来源	`转写结果\*.srt / *.md`（讯飞「录音文件转写大模型」，含时间戳与句子级切分）
对齐对象	`大模型训练与推理\IMG_20261016_*.jpg` 共 96 张投影片照片（13:32:19–16:26:59）
场次结构	**4 场分享 / 5 位讲者**（由录音中的交接语与自我介绍确认）
对齐方法	照片拍摄时刻 − 录音起始时刻 = 音频偏移；每页取 `[拍摄 -8 s, 下一张拍摄]` 的转写窗口
转写规模	下午两段录音共 2827 句；落入 96 个页面窗口的文本 47,899 字
配套文件	《大模型训练与推理_内容整理与总结分析.pdf》（投影片整理）、《..._图片影印本.pdf》（原图影印）

说明：转写文本是**口语即席表达**的机器识别结果，存在错字、断句与术语音译错误（如"葡萄"≈CUDA Graphs、"脱矿"≈TurboQuant、"p 死"≈OOM）。本文在保留讲者原意的前提下按上下文纠正术语，凡无法确认处标 `(?)`；引号内的短句为转写原文（未逐字校对）。所有"幻灯片区"数字来自投影片 OCR，"口头区"数字来自录音转写，二者出处已在正文区分。

目录

摘要

一、录音与转写概况

- 1.1 四段录音的基本事实
- 1.2 对齐方法
- 1.3 可信度与局限

二、场次、讲者与议程（录音确认）

- 2.1 四场分享的交接点
- 2.2 讲者与投影片页的对应

三、时间轴对齐总表

四、口头讲解的增量信息与现场问答

- 4.1 第 1 场（孙方宇 / 张毅）：投影片讲"怎么做"，口头讲"为什么"
- 4.2 第 2 场（白洪潇）：优化顺序、代价与能力边界
- 4.3 第 3 场（徐文杰）：选型三步法，以及"无损也有代价"
- 4.4 第 4 场（卢祥龙）：落地与部署建议
- 4.5 四场问答精选
- 4.6 宣讲节奏的三个现象（对齐时发现的）
- 4.7 口头与投影片的几处差异（阅读时注意）

五、逐页口头要点（96 页）

- 5.1 第 1 场（孙方宇 / 张毅）：MegaMoE，第 1–34 页
- 5.2 第 2 场（白洪潇）：大规模训练实践，第 35–52 页
- 5.3 第 3 场（徐文杰）：KV Cache 压缩，第 53–67 页
- 5.4 第 4 场（卢祥龙）：Dynamo 与智能体推理，第 68–96 页

六、对《内容整理与总结分析》的修订清单

- 6.1 需要修订的表述
- 6.2 口头补充的事实（原文档未收录）
- 6.3 只出现在口头、投影片完全没有的内容

附录 A：上午场纪要

- A.1 补录段：主论坛英文演讲 10:54:48–11:07:01（据新补视频）
- A.2 上午两段录音的纪要（原稿）
- 1. 场次与讲者
- 2. 各场主要观点
- 3. 与下午"LLM 训练与推理"主题的关联
- 4. 存疑与说明

附录 B：ASR 术语对照与纠错表

附录 C：产物与方法清单

摘要

这份文件是《大模型训练与推理_内容整理与总结分析》的**录音补充册**：把 2026-10-16 下午现场的两段录音转写，与 96 张投影片照片按时间轴对上，回答三个问题——**谁在讲、他口头补了什么、哪些地方要修正我们只看投影片时的判断。**

一、场次与讲者（录音确认） 这不是一场连续演讲，而是**同一分会场的 4 场分享、5 位讲者**：

场次	时间	讲者	主题（投影片页）
第 1 场	13:31–14:11	**孙方宇**（前 13 分钟）、**张毅**	MoE 算子实现与通信计算融合（MegaMoE），第 1–34 页
第 2 场	14:12–15:00（含问答）	**白洪潇**（NVIDIA Megatron 团队）	结合 Megatron 的 DeepSeek-V4-Pro 大规模训练实践，第 35–52 页
第 3 场	15:00–15:31（问答延至 15:44）	**徐文杰**	面向高效 LLM 服务的 KV Cache 压缩（TurboQuant / KVTC / nvCOMP-ANS），第 53–67 页
第 4 场	15:44–16:27	**卢祥龙**（NVIDIA 解决方案与工程）	Dynamo 为智能体推理基础设施加速（KV Router / Flash Indexer / ThunderAgent），第 68–96 页

二、口头讲解带来的增量信息（只看投影片看不到的）

1. **明确的边界声明**：第 2 场讲者在问答中直接说"推理部署我这边不是非常熟悉，前面讲的这些其实**跟训练更相关**，推理方向可能完全不一样，**没法直接迁移过去**"。这修正了"训练与推理优化可以互相搬用"的直觉——两者是同一套思路在不同负载上的两次求解，但实现不可平移。
2. **口头数字比投影片更细**：第 1 场在讲 token sweep 时说清了"小 token 档位加速比只有 **3%**、到大 token 档位 **24%**、blockwise 在 32768 档可达 **72%**"，并解释了原因（小 token 时通信与 launch 主导、两种配置都是 memory-bound）。
3. **压缩路线的选型顺序**：第 3 场在问答中给出**先精度、后形态、再目标**的三步选型法，并给出甜点区（TurboQuant 约 3.6×、KVTC 约 8×），还明确"无损也是有代价的"——解压占 SM、与推理争用，**并发越高越不稳定**（吞吐波动约 −5.6% ~ +5%）。
4. **落地细节**：CUDA Graphs 需要额外几 G 到十几 G 显存（第 2 场问答）；Dynamo 的相关策略需要开源版本 **≥1.5**（问答中提 1.5/1.5.1）才可开启；建议把本地 L2 缓存设得很小而用一个大的共享 CPU/SSD 池，让所有节点互相可见——"就不会有不命中"（第 4 场问答）。
5. **风险与未验证处**：FP8 Indexer 的精度损失**未在长时训练中验证**；DeepSeek 官方参考实现用什么精度"其实没说"，属"不确定状态"（第 2 场问答原话）。

三、对原文档的主要修订 - 把"五段主题"改为**"4 场分享 / 5 位讲者"**，并补上姓名（原文档因只有投影片而无法确认讲者）。 - 原文档把第 68–78 页与第 79–96 页当作两部分，实际同属**第 4 场**（Dynamo 主题），内部是从"路由与索引"过渡到"智能体会话调度"。 - 会议名称确认：**NVIDIA 开发者日（Developer Day）中国站**的"**大语言模型训练与推理**分会场"（主持人开场与收尾均如此称呼）。 - 若干口头细节（24%/72%、解压占 SM、CUDA Graphs 显存代价等）补入对应章节。

一、录音与转写概况

1.1 四段录音的基本事实

录音文件	起始	时长	转写句数	内容
`录音机-20261016-0945.wav`	09:45	53:47	1086	上午主论坛：开场致辞、主题演讲（世界模型 / Cosmos）、开源生态
`录音机-20261016-1103.wav`	11:03	67:28	1490	上午：主论坛英文演讲后半（Nemotron Ultra 流程）、Nemotron 3 / Megatron、圆桌（6 位嘉宾）
`VID_20261016_105447.mp4`	10:54:48	15:00	81 句	**补录**：主论坛英文演讲前半（四大 scaling law、开放生态、NVL-72 的设计判断）
`VID_20261016_110133.mp4`	11:01:33	05:29	31 句	**补录**：与上段衔接的部分（预训练数据质量 4×、后训练、测试时计算）
`VID_20261016_110211.mp4`	11:02:11	0:08	1 句	极短片段（"NVIDIA 不是为了好玩才做开放技术"），与 110316 属同一段
`VID_20261016_110316.mp4`	11:03:16	1:01	5 句	承接 110211 的同一段（与录音二开头重叠，可互为校对）
`录音机-20261016-1331.wav`	13:31	**120:00**	**1879**	**下午分会场前 3 场（MegaMoE / 训练实践 / KV 压缩前半）**
`录音机-20261016-1531.wav`	15:31	64:00	948	**下午分会场后段（KV 压缩问答 / Dynamo）与收尾**

- 原始音频规格：48 kHz、双声道、24-bit PCM（未压缩 WAV，共约 5.0 GB / 4.9 小时）。
- 转写引擎：讯飞「录音文件转写大模型」（autodialect 方言自适应模式），输出含句子级时间戳的 `.srt`、整段 `.md` / `.txt` 与原始 `.json`。
- 本文件只使用**下午两段**（13:31、15:31）做投影片对齐；上午两段无对应投影片，其纪要见附录 A。

1.2 对齐方法

1. **时间基准**：录音文件名即录音起始时刻（`1331` = 13:31、`1531` = 15:31），与照片 EXIF 时刻同一时钟（同一台设备拍摄与录音的连续操作，未见跨设备时钟偏差迹象）。
2. **偏移换算**：`音频偏移 = 照片拍摄时刻 − 录音起始时刻`。13:32:19 的第一张照片 → 偏移 71 s；最后一张 16:26:59 → 第二段录音偏移 3359 s。
3. **窗口切分**：每页取 `[拍摄时刻 − 8 s, 下一张照片拍摄时刻]`（上限 120 s）的转写文本——前伸 8 s 是因为讲者通常"先说、后翻页"，取下一张照片的时刻作为结束是因为翻到下一页即说明本页讲完。

4. **人工核对**：逐页检查窗口内容与该页投影片主题是否一致（例如第 33 页窗口应讨论 token sweep 的加速比、第 50 页窗口应讨论 CP rank 的 CSA/Indexer 时间），错位处在逐页稿中标注。

1.3 可信度与局限

项目	说明
对齐精度	照片拍摄到讲者开口之间通常有 5–30 s 差（讲者先讲再看/先看再讲都有），因此窗口**只保证"这一页的主体讲解落在窗口内"**，不保证逐句对应
转写质量	中文口语识别总体可用，但**专有名词与英文术语错得较多**（`CUDA Graphs`→"葡萄"、`TurboQuant`→"脱矿/拖拉机"、`nvCOMP`→"pmc/pvpc"、`epilogue`→"IPA/IP"、`TMEM`→"他明"等），本文已按上下文纠正，仍存疑处标 `(?)`
丢失内容	现场投影片翻页声、观众提问（未用麦克风的部分）无法转写；部分问答只有讲者回答、没有提问原文
双人接力的场次	第 1 场由孙方宇、张毅两人接力（约 13:09 交接），页 1–15 属孙、页 16–34 属张
语速与冗余	口语重复、口头语（"就是说""对吧""嗯"）较多，逐页稿只保留信息点，不复述口语填充

二、场次、讲者与议程（录音确认）

2.1 四场分享的交接点

时间（录音偏移）	事件	原话（转写）
1331 @ 0:00	第 1 场开始	"实现，掌声欢迎两位。"（主持人）
1331 @ 0:13	孙方宇自我介绍	"我是孙方宇，然后我们这个分享分成两部分，我负责介绍我们推的算子……上的优化，接下来是张毅负责在（GPU？）上的优化"
1331 @ 13:13	交给张毅	"嗯好的，谢谢方宇的介绍，我是张毅，我接下来会介绍一下 op 相关的 MMA 的优化"
1331 @ 40:26	第 2 场开始	"感谢张毅和石光宇（应为'孙方宇'）老师的分享……下面有请白洪潇老师带来**结合 Megatron 的 DeepSeek-V4-Pro 大规模训练实践解析**"
1331 @ 41:19	白洪潇自我介绍	"我是来自 NVIDIA Megatron 的白洪潇"
1331 @ 86:03	第 3 场开始	"接下来我们从训练转向模型的服务……有请**徐平杰**（转写，正文中自称'徐文杰'）老师带来**面向高效 LLM 服务的 KV Cache 压缩**"
1531 @ 13:19	第 4 场开始	"下面有请**卢祥龙**老师带来 **Dynamo 为智能体推理基础设施加速**"
1531 @ 13:42	卢祥龙自我介绍	"卢晓龙（转写），来自于 NVIDIA 解决方案与工程团队"
1531 @ 60:28	分会场收尾	"今天我们 4 场分享已经结束……我们今天的分会场结束了"

2.2 讲者与投影片页的对应

场次	讲者	页段	时长（讲解 / 含问答）	说明
1	孙方宇	第 1–15 页	约 13 min	讲 MoE 推理的通信-计算融合动机、Blackwell 上的优化清单与 GEMM 侧三项优化
1	张毅	第 16–34 页	约 26 min	讲单内核架构、warp 分工、swapAB、FP8 量化、十一项优化与 token sweep、总结
2	白洪潇	第 35–52 页	约 40 min + 7 min 问答	训练 playbook：并行布局、显存策略、STEP 1–4、Lessons

场次	讲者	页段	时长（讲解 / 含问答）	说明
3	徐文杰	第 53–67 页	约 30 min + 13 min 问答	KV Cache 三重瓶颈、方法分类、TurboQuant / KVTC / nvCOMP-ANS、Takeaways
4	卢祥龙	第 68–96 页	约 44 min + 3 min 问答	Dynamo 能力全景、KV Router / Flash Indexer、智能体 SLA 与四组目标、ThunderAgent、资产与后续

> 注：讲者姓名以自我介绍为准，主持人的念法与转写存在差异（"石光宇"↔孙方宇、"徐平杰"↔徐文杰、"卢晓龙"↔卢祥龙），本表取**自述版本**，异写已在括注中标出。讲者所属单位只在自述中出现时才记录：白洪潇（NVIDIA Megatron 团队）、卢祥龙（NVIDIA 解决方案与工程）；孙方宇、张毅、徐文杰三位的单位在转写中**未被清晰说出**，故不作推断。

三、时间轴对齐总表

下表把 96 张照片逐页对到录音时间轴。**音频偏移**是该页照片拍摄时刻相对录音起始时刻的秒数（`分:秒`）；**场次/讲者**由录音中的交接语与自我介绍确定（分工边行标出）；**窗口字数**是该页对应转写窗口内的中文字数（0 表示该窗口没有有效语音，多为翻页/换场的空档）。

页	拍摄	音频偏移	场次/讲者	该页投影片标题（OCR）	窗口字数
1	13:32	1:11	T1 · 孙方宇	MegaMoE recap	397
2	13:33	2:15	T1 · 孙方宇	MegaMoE support matrix	330
3	13:34	2:58	T1 · 孙方宇	MegaMoE on Blackwell	433
4	13:35	4:03	T1 · 孙方宇	MegaMoE on Blackwell	87
5	13:35	4:08	T1 · 孙方宇	MegaMoE on Blackwell	243
6	13:35	4:33	T1 · 孙方宇	MegaMoE on Blackwell	274
7	13:36	5:16	T1 · 孙方宇	MegaMoE on Blackwell	292
8	13:37	5:54	T1 · 孙方宇	From a 32x4 Trick to a Full TMEM Transpose	414
9	13:37	6:50	T1 · 孙方宇	CuteDSL MegaMoE on Blackwell	592
10	13:39	8:11	T1 · 孙方宇	CuteDSLMegaMoEonBlackwell	136
11	13:39	8:25	T1 · 孙方宇	CuteDSL MegaMoE on Blackwell	287
12	13:40	9:05	T1 · 孙方宇	MegaMoE on Blackwell	397
13	13:41	9:59	T1 · 孙方宇	MegaMoEonBlackwell	258
14	13:41	10:35	T1 · 孙方宇	MegaMoE on Blackwell	708
15	13:43	12:12	T1 · 孙方宇	CuteDSL MegaMoE on Blackwell	413
16	13:44	13:13	T1 · 张毅	MegaMoEonBlackwell	264
17	13:45	13:57	T1 · 张毅	Overall Architecture: One Kernel Does Everyth...	435
18	13:46	15:06	T1 · 张毅	Warp Specialization	567
19	13:47	16:34	T1 · 张毅	The swapAB Variant: the Cure for Small Token ...	570
20	13:49	18:05	T1 · 张毅	Fp8 Quantization: per_tensor vs. blockwise	605
21	13:50	19:42	T1 · 张毅	Optimization 1: Cooperative Warp-group Schedu...	519
22	13:52	21:08	T1 · 张毅	Optimization 2: Task-tile Ping-pong	513
23	13:53	22:31	T1 · 张毅	Optimization 3:Multi-CTA CGA +TMA Multicast	613
24	13:55	24:12	T1 · 张毅	Optimization 4: Mainloop & Store Pipelines	568
25	13:56	25:46	T1 · 张毅	Optimization 5: All-Register Epilogue Fusion ...	541
26	13:58	27:07	T1 · 张毅	Why Gate/Up Interleave = 8: the WGMMA Registe...	699
27	14:00	29:09	T1 · 张毅	Optimization 6: A Single Active Dispatch Warp	184
28	14:00	29:29	T1 · 张毅	Optimization 7: Folding the Producer Warpgrou...	425
29	14:01	30:44	T1 · 张毅	Optimization 8: Early fc1_done Publish	537
30	14:03	32:14	T1 · 张毅	Optimization9:BlockwiseFC2Dual-Accumulator Ov...	746
31	14:05	34:38	T1 · 张毅	Optimization 10: Scheduler Groups and the fc1...	656
32	14:08	36:54	T1 · 张毅	Optimization 11: 16-byte FC2 Output Stores	623
33	14:09	38:32	T1 · 张毅	Token Sweep Results	520

页	拍摄	音频偏移	场次/讲者	该页投影片标题（OCR）	窗口字数
34	14:11	39:59	T1 · 张毅	Summary	623
35	14:13	42:10	T2 · 白洪潇	Two Challenges Define Training at Scale	686
36	14:15	44:43	T2 · 白洪潇	One Playbook, Two Workloads	395
37	14:16	45:39	T2 · 白洪潇	DeepSeek-V4-Pro Overview	714
38	14:19	48:21	T2 · 白洪潇	Model Features Drive Optimizations	279
39	14:20	48:57	T2 · 白洪潇	Choose the Parallel Layout	281
40	14:20	49:30	T2 · 白洪潇	Choose the Parallel Layout	728
41	14:24	53:01	T2 · 白洪潇	Context Length Drives Memory Policy	404
42	14:25	54:01	T2 · 白洪潇	Context Length Drives Memory Policy	393
43	14:26	55:10	T2 · 白洪潇	The Starting Points	488
44	14:27	56:27	T2 · 白洪潇	STEP 1: Runtime Overhead and Memory Trade-offs	740
45	14:32	61:07	T2 · 白洪潇	WheretheStepTime Goes	476
46	14:33	62:28	T2 · 白洪潇	STEP 2:Optimize the Hybrid Attention	752
47	14:37	66:35	T2 · 白洪潇	STEP 2: Optimize the Hybrid Attention	279
48	14:38	67:18	T2 · 白洪潇	STEP 3:Optimize the MoE	724
49	14:41	70:51	T2 · 白洪潇	STEP 3: Optimize the MoE	595
50	14:43	72:30	T2 · 白洪潇	STEP 4:Balance CP Workloads	479
51	14:44	73:47	T2 · 白洪潇	STEP4I Balance CP Workloads	689
52	14:49	78:20	T2 · 白洪潇	Lessons for Scalable Training	628
53	14:58	87:33	T3 · 徐文杰	Motivation: KV Cache as a Serving Bottleneck	323
54	14:59	88:18	T3 · 徐文杰	KV Cache Is a Serving Bottleneck	759
55	15:03	92:00	T3 · 徐文杰	KV Cache Compression Methods	600
56	15:04	93:31	T3 · 徐文杰	Where KV Compression Fits	685
57	15:07	96:34	T3 · 徐文杰	TurboQuant: Online KV Quantization	700
58	15:10	98:53	T3 · 徐文杰	TurboQuant: Layout Compressed KV for Decode	661
59	15:11	100:46	T3 · 徐文杰	TurboQuant: Tiled MMA Decode Attention on 4-b...	689
60	15:15	104:21	T3 · 徐文杰	TurboQuant:Decode Performance	666
61	15:17	106:13	T3 · 徐文杰	KVTC: Compression for Storage and Transfer	759
62	15:20	109:20	T3 · 徐文杰	KVTC: Integration and Pipelined Decompression	660
63	15:23	111:53	T3 · 徐文杰	KVTC: Single-Request TTFT	294
64	15:23	112:37	T3 · 徐文杰	KVTC: Throughput Under Memory Pressure	580
65	15:25	114:24	T3 · 徐文杰	nvCOMP-ANS: Lossless KV Cache Transfer	725
66	15:27	116:32	T3 · 徐文杰	nvCOMP-ANS: Compression and Serving Results ...	651
67	15:29	118:35	T3 · 徐文杰	Takeaways	447
68	15:45	14:49	T4 · 卢祥龙	What Dynamo can do today	812
69	15:49	18:02	T4 · 卢祥龙	KVRouter在生产环境中被广泛验证	141
70	15:49	18:21	T4 · 卢祥龙	请求进入后，KVRouter用两类信号为Worker打分	315
71	15:50	19:15	T4 · 卢祥龙	Flash Indexer 在链路里做什么	352
72	15:51	20:04	T4 · 卢祥龙	索引里存的是一张“位置到worker"的地图	367

页	拍摄	音频偏移	场次/讲者	该页投影片标题（OCR）	窗口字数
73	15:52	21:01	T4 · 卢祥龙	GEN01/06 THROUGHPUT	368
74	15:53	21:56	T4 · 卢祥龙	GEN04/06	281
75	15:53	22:33	T4 · 卢祥龙	GEN06/06 THROUGHPUT	298
76	15:54	23:22	T4 · 卢祥龙	性能数据	649
77	15:56	25:06	T4 · 卢祥龙	Standalone KV Indexer: 定位	285
78	15:56	25:45	T4 · 卢祥龙	Standalone KV Indexer:架构	530
79	15:58	27:04	T4 · 卢祥龙	SLAs of New Agentic Era	675
80	16:00	29:03	T4 · 卢祥龙	AgentX Trace	418
81	16:01	30:14	T4 · 卢祥龙	Single AgentX Session Characteristics	615
82	16:03	32:07	T4 · 卢祥龙	3 Layer Software Stack of Agentic Inference	636
83	16:05	34:03	T4 · 卢祥龙	AgentX Dashboard	424
84	16:06	35:19	T4 · 卢祥龙	Observability - Dynamo Provider Plugin	403
85	16:07	36:32	T4 · 卢祥龙	How many tokens fit in 1 GB	644
86	16:10	39:10	T4 · 卢祥龙	Goal 1: actual KV hit rate theoretical max	672
87	16:13	42:51	T4 · 卢祥龙	KV capacity for diff session cc	632
88	16:16	45:17	T4 · 卢祥龙	Goal 2: decode concurrency must keep up	646
89	16:18	47:13	T4 · 卢祥龙	Hard affinity mode DEP 下的负载均衡问题	438
90	16:19	48:35	T4 · 卢祥龙	Goal3: Agentic Routing Strategy	624
91	16:22	50:58	T4 · 卢祥龙	Goal 4 - Clever Session Gateway - Thunder Age...	337
92	16:23	51:52	T4 · 卢祥龙	ThunderAgent · how it decides	371
93	16:24	53:04	T4 · 卢祥龙	Agentic Serving ThunderAgent Benchmarks	257
94	16:24	53:46	T4 · 卢祥龙	ThunderAgent + KV offload	401
95	16:26	54:56	T4 · 卢祥龙	Thunderagent in RL rollout - Dynamo VeRL Inte...	346
96	16:26	55:51	T4 · 卢祥龙	Assets & Future Work	564

四、口头讲解的增量信息与现场问答

本章只收**投影片上没有、或讲得比投影片更细**的内容。所有条目来自转写，凡术语按上下文纠正，仍存疑处标 `(?)`。

4.1 第 1 场（孙方宇 / 张毅）：投影片讲"怎么做"，口头讲"为什么"

4.1.1 开场的"复习"给出了 MegaMoE 的问题定义

讲者用约 1 分半钟重述 MoE 为什么需要新方案，这段在投影片上只是一张对比图，口头把因果讲清楚了：

- 传统 MoE 推理中 **FC1/FC2 与通信使用的是不同资源，但被分开调度**，所以"**没法实现任何 overlap**"；
- 后续多卡并行能实现 dispatch/combine 与计算的**有限重叠**；
- 但 MoE 中 **FC1 的算量远大于 FC2**，而 dispatch/combine 的数据量与之**不匹配**，"所以在（CUDA）单独的 fused 实现里，**通算融合并不高效**"；
- 更细粒度的做法（讲义中称"第 4 个开源的 MoE 工作"）是"**用一个 scheduler、把数据切成更小单位、做更细粒度流水**"，从而实现通信的完美重叠；
- 本场的工作是**在这条路线上针对自家硬件做特定优化**（原话："我们这边主要是基于那个工作，然后针对我们 NVIDIA 的（Blackwell）硬件进行了一些特定的优化"）。

4.1.2 "融合不是免费"的三个具体代价

投影片只写了三条要点，口头把每条都说透：

1. **通信要借 shared memory**，并且"通信的 traffic 对我们的计算 traffic 是有影响的——如果通信的带宽利用率特别高，计算的效率是会下降的"。
2. **精度不对称放大通信占比**：用 NVFP4 时"计算量没变、速度翻倍"，但 combine 侧仍要用 **bf16**（因为要做 topK 之类的归约需要更高精度），于是一算一传之下，"**NVLink 上的 traffic 占比更大**"，对融合提出更高要求。口头还给出了一个更细的对比：**FP8 场景下 combine 还藏得进 L2 的 epilogue，换成 FP4 就藏不进去了**——这正是"combine 侧还要再优化"的直接诱因。
3. **内存序（memory ordering）昂贵**：单卡内用 mbarrier 就能保证；一旦跨卡走 HBM flag，"写 flag 并不简单，它包含两个步骤：先一个 fence 指令确保前面所有 traffic 到达 HBM，然后才能发 barrier 指令更新 flag"。**这就是后续 `batch-commit` 优化的动机**——把多次 flag 写入合并。

4.1.3 每个优化背后的"物理约束"

优化	口头给出的理由（投影片未写）
256×256 大 tile	"tile 越大，数据流开启率越好、性能越好"，但 **TMEM 只有 512 列**，所以引入了 "overlap ACC" 技巧：提前读取一块 ACC，"让下一个 tile 提前上来"，从而"接近两段式 ACC"
swapAB 动态 tileN	WGMMA 的 **M 起步就是 64**；推理时 token 数很少（"极端情况下可能只有 8"），"50 多个位置其实都没有做计算，浪费比较严重"；把 token 挪到 N 维后可以用 N=8 的 MMA 指令

优化	口头给出的理由（投影片未写）
FC2 用 1×64 量化	FC1 的 tile 是 64×128，**SwiGLU 会把 128 维变成 64 维**，所以"在这个维度上做量化就是 1×64"；不用 1×256 是因为 64×256 的 tile **寄存器压力太大**，拆成两个 64×128 又**需要额外的同步**、影响耗时
单 dispatch warp	"实测下来**只有一个（warp）开着就可以打满整个 HBM 带宽**"，原本设计 4 个"太多了"，把剩下的砍掉还能"省下相关计数器"、缓解寄存器压力，从而让更大的 tile 也能开两组 warp-group
两个 warp-group 协作	单个 warp-group 发 MMA 时"排布得不是特别满"，两组同时发射"可以在 MMA 上排布得更满，MMA 耗时占比提升，epilogue 暴露的耗时占比减小"
乒乓（ping-pong）	与协作相反，是**强制依赖顺序**，把两个 warp-group 的 MMA 与 epilogue 错峰，"把 epilogue 部分完全隐藏掉"；讲者强调"**两种策略要根据 tile 配置实测后再选**"
16 字节写出	`STG.16/STG.32` 写不连续数据效率低，"通过 shuffle 把不连续数据排成连续的，让一个线程拿 8 个连续元素，就可以调用位宽更高的 `STG.128`"
异步归约（UREDGR）	讲者强调这类异步归约"**不会阻塞后续的发射**"，因此可以叠在流水里；而 M 维在两种变体下都固定在 128 / 256

四步数据流（口头重述的完整版）：`dispatch 把远端 rank 的数据拉过来 → 等 flag 生效（依赖链的关键）→ FC1 + SwiGLU + FP8 cast（在 epilogue 内完成）→ 写 SMEM 并置信号量，FC2 等待后开始`。讲者用这条链解释了为什么"flag 的发布点"要前移、为什么"单 dispatch warp"够用——两者都是在压缩"等"的时间。

4.1.4 token sweep 的口头数字（比投影片更具体）

讲者在总结性能时给了一组**档位级的口语数字**（与投影片的文字结论一致，但投影片没有逐档口述）：

- 小 token 档位：加速比"大概 **3%**"；原因："**通信占比高、实际计算量小、launch 也不饱和**，两种配置其实都是 **memory-bound**，所以加速不明显"；
- 大 token 档位：per-tensor 可达"**24%**"；blockwise（DeepGEMM 风格）"从 token=8 的 3% 一路到 **32768 时的 72%**"；
- 原因归结为"**通信与计算融合、以及 epilogue 与 SwiGLU 的融合，减少了 HBM 的读写量**"。

> 这与主文档 3.1.4 的表述（per-tensor 几何平均 1.16×、峰值 1.35×；blockwise 峰值 1.72×）**不冲突**：口头讲的是"某个档位"的加速比，投影片写的是"几何平均/峰值"。

4.2 第 2 场（白洪潇）：优化顺序、代价与能力边界

4.2.1 优化顺序的依据被讲清楚了

- 先看 profile："做完第一步（fusion + CUDA Graphs）之后，再看就看不出直观的问题了，所以用 profile 把每个组件的运行时间统计出来"，结论是"**模型 80% 以上的时间花在 attention 和 MoE 两部分**"，于是优化集中在这两块。
- attention 内部又分成三块：**Indexer（蓝）**、**CSA（紫）**、**Group-O 投影（橙）**——这就是 STEP 2 三个模块级优化的由来。

- **收益随上下文变化的机制**："1M 的 case 下 indexer 占比会非常长，所以针对它的优化带来更多收益"——解释了为什么 STEP 2 在 64K 只有 +4.6%、在 1M 有 +9.0%。

4.2.2 STEP 1 的收益结构（口头）

- `fusion` 是"相对直接的方法，基本不会有负面影响，大部分情况都有收益"，但"**复杂结构不容易 fuse**"（比如 MoE 内部有很多不同 tile、里面还有 epilogue），所以"实际执行时还是会有很多 kernel 留在那里"。
- `CUDA Graphs` 是"开箱即用"的，但**要求整个 kernel 范围是静态的**，"里面有动态东西会导致 capture 失败"，且"**显存增长会比较明显**"。
- 因此 STEP 1 的效果是**上下文相关**的：64K 上"**超过 30%**"（因为 GPU idle 大），1M 上"**不到 3%**"（因为显存已经太紧、开不了 CUDA Graphs）。这与投影片第 44 页"64K 场景主要问题是大量 GPU idle"完全吻合，但口头把"为什么 1M 不行"补齐了。

4.2.3 问答揭示的三条边界（重要）

1. **能力边界**：被问"推理部署怎么配合这些优化"时，讲者回答"推理部署我这边不是非常熟悉，因为我前面讲的这些其实**跟训练更相关**；**推理方向可能完全不一样，这些东西没法直接迁移过去**"。→ 主文档中"训练与推理思路互通"的说法需要加这层限定（见第六章修订清单）。
2. **精度验证边界**：被问 FP8 Indexer 的精度损失（"4×V4 时用的是 bf16 算 QK^T 得到分数矩阵 S，现在改成 FP8"）时，回答"**非常长时间的训练确实没有做过**"，短期测试影响不明显；DeepSeek 官方"**没说用什么精度**"、其 reference 里"**并没有给具体实现**"，所以"只能是一个**不确定的状态**"，但"猜测他们应该验证过、损失有限"。
3. **显存代价**：CUDA Graphs"跟卡的类型没关系"，但"**可能带来几 G 甚至十几 G 的额外显存消耗**"，取决于剩余余量；可以"试一下给它开起来，看看会不会 OOM"。

4.3 第 3 场（徐文杰）：选型三步法，以及"无损也有代价"

4.3.1 生产环境的选型顺序

问答中最有价值的一段，讲者把"三条路线怎么选"讲成了一个决策顺序：

1. **先看精度**：`KVTC` 的计算很重（"肯定不可能完全是代价"），但"**压缩率可以做得很大，精度更高**"——"别看它是 8 倍，但它比 3.6 倍的 TurboQuant 精度更高"；
2. **再看算法形态**：是**存储型**还是**计算型**、"重不重"、"能否与 attention 等上层做融合"；
3. **最后看优化目标**：是要缓解 **HBM 压力**、**内存容量压力**，还是 **系统 I/O 压力**。

同时给出**甜点区**：TurboQuant 极限约 **4.5×**、实际甜点约 **3.6×**；KVTC **8×–64×**、**8×** 比较合适；两者"最性感的场景"都是**超长上下文**。

4.3.2 "无损也有代价"的量化解释

投影片只说"无损压缩约 1.4×/1.1×"，问答里把代价说透了：

- **解压要占 SM**："解压算子虽然不需要（专门硬件），但**绝对会占用 SM**，并且实测对 SM 的压力还挺多"，"解压执行的时候推理会受到一定程度的阻碍"；
- **但时间短**："解压带宽基本上在 **50 GB/s** 这个量级，所以解压会比较快"，真实场景下"时间短、对并发的挤占就比较少"；
- **并发越高越不稳定**：干扰会让"整个推理系统的稳态变得特别复杂"，"并发高了以后 TTFT 就下降了"，性能波动"**−5.6% 到 +5%**"这个量级；
- **结论**：如果"存储是绝对的压力"，选无损压缩"绝对是最好的"；如果"对存储不敏感、关注性能"，那么它在**低并发**下好、**高并发**下会变得不稳定。

4.3.3 集成与可移植性

- **KVTC 集成在 KV 缓存管理后端**：上层推理引擎发请求后，后端调用 KVTC，"整个 Transform 的映射是**完全动态做的**，**完全不需要模型做任何配（改动）**"；团队**尚未开源**，可定向提供代码。
- **换硬件**：NVIDIA GPU 上"提升曲线基本一样"；非 NVIDIA 硬件需要按具体数据的存储特点重做，因为"SM 的编程方式对数据的访问和合并很关键，**对数据对齐也很看重**"，不同硬件指令集（是否支持标量高效访问等）表现不同。

4.4 第 4 场（卢祥龙）：落地与部署建议

- **版本门槛**：问答提到"开源项目里有对应的设置，但比较新，**需要更新到 1.5 或者 1.5.1 以上**才能开启这样的策略"（指 KV offload/亲和相关策略）。
- **KV 分层建议**（问答中最具体的一条）：面对"多个 worker、上层还有会话代理"的分层结构，与其纠结各种淘汰/亲和策略，"我个人认为比较好的就是**把本地 L2（CPU 缓存）设得很小、甚至设为 0**，然后用一个**很大的共享 CPU 或 SSD 池**，让**所有节点都可以互相看到对方的 KV**——这样你就**不会有任何缓存不命中的问题**"。
- **RL 阶段的复用**：投影片第 95 页只写了与 VeRL 的集成，口头补充"这些调度策略在 **RL 的 rollout 阶段**也已经有应用"，并给出量级——**rollout 约 2.5× 提升，且 rollout 本身占到训练时间约 70%**（口语："rollout 占 70% 的时间"）。
- **容量与调度实况（口头）**：三级存储的容量被讲成"GPU 侧约 **1710 GB**、主机侧 G2 约 **2048 GB**、再往下的 G3 约 **1024 GB**"；**G3 大约只装得下 41 分钟**的会话写入；命中率则从 **95.8% 掉到 91.9%**（即投影片写的 −3.9 个百分点）；TP8 会话并发**过了 32 之后直接雪崩**。
- **ThunderAgent 的真实节奏（口头）**：讲者提到实测中"**一小时里 pause 45 次、resume 77 次**"——这个数字比只看"5 s 一拍、三个 util 门限"更能说明网关在实际负载下的抖动程度。
- **收尾**：主持人确认"今天我们 **4 场分享**已经结束"，并说明本分会场主题为"**大语言模型训练与推理**"，活动为 **NVIDIA 开发者日**。

4.5 四场问答精选

#	场次	问题（观众）	回答要点
1	2	这些优化能不能迁到推理部署？	讲者：与训练更相关，**推理方向可能完全不同，无法直接迁移**
2	2	CP 的"首尾"划分依据是什么？	确认 attention 计算量随序列长度（平方）增长、越往后越多；"理论上（各 rank）是完全均衡的"
3	2	有没有和类似 DCP（按 token 位置划分）的方法比较？	对方方案"覆盖粒度会更细"，但"推理场景不一样，不太好直接比较"
4	2	FP8 Indexer 的量化损失怎么办？	长时训练未验证；短期影响不明显；DeepSeek 参考实现精度不明（不确定状态）
5	2	CUDA Graphs 在消费级显卡上适用吗？阈值怎么衡量？	与卡类型无关；额外显存几 G 到十几 G；看余量、可试开看是否 OOM
6	3	无损压缩在上下文变长时表现如何？	测试只展示了一个上下文长度，"结论是**上下文越长越好**"
7	3	两个有损算法的甜点区对应什么业务场景？	超长上下文场景；TurboQuant 约 3.6×、KVTC 约 8×
8	3	数据访问效率提升 8× 是否更吃硬件/底层实现？	NVIDIA GPU 上曲线一致；其它硬件要按数据合并与对齐特性重做
9	3	生产上该放推理引擎还是 KV 后端？	三步法：精度 → 形态 → 目标压力（见 4.3.1）
10	3	三种算法对 SM 算力的开销有定量测试吗？无损是否任何场景都建议开？	解压占 SM 明显（约 50 GB/s 带宽）；**无损也有代价**，低并发好、高并发不稳（−5.6%~+5%）
11	3	KVTC 集成是否需要模型改动？	不需要，映射完全动态；尚未开源，可定向提供
12	4	单个 worker 的 offload 策略怎么用？	开源项目需 ≥1.5 / 1.5.1 版本
13	4	PD 分离 + 想在 KV 层再加一层共享层，怎么配？	建议本地 L2 设很小甚至 0，用大的共享 CPU/SSD 池，节点互相可见，"不会有不命中"

4.6 宣讲节奏的三个现象（对齐时发现的）

整理过程中反复出现三类"对不上"，它们本身就是关于这场分享的信息：

1. **照片普遍滞后于口播**。投影内容切换后讲者往往**先讲、后翻页**（或先翻页、观众稍后才拍照），实测滞后约 **5–45 秒**，个别页达 **1–2 页**。举三个确凿的例子：第 4/5 页是同一张 slide 相隔 5 秒的重拍，窗口里讲的已经是 256×256；第 10 页的内容实际讲在 `00:07:36-00:07:51`（照片按下时口播已在讲 UREDGR）；第 16 页"低精度 combine"真正讲于 `00:12:37-00:13:05`，落进了第 15 页的窗口——而第 16 页自身的窗口已经是张毅的开场流程。第 4 场同理（第 73、76、84、92、94 页都晚一页；第 63 页的 TTFT 窗口里，讲者其实已在讲第 64 页的三张吞吐图）。因此逐页稿的"口头要点"按**主题**对齐，而不是按秒硬对。
2. **技术密度高的页反而被跳过**。第 1 场里，第 8 页（`32×4` 起点、与 LLM 协作建模、Cayley/BFS 搜索、4 轮 10 条指令的搜索过程与贡献切分）**全程未口播，只用约 50 秒讲结论**；第 16 页（精度格式表

`E4M3/E8M0` block 32、`E2M1/bf16` amax block 16，以及"**bf16 是 REDG 唯一可原子加的格式，所以需要后置 topK 归约 kernel**"这条关键因果）**一句没提**。第 4 场里，第 69 页的 Baseten / 美团案例、第 76 页的 `440× / 42×` 与 benchmark 配置也都**只写在投影片上**。这提示：投影片承担"留档"，口头承担"讲故事"，两者不可互相替代——**也说明只看录音会漏掉最硬的技术内容，只看投影片会漏掉动机与代价**。

3. **录音本身有缺口**。第 3 场在两处各缺约 **2 分钟**音频（第 44→45 页、第 46→47 页之间），第 2 场第 27 页的窗口只剩约 28 秒、第 31 页前约 16 秒无声；第 4 场在 `[41:14]-[42:49]` 有约 **95 秒**缺失，正好盖住第 87 页的 `KV to keep = sessions × context × headroom` 与 cc=16/32/64/128 的逐行数字；录音在 **57:59** 结束，所以收尾问答只捕捉到前两条（其一答"soft affinity + KVCR"，其二关于单 worker/会话接入，未答完即转线下）。这些位置的数字只能回到投影片取，逐页稿中已标 `(?)`。

4.6.1 补记几处只在口头出现的关键数字

主题	口头内容	出处窗口
NVLink 利用率	融合后 NVLink 利用率从"**80% 多**"提到"**95% 以上**"	第 13–15 页附近
FP4 下的 combine	用 NVFP4 后 combine 更难藏——"**藏不进 L2 的 epilogue**"，所以要把 combine 移出 epilogue warp	第 12–14 页
依赖链	FC1 算完 → 写 flag → FC2 才能开始，这条依赖是"单 dispatch warp + early publish"要解决的根源	第 29–31 页
容量归因链	单会话 ISL 峰值 **219K** → 每 token KV **55.9 KB** → TP8 复制惩罚 vs DEP 无复制 → G3 只装得下约 **41 分钟**的写入 → 命中率 **95.8%** → **91.9%**	第 85–88 页
TurboQuant 的两个数	**4.5×** 是论文值，**3.6×** 才是社区/生产实测的"甜点"	第 57–60 页 + 问答
双 accumulator 的开启条件	`N = 64` 且寄存器足够时才开两组（不是所有配置都能开）	第 30 页
调度分组大小	group 大小要按 **expert 数与 token 数动态调整**	第 31 页
三级存储容量	GPU 侧约 **1710 GB**、主机 G2 约 **2048 GB**、G3 约 **1024 GB**；G3 只装得下约 **41 分钟**写入	第 86–87 页
TP8 并发拐点	会话并发**过了 32 就雪崩**（对应投影片的 TP8 ceiling = 33）	第 88 页
ThunderAgent 实况	实测"**一小时 pause 45 次、resume 77 次**"	第 91–92 页
RL rollout	rollout 约 **2.5×** 提升，rollout 占训练时间约 **70%**	第 95 页
单 GB 容量对比	TP8 **2.2K** vs DEP **17.9K** token/GB；4 节点合计 **5.0M** vs **30.6M**	第 85 页
KVTC 内存压力账本	命中率 **80%** → **20%**、吞吐 **140** → **50** 的断崖；KVTC 侧吞吐 **2.49×**、波动 **±3%**；解压开销约 **4~5×**（相对）`(?)`	第 63–64 页

4.7 口头与投影片的几处差异（阅读时注意）

主题	投影片	口头	处理建议
token sweep 加速比	geomean 1.16×/1.30×、峰值 1.35×/1.72×	"3% / 24% / 72%"（档位值）	两者口径不同，**并存引用**
TurboQuant 解码加速比	峰值 **4.84×**（长上下文，相对各代 FA 基线）	讲者口称"相对 FA 约 **1.5×**"`(?)`	**两者差异较大**：很可能对应不同上下文长度与不同 FA 基线（投影片柱值随上下文从 1.2× 升到 4.8×），引用时以投影片柱值为准并标明上下文
STEP 4 收益	**+12.2%** @1M	讲者口称"**10% 以上**"	口径一致（口头取整），采用投影片数值
KV 压缩与 TPOT	"没有直接的 TPOT 收益"	解压占 SM、并发高时 TTFT 下降、波动 −5.6%~+5%	口头版更接近"为什么没有收益、以及在什么条件下有害"
无损压缩	收益"温和"（≈1.4×/1.1×）	"免费午餐也是有代价的"，高并发不稳	补入代价，避免读者误以为"无损 = 无脑开"
训练 vs 推理	主文档推断"思路互通"	讲者："没法直接迁移过去"	已按此修订主文档
会议身份	只能从页脚与 `gtc26-*` 链接推测	明确为 NVIDIA 开发者日、"大语言模型训练与推理"分会场、4 场分享	已修订

五、逐页口头要点（96 页）

本章按场次顺序给出每一页的“讲者口头要点”。每页的结构固定为：**口头要点**（讲者实际讲述的内容，已按上下文纠正 ASR 术语）、**口头补充**（幻灯片上没有写、只出现在口头的信息）、**ASR 存疑**。页码与投影片影印本、以及《内容整理与总结分析》第六章的逐页条目一一对应。

5.1 第 1 场（孙方宇 / 张毅）：MegaMoE，第 1–34 页

投影片讲“怎么做”，口头更多地讲“为什么这么做”和“踩过什么坑”。

第 1 页 | MegaMoE recap — What is MegaMoE

> 拍摄 13:32:19 ｜ 音频 1331 段偏移 71 ｜ 讲者 孙方宇

- **口头要点**
- 开场先给判断：**"我们会发现就这样子的通算融合并不高效"**（00:01:10）——即当时这种通信与计算融合的做法效率不高。
- **DeepSeek 那个开源的工作**做了**更细粒度**的优化（00:01:14）：用一些（warp/单元 `(?)`）来做 tile，**把数据切分成 chunk 单位**（00:01:24，"创口单位"）。
- 通过这种更细粒度的切分，**实现通信与计算的完美 overlap**（00:01:26，"完美 OA"）。
- 明确归属：这是（DeepSeek）的工作（00:01:33）；**他们这边是基于 DeepGEMM 的那个工作**（00:01:37，"DC个的那个 v 的工作"），**再针对 NVIDIA 自己的 Blackwell 硬件做了特定优化。**
- 该实现**已经开源在 FlashInfer 的 repo 里**（00:01:44–00:01:49，"天2TM……不 flash行的 rp"）。
- 能力与范围：支持 **NVFP4 / MXFP8**、支持 **bf16**、支持一些**混合精度**（00:01:50–00:01:56）；架构上对（Hopper / Blackwell / Rubin 之一或数个，ASR 只吐出两个像人名的词 `(?)`）都有支持。
- **口头补充（幻灯片上没有的）**
- 幻灯片只画三条时间轴与 1.42x / 1.92x 的理论加速比，**没解释"为什么要细粒度"**；口播给了因果链：原来的通算融合不高效 → 所以要切得更细 → 才能拿到完美 overlap。
- **"chunk 单位"这个量化口径是口播给的**（幻灯片上只出现 Expert Wave / tile 的说法），说明他们对"细粒度"的落点是按 chunk/token 切分通信。
- 交代了**工程归属与适配动机**：算法思路来自 DeepSeek + DeepGEMM，他们做的事是"针对自家 Blackwell 硬件做特定优化"——这为后面 3–16 页全部是硬件适配型优化埋了伏笔。
- 这段完全没有解释幻灯片上的 **Comet（1.42x）** 这条对照线，也没提 1.92x 这个具体数字（见下条存疑）。
- **ASR 存疑**
- "d4个开源的那个 m工艺" 疑为 "**DeepSeek 开源的 MoE（MegaMoE）**"（`(?)`）；"这个是第四的工作" 疑为 "**这是 DeepSeek 的工作**"。
- "k天为个……天2TM或者不 flash行的 rp 里面" 只能确认含 **FlashInfer 的 repo**，前半截（疑为 **CuteDSL MegaMoE**）无法还原（`(?)`）。

- "使用1个 special 来自的""用一些外部来做派系" 两句完全不通，无法判断讲的是 special warp、sub-warp 还是 (?)（`(?)`）。
- "map4 的硬件" 疑为 **Blackwell**（或 **NVFP4**）相关表述（`(?)`）。
- "乌特威尔和莱威尔及 ps 路音" 三词与 OCR 的 **Hopper / Blackwell (GeForce) / Rubin** 无法一一对应（`(?)`）。
- **口播未出现 1.42x / 1.92x、也未提 Comet**；若需要引用这两个理论加速比，只能以投影片为准。
- 本窗口（00:01:10–00:02:20）**后半段已进入第 2 页 support matrix 的内容**（00:01:44 起），第 1 页真正对应的口播是 00:01:10–00:01:42。

第 2 页 | MegaMoE support matrix

> 拍摄 13:33:23 ｜ 音频 1331 段偏移 135 ｜ 讲者 孙方宇

- **口头要点**（实际讲述时间为 00:01:44–00:02:12，落在第 1 页的音频文件里）
- **已经开源在 FlashInfer 的 repo 里**（00:01:44–00:01:49）。
- 精度支持：**NVFP4**、**MXFP8**（00:01:50，"nvp4 或者 msp8"）、**bf16**（00:01:54，"f16"），以及**一些混合精度**（00:01:56，"一些子 c" `(?)`）。
- 架构支持：**Hopper 与 Blackwell**（以及 Rubin 一档 `(?)`）都有支持（00:01:56–00:02:02）。
- 会继续做 **NVIDIA 更先进架构的探索与支持**（00:02:02）。
- 具体点了两个方向：**NVL72**，以及**基于 NVLink 建的一些 RDMA 优化支持**（00:02:06–00:02:11，"nvl72" + "基于尼口建的一些 ordma"）。
- 讲完这张矩阵后立刻转入"推理/Blackwell 上更细粒度的优化，先总体分析这个问题"（00:02:15–00:02:20），即第 3 页。
- **口头补充（幻灯片上没有的）**
- 幻灯片只列条目，口播给了**路线图口吻**："还会做一些更先进架构的探索和支持"——把 NVL72 / RDMA 明确说成**未来方向（在研）**，与 OCR 里 `(WIP)` 的标注互相印证。
- 口播把 NVFP4 / MXFP8 / bf16 说成"现在支持"的既有能力，语气上区分于上面那句"还会做"，说明矩阵页里两类条目的成熟度不同。
- **ASR 存疑**
- "支持一些子 c" 疑为"支持一些**混合精度（mixed precision）**"（`(?)`）。
- "乌特威尔和莱威尔及 ps 路音" 疑分别对应 **Hopper / Blackwell / Rubin** 中的某两个或三个，无法一一指定（`(?)`）。
- "基于尼口建的一些 ordma" 中"尼口"疑为 **NVLink / NCCL**，"ordma" = **RDMA**（`(?)`）；OCR 记作 `72 and IB RDMA with NCCL GIN (WIP)`。
- **照片时刻（13:33:23 ⇒ 00:02:15）口播已转到第 3 页的问题分析**，本页内容实际讲于 00:01:44–00:02:12，错位约 30 秒。
- 幻灯片上的 **"Inference and training are both supported"**、**W4A4 / W8A8 / W16A16 / W4A8**、**W8A16** 这些具体精度组合、以及 **FlashInfer / cuDNN FE 集成**，在本窗口**都没有被口播**。

第 3 页 | MegaMoE on Blackwell — Challenges from fusion

> 拍摄 13:34:06 | 音频 1331 段偏移 178 | 讲者 孙方宇

- **口头要点**（00:02:27–00:03:23）
- 先做"总体分析"：**第一，fusion 不是免费的**（00:02:27，"优行不是免费的"）。意思是不**可能只是把 fusion 直接塞到 kernel 里面**就拿到提升（00:02:30，"把这个飞行的沃尔直接 s 到我们客人里面"）。
- 理由一：**通信需要消耗 SMEM**（00:02:36，"消耗我们的 m 消"），还要消耗别的资源（`（?）`）。
- 理由二：**通信的 traffic 对计算的 traffic 是有影响的**（00:02:41–00:02:45）。
- 具体表述：**如果通信的带宽利用率特别高，我们的计算效率是会下降的**（00:02:48–00:02:52）。
- 接着讲 NVFP4 场景：用 NVFP4 时**计算量没变，但速度翻了倍**（00:02:58–00:03:02）。
- 但 **combine 的时候仍然要用 bf16 精度**（00:03:05，"抗班的时候……仍然会使用 b26 的抗班精度"），因为需要做（累加/精度保证 `（?）`），**bf16 的精度更有保障**（00:03:13）。
- 结论：这导致 **NVLink 上 combine 的 traffic 占比更大**（00:03:18），**对他们的 MoE 提出了更多挑战**（00:03:23，"对我们这个肉就提出了更多的挑战"）。
- **口头补充（幻灯片上没有的）**
- 幻灯片只写 "Fusion is not free"，口播把它展开成"**不可能把 fusion 塞进 kernel 就完事**"这个更具体的设计直觉。
- 补了"**通信带宽利用率高 → 计算效率下降**"这条可观测现象（幻灯片只有 "Comm traffic can interfere mainloop TMA efficiency" 这一条被动表述，口播把它说成"你可以看到"的经验观察）。
- 补了 "**bf16 是为了精度更有保障**" 这条动机——幻灯片只断言 combine 一直是 bf16，没说为什么不能换低精度（这正是页 16 低精度 combine 要解决的问题，两页在口播里被显式连起来了）。
- 口播里"NVFP4 让计算翻倍但 combine 仍是 bf16"被讲成一条**比例失衡**的论证：不是通信变多了，而是**计算变快了，于是通信占比变大**。
- **ASR 存疑**
- "m 消""rste" 疑为 **SMEM** 与 **（其它）资源/register**（`（?）`）；OCR 对应 "Comm borrows smem from compute"。
- "托配置"（00:03:11）完全不通，无法还原（`（?）`）。
- "抗班 / 抗班精度" = **combine / combine 精度**；"b26" = **bf16**；"p16" = **bf16**；"nb 的场"疑为 **NVLink 上**（`（?）`）；"肉" = **MoE**。
- "我们的 AP4 的这个场景" 疑为 **NVFP4 场景**（`（?）`）。
- 幻灯片上的 **"Communication and compute in the same SM"** 与图的右侧两根指标（**GEMM efficiency / NVLink efficiency**）在口播里没有被点名，只以"带宽利用率高则计算效率下降"这一句概括。

第 4 页 | MegaMoE on Blackwell — Optimizations（大纲页，与页 5 同图）

> 拍摄 13:35:11 | 音频 1331 段偏移 243 | 讲者 孙方宇

- **口头要点**（实际讲述时间为 00:03:26–00:04:04）
- 总起：这是他们在 **Blackwell 上做的一些优化**的汇总（00:03:26，"在飞机上面干位的做做了一些优化"）。
- 分类讲法：**分为 matrix（GEMM）相关的、通信相关的、和 combine 相关的几个重要部分**（00:03:35）。
- **矩阵侧**：会用**更大的 kernel / tile**（00:03:42，"用更多的更大的开 s"）。
- **通信侧**：会针对它的 **memory order（"meyor"）做深度优化**（00:03:47）。
- **combine 侧**：有**各种方式让 combine 的流量不要影响矩阵计算**（00:03:51–00:03:55）。
- 紧接着就进入矩阵侧的第一条（00:03:59，属第 6 页）：这版用 **256×192** 的 tile，他们想用 **NVIDIA 能提供的最大 tile**。
- **口头补充（幻灯片上没有的）**
- 幻灯片是三条并列的大纲标题，口播给了**分类的标准**：按"会互相干扰的子系统"分（矩阵 / 通信 / combine），并明确把 combine 侧的目标说成**"不要让 combine 流量影响矩阵计算"**，这比 slide 上 "Combine perf issues and optimization(s)" 更具体地表达了设计意图。
- 口播把"通信侧"直接点名为 **memory order 优化**——这提前揭示了页 9–11 的主题，而大纲页上的写法是 MEMBAR synchronization，两者在口头叙述里被合并成了"内存序"这一件事。
- **ASR 存疑**
- "干位" / "单位" 疑为 **Blackwell**（`(?)`）。
- "meyor" 疑为 **memory order**（`(?)`），也可能是 **barrier**，但按后面的页 9–11 判断更可能是 memory order。
- "开 s" = **kernel / tile**（tile 更合上下文）。
- **页 4 与页 5 是同一张投影片的两次翻拍**（照片相隔 5 秒：13:35:11 / 13:35:16）；本窗口 00:03:59 起口播已经滑到**第 6 页的 256×192 → 256×256**，因此照片时刻的内容与页 4 自身不严格对应。
- 幻灯片上的 **Dynamic tileN with swapA/B**、**In-TMEM epilogue transpose**、**Batch-commit / Async UREDGR**、**Three modes / Phase interleaving / Adaptive traffic control / low-precision combine** 这些具体条目在本窗口**没有被逐条念**，只是被概括成三类。

第 5 页 | MegaMoE on Blackwell — Optimizations（页 4 的重拍）

> 拍摄 13:35:16 ｜ 音频 1331 段偏移 248 ｜ 讲者 孙方宇

- **口头要点**（本页没有独立讲解，窗口 00:04:04–00:04:39 讲的是下一张 slide 的内容）
- 这版（已有实现）用的是 **256×192 的 tile**（00:04:04–00:04:10，"256×1.2 的跳赛"）。
- 他们**想用 NVIDIA 能提供的最大的 tile**（00:04:10）。
- 理由很简单：**tile 越大，访存预算越好，性能就越好**（00:04:15–00:04:18，"你他要赛特越大，你的数据流……越好"）。
- 所以这次改成了 **256×256**（详见第 6 页）。

- **口头补充（幻灯片上没有的）**
- 讲者对这张大纲页**几乎完全跳过**：两张照片（13:35:11、13:35:16）只隔 5 秒，说明现场是同一张幻灯片停留，他嘴里已经在讲 256×256 的动机了——页 5 的价值主要在于**和页 4 互相校对 OCR 被裁掉的右侧条目**，而不是独立内容。
- **ASR 存疑**
- "256×1.2 的跳赛" 应为 **256×192 的 tile**（`(?)`，与 OCR 的 "Using 256x256 instead of 256x192" 对得上）。
- "数据流开了题越好" 疑为"**（访存）预算越好**"（`(?)`），对应 slide 的 "Larger tile has more memory access budget"。
- 本页窗口（00:04:04–00:04:39）内容属第 6 页，**页 5 自身无口播**；页 4 / 页 5 的对应关系见上一节。

第 6 页 | MegaMoE on Blackwell — Large tile size (256x256)

> 拍摄 13:35:41 ｜ 音频 1331 段偏移 273 ｜ 讲者 孙方宇

- **口头要点**（00:04:15–00:04:39）
- 换大 tile 的理由：**tile 越大，访存预算越好，性能就越好**（00:04:15，"他要赛特越大，你的数据流……越好"）。
- 但受 **NVIDIA 现在的设计**限制：**它（SMEM/TMEM）只有 512**（00:04:21–00:04:23，"它这边位只有 512 行"）。
- 所以真做 **256×256** 的时候，需要用一个 **overlap ACC 的 trick**（00:04:26，"会有一个 overaappabc 的 t"）。
- 具体做法：**在（epilogue `(?)`）的时候对一段（stage `(?)`）做提前读取**（00:04:30，"对一个 STL 进行一个提前读取"）。
- 于是**下一段可以提前上来**（00:04:35，"让下一个……提前上来"）。
- 效果：**接近实现"两个 ACC"的效果**（00:04:39，"接近实现两个 scs 的起"）。
- **口头补充（幻灯片上没有的）**
- 幻灯片只有一句 "Larger tile has more memory access budget" 与一句 "Overlapping acc trick …… nearly 2-stage acc"，口播补了**被 512 卡住**这个前提：大 tile 的收益不是无条件的，硬件只有 512，所以必须靠 ACC 的重叠搭接来"假装"有第二块缓冲。
- "提前读取/预取，让下一段提前上来"是**对图意（ACC0 / ACC1 在 512 列上错位搭接）的口语化解释**，OCR 的图只给了 512 columns、ACC0/ACC1、`48 48 for SF`，没有讲预取动作。
- **ASR 存疑**
- "它这边位只有 512 行" 与 OCR 的 **512 columns** 冲突——究竟是"列"还是"行"按 slide 应取 **512 列**（`(?)`）。
- "对一个 STL 进行一个提前读取" 中 "STL" 无法还原，疑为 **stage / ACC 的一半**（`(?)`）；"两个 scs" 疑为"**两个 acc / 2-stage acc**"（`(?)`）。
- "overaappabc 的 t" ＝ **overlap acc trick**。

- 幻灯片上的 `48 48 for SF`（缩放因子分区）在口播里**完全没有提**；如果听众只看口播，不会知道 512 列里还留了 SF 的位置。
- 本窗口（00:04:30–00:05:24）后段已进入**第 7 页的 swapAB**（00:04:45 起）。

第 7 页 | MegaMoE on Blackwell — Dynamic tileN with swapA/B

> 拍摄 13:36:24 | 音频 1331 段偏移 316 | 讲者 孙方宇

- **口头要点**（00:04:45–00:05:31）
- **第二个优化是 swapAB**（00:04:45）；讲者顺带说这个优化在（别处 `(?)`）**已经用到**（"丁丁建议会已经用到的优化" `(?)`）。
- 动机：**MMA 的形状在 M 维度通常是固定的 128 和 256**（00:04:50–00:04:56），**而在 N 维度上是非常（动态/不规则 `(?)`）的**（00:04:59）。
- 做法：**把 token 维度融合/挪到 N 维度**（00:04:59–00:05:05，"将头疼的维度融合到挪到 n 维度"）。
- 于是可以**动态决定现在应该存多少数据、M 应该算多少数据**（00:05:05–00:05:09）。
- 好处：**特别是在（N 很小 / decode `(?)`）的时候，可以跳过很多不需要的计算**（00:05:11–00:05:14，"创过很多不需要的……计算"）。
- 顺势转到转置（00:05:24 起，属第 8 页）：**用 FP4 的时候，epilogue 也需要转置**。
- **口头补充（幻灯片上没有的）**
- 补了**具体形状数字**：M 维固定 128 / 256——幻灯片上只写 "map N onto the token dimension to reduce tile quantization"，没有给 M 的取值。
- 把 slide 的 "**reduce tile quant(ization)**" 翻译成了"**动态决定存多少、算多少，从而跳过无效计算**"，这是同一件事的两种说法（量化浪费 ≈ 无效计算），口播那版更容易理解收益来自哪里。
- 口播用"在 N 很小的时候收益最大"点了适用场景（对应图上的 `Valid tokens` 空洞区），幻灯片只有图的标签。
- **ASR 存疑**
- "丁丁建议会已经用到的优化" 无法还原，疑为"**（DeepGEMM / 某处）已经用到的优化**"（`(?)`）。
- "n 个维度上是非常的来的" 疑为"**非常动态 / 非常稀疏**"（`(?)`）；"特别是我们的来的时候" 疑为"**在 N 很小 / decode 的时候**"（`(?)`）。
- "创过" = **跳过**；"调赛 / 挑赛" = **tile**。
- 幻灯片右下的 **"View of one CTA"** 布局图（Token / Weight / tileN / tileM / Valid tokens / ACC）在口播里**没有被逐块解释**。
- 本页窗口 00:05:21 之后（"我们还做了一个转制的优化"）已经是**第 8 页**的内容。

第 8 页 | From a 32x4 Trick to a Full TMEM Transpose

> 拍摄 13:37:02 | 音频 1331 段偏移 354 | 讲者 孙方宇

- **口头要点**（00:05:24–00:06:15）

- 他们又做了一个**转置的优化**：因为**用 FP4 的时候，epilogue 也需要转置**（00:05:24–00:05:27）。
- 分情况：在 NVFP4 场景下 **FC1 的 epilogue 要转置**；如果数据是 **FP8**，可以**用硬件自带的指令**完成（00:05:33–00:05:37，"用……自带的指令 ST vs"）。
- **但如果是 NVFP4 的数据，没有现成的指令**（00:05:43）。
- 如果只用**单纯的 ST / STS / LDS** 去搬，就会**特别慢**（00:05:45–00:05:50）。
- 他们的办法：观察到**通过 LDTM ＋ 寄存器置换（permute）能实现一种"互换"操作**（00:05:55，"对 l 不仅拍成的漏的的 s"）。
- **多互换几次，就能在 TMEM 上完成转置**（00:06:01–00:06:06，"在 t 面上进行转置"）。
- 收益：**节省了时间、省掉很多（额外指令 / 开销 `(?)`）**（00:06:07–00:06:09），把转置放到**更高效的 TMEM** 上做，**NVFP4 相关的（kernel）会快很多**（00:06:12–00:06:15）。
- **口头补充（幻灯片上没有的）**
- 幻灯片完全没写"**FP8 有现成指令、NVFP4 没有**"这个对比，是口播给的——它解释了为什么这套 TMEM 转置技巧只在 FP4 路径上必需。
- 明确否掉了朴素方案：**纯 ST/LDS 路线"特别慢"**，这是"为什么要折腾位级置换"的直接理由（幻灯片只有结论 "Fuse TMEM 16dp and 32dp ops to transpose 16x32 → 32x16"）。
- 口播把最终结果用"**多互换几次**"描述，等价于幻灯片上"融合 16dp / 32dp 操作、在生成器模型里搜索 4 轮 / 10 条 TMEM 指令"的工程结果。
- **ASR 存疑**
- "漏的的 s / l 不仅拍成的" 疑为 **LDTM（Load TMEM）＋ register permutation**（`(?)`）。
- "自带的指令 ST vs" 疑为硬件 TMEM 相关指令（**STTM** 或 ST 系列 `(?)`）。
- "MR 就我们节省了很多 MR 的一些咖"（00:06:09）无法还原，只能推断为"省掉了很多额外的（指令 / 访存）开销"（`(?)`）。
- **本页最核心的叙事在录音里几乎缺席**：32×4 起点、16×8→32×4 copy-pattern trick、"human hardware intuition × LLM-assisted modeling"、把 LDTM/置换/STTM 视为**位级置换**、Cayley/BFS 搜索、4 轮 / 10 条指令、以及页面上"LLM 贡献 vs 开发者贡献"的两栏切分，**全部没有被口播**（讲者只用了约 50 秒讲结论）。该页口述密度远低于幻灯片信息量。
- 本窗口 00:06:20 起已进入第 9 页的通信优化。

第 9 页 | CuteDSL MegaMoE on Blackwell — MEMBAR Synchronization optimization: batch-commit

> 拍摄 13:37:58 ｜ 音频 1331 段偏移 410 ｜ 讲者 孙方宇

- **口头要点**（00:06:20–00:07:33）
- 转到**通信上的优化**（00:06:20）。
- 先给对照：**在单个 kernel 里做 fusion，与现在（MegaMoE）这种做法是不一样的**（00:06:22–00:06:28）。

- 单 kernel 时，warp 之间**数据交换都在 SM 内部**，还可以**直接用 memory barrier 保证通信与内存序**，所以**比较高效**（00:06:33–00:06:39）。
- 但在 MegaMoE 里，**dispatch / 计算 / combine 是通过 HBM 上的 flag 来同步的**（00:06:42–00:06:48）。
- 要保证 **HBM 上的（flag / 数据可见性 `(?)`）正确**，就必须对 **memory order 做额外的保障**，而这个保障**比较昂贵**（00:06:53–00:06:59）。
- 看下面这张图：**你 load 了 token、store 了 token 之后，需要写一个 flag**（00:07:04），**写 flag 并不简单，包含两步**（00:07:09–00:07:12）。
- 第一步是（**MEMBAR**）指令：**确保前面所有 in-flight 的 traffic 都已经到达 HBM**（00:07:17–00:07:22）。
- 第二步才是 **atomic add**：**这时才能正确地更新这个 flag**，从而**别的 warp 直接读 HBM 就能读到有效数据**（00:07:22–00:07:29）。
- **口头补充（幻灯片上没有的）**
- 补了**"为什么这套在 MegaMoE 里不成立"的对照**：单 kernel 内部有 SMEM ＋ barrier 就够；一旦 dispatch / 计算 / combine 三方协作要走 **HBM 上的 flag**，就必须用 GPU scope 的 MEMBAR 兜底。幻灯片只写 "L2 flag is used for pipeline synchronization"，没说这是被迫的选择。
- 反复强调这份保障**"非常昂贵"**（00:06:59、00:07:33 两次），把页 9–11 三个优化统一到"降低内存序开销"这一条主线下——这是幻灯片没有的组织方式。
- 讲者特别提醒**写 flag 是两步（MEMBAR ＋ 原子加）**，因此"每写一次 flag 都很贵"，这为页 10 的 batch-commit 提供了量化直觉。
- **ASR 存疑**
- "没改工艺" / "没改 Moe" 疑为 **MegaMoE**（`(?)`）。
- "它需要发生通信才能推 a 进点"（00:06:30）断句错乱，按上下文是"**数据交换都在 SM 内部，所以比较高效**"（`(?)`）。
- "这个 f 子指令" 疑为 **MEMBAR(.GPU)**（`(?)`）；"他明 a 的指令" 疑为 **atomic add**（`(?)`）。
- "我们要保证 hbm 上的生 PA 是正确的" 中 "生 PA" 无法还原，疑为 **（flag 的）可见性 / 正确性**（`(?)`）。
- 本页窗口（00:06:50–00:08:19）**已包含页 10 的 batch-commit（00:07:36 起）与页 11 的 UREDGR（00:07:52 起）**，本页真正对应的口播是 00:06:20–00:07:33。

第 10 页 | CuteDSL MegaMoE on Blackwell — MEMBAR batch-commit 的权衡

> 拍摄 13:39:19 | 音频 1331 段偏移 491 | 讲者 孙方宇

- **口头要点**（实际讲述时间为 00:07:36–00:07:51，落在第 9 页的音频文件里）
- 想出的第一个办法：**batch（-commit）**（00:07:36，"第一就是 best"）。
- 核心思路：**不必每读一条 token 就（发一次 MEMBAR / 更新 flag）来保证正确性**（00:07:40）。
- **可以用多条 token，然后只释放一次 MEMBAR**（00:07:41–00:07:47）。

- 目的：**减少整个（内核）生命周期里 MEMBAR 的次数，从而提高性能**（00:07:47–00:07:51）。
- **口头补充（幻灯片上没有的）**
- 口播把这条优化讲成一条纯收益链："写 flag 贵 → 那就攒批 → MEMBAR 次数变少 → 性能变好"。
- **幻灯片上的代价一侧他没有提**：OCR 写着 "A tunable trade-off between fewer MEMBARs and higher latency"、以及 "MEMBAR can stall requests"——口播既没说"会变慢/延迟变高"，也没说这是**可调**的折衷参数。
- 因此如果只看口播，会误以为 batch 越大越好；幻灯片的口径是"要在 MEMBAR 数量与延迟之间调"。
- **ASR 存疑**
- "to / 门块" 疑为 **MEMBAR**（`(?)`）； "第一就是 best" 疑为 "**第一个就是 batch(-commit)**"（`(?)`）。
- "我们可以不要读一条 to，肯就保证正确性" 断句错乱，按上下文是"**不必每读一条 token 就同步一次**"（`(?)`）。
- **幻灯片上的具体批量数字（Dispatch to FC1：batch of 2；FC2 to Combine：batch of 4）在录音中完全没有出现**，口播只说"多条 / 一次"。
- 照片时刻（13:39:19 ⇒ 00:08:11）口播已在讲**第 11 页的 UREDGR**，本页内容实际讲于 00:07:36–00:07:51，错位约 35 秒。

第 11 页 | CuteDSL MegaMoE on Blackwell — MEMBAR Synchronization optimization: async UREDGR

> 拍摄 13:39:33 ｜ 音频 1331 段偏移 505 ｜ 讲者 孙方宇

- **口头要点**（00:07:52–00:08:28）
- 第二个办法：**用（异步原子加的）指令**（00:07:52–00:07:57）。
- 指令形如 `red.async.release.gpu.global.add.u32`（00:07:57，"r 点 ac 点 re 历史点 GPU"）。
- 这条指令被做成**单独的一条 UREDGR 指令**（00:08:02，"uredg 二"）。
- 语义：**对某个地址的数据做 atomic add，并且这次更新自带（release / 内存序）语义**（00:08:08–00:08:12）。
- 关键差异：**它不会阻碍后续指令的发射**（00:08:16，"不会阻碍我们后续指令的……"）。
- 对比结论：**原来用 MEMBAR 会（阻塞），换成 async 之后就可以很快地发出去**（00:08:21–00:08:24）。
- 一句话收束这一段：**这就是 Blackwell 上关于通信内存序上的一些优化**（00:08:28）。
- **口头补充（幻灯片上没有的）**
- 口播把 UREDGR 的价值说成"**把 barrier 语义内建进指令、因此不再挡住后续发射**"，比幻灯片 "Async atomic add instruction with built-in MEMBAR / UREDGR is non-blocking" 多了一层"**发射不被阻塞**"的机制解释。

- 他明确说这条指令是**被拆成单条指令形态**用的（"会被弄成一条单独的 UREDGR 指令"），暗示这不是直接用 PTX 原语，而是一层封装——幻灯片上只有一行 PTX 指令文本。
- **ASR 存疑**
- "r 点 ac 点 re 历史点 GPU" 对应 `red.async.release.gpu`，但口语里**没有念出** `.global.add.u32`，后半截是从幻灯片补的（`(?)`）。
- "数据公司的一个指令"（00:07:57）无法还原（`(?)`）。
- "呃艺许" = **发射**；"uredg 二" = **UREDGR**。
- 幻灯片上 Before / After 两条时间轴的具体排布（After 去掉了独立 MEMBAR 块、Dispatch 行的加载可以更早连续发射）在口播里只用一句"可以快速发出"概括，没有逐行对图。

第 12 页 | MegaMoE on Blackwell — Three modes for combine traffic（模式一）

> 拍摄 13:40:13 | 音频 1331 段偏移 545 | 讲者 孙方宇

- **口头要点**（00:08:33–00:09:17）
- 接下来是对 **combine 的特别优化**（00:08:33）。
- 回顾历史场景：**之前当你在 FP8 的时候**（00:08:36），combine 流量**可以被完美地藏在 L2 的 epilogue 里面**（00:08:51），所以**那个时候这种 fusion 方式是有效的**（00:08:58）。
- 但**到 NVFP4 的时候，你的计算变快了，留给 epilogue 的时间更短了**（00:09:02–00:09:07）。
- 于是（原来那套）**combine 的 traffic 还是照旧用**，就会导致 **epilogue 这一块变得很长**（00:09:07–00:09:10）。
- 他指图补充：**这里是 8 个（stage / wave `(?)`），有 8 就代表性能能达到最优**（00:09:14–00:09:17）——意思是这个数字原本是"理想状态"的标志，现在被拉长了。
- **口头补充（幻灯片上没有的）**
- 幻灯片只陈述 "the issue makes nvfp4 worse since it halves mainloop"，口播给了**完整因果链**：计算变快 → 主循环时间减半 → 留给 epilogue 的窗口变短 → combine 藏不进去 → epilogue 变长。
- 补了**历史对照**："在 FP8（MXFP8）时代这套是成立的"——这是幻灯片上只以 "(DeepGEMM solution, long time ago)" 一句带过的信息，口播把它讲成了"当年能成立、现在失效"的时间线。
- 图上那根 "8" 的读数是他现场读图给的（幻灯片上没有数字），说明他在用图的某个计数来论证"原本已到最优"。
- **ASR 存疑**
- "就近的 msp8 乘 MS4"（00:08:45）无法还原，疑为 **MXFP8 / FP8 与 NVFP4 的组合场景**（`(?)`）。
- "这里是 8 布的 / 有 8 就代表我们的（?）性能达到最优" 只能确认他在读图上某个"**8**"，无法确定是 stage、wave 还是第 8 个块（`(?)`）。
- "健康的模型实现的比较贴合硬件"（00:08:40）疑为"**（原来的 combine）实现比较贴合硬件**"（`(?)`）。
- 幻灯片上的 **"Burst NVLink traffic"**、**"epilogue becomes longer, blocking L1/L2"** 这两个词口播都**没有直接说**，只说了"epilogue 变得很长"。

第 13 页 | MegaMoE on Blackwell — Three modes for combine traffic（模式二、模式三）

> 拍摄 13:41:07 ｜ 音频 1331 段偏移 599 ｜ 讲者 孙方宇

- **口头要点**（00:09:21–00:10:02）
- 于是想到**用一些其他方式来传输 combine 的流量**（00:09:21–00:09:23）。
- **一共有两个方式**（00:09:27）；总思路是**把 combine 的流量分出来**（00:09:29）。
- **Mode 2**：用 **dispatch 的 warp（group）**来做 combine；好处是**可以直接复用 dispatch（已有的 warp），不额外增加占用**（00:09:41）。
- **Mode 3**：**单独（新开）一个 warp（group）**来做 combine；好处是**combine 可以提前得更早，NVLink 流量（利用率）更高**（00:09:45–00:09:48）。
- 在 Mode 3 下，**FC2 的 epilogue 不再是一个（带 barrier 的 `(?)`）东西，只是一个单独的 store**（00:09:50–00:09:54）。
- 因此**这段变得很短，计算的推进不受影响**（00:09:56）。
- **口头补充（幻灯片上没有的）**
- 幻灯片只有两句标题式的定义，口播补了**两条收益解释**：Mode 2 的"不额外占用 warp"、Mode 3 的"combine 更早 + NVLink 利用率更高"。
- 还补了 **Mode 3 之后 FC2 epilogue 退化成"单纯 store"** 这一后果——这正是 epilogue 不再变长的原因，幻灯片上的时间轴画出来了但没文字说明。
- 口播把这两个模式和页 15 的 traffic control 连成一条线（"combine 不在 epilogue warp 里"才用得上后面的限速），幻灯片本身没写这个依赖。
- **ASR 存疑**
- "用的开始的外尔部" 疑为"**用 dispatch 的 warp group**"（`(?)`）；"单独用卡的外来做 o 办" 疑为"**单独用（新）的 warp 来做 combine**"（`(?)`）。
- "由一个 c 二的 ipl 它不是一个 mo / ST 可它只是一个单独的 lokst"（00:09:50）无法确定讲的是 **mbarrier** 还是 **local / remote store**，只能确认"FC2 epilogue 现在只剩一个 store"（`(?)`）。
- 幻灯片上的 **"Decouple combine from epilogue warps"** 与 **"Backpressure from NVLink still blocks the TMA LDG issue"** 两句**没有被单独口播**（后者只在页 15 以"NVLink 流量影响计算"的形式回响）。
- 本页窗口末段 00:10:02 起已进入**第 14 页的相位交织**。

第 14 页 | MegaMoE on Blackwell — Phase interleaving between FC1 and FC2

> 拍摄 13:41:43 ｜ 音频 1331 段偏移 635 ｜ 讲者 孙方宇

- **口头要点**（00:10:02–00:11:42）
- 这是一个**（FC1/FC2 相位）特定的优化**（00:10:02），先看图（00:10:05）。
- 之前做 MegaMoE 时的做法是：**需要多个 FC1 的 tile，然后多个 FC2 的 tile**（00:10:13），**但这并不符合硬件最初（论文）的设计**（00:10:19）。

- 硬件最初的设计意图：**第二阶段（remote store）比较长，希望它藏在 FC1 里面**（00:10:24–00:10:31）；**FC1 的 epilogue 比较短，这样能提供多一点时间给第二阶段的 epilogue**（00:10:32）。
- 但之前**只能做成（固定/成组 `(?)`）的形式**，原因是**存在数据依赖**（00:10:38–00:10:41）。
- 依赖细节：**FC1 把数据写到（SMEM）里面的时候，需要一个 flag 的更新，而这个更新还需要一段时间，后续 FC2 的 tile 才能看到**（00:10:43–00:10:52）。
- 所以如果**立马写一个 FC1 tile 再接一个 FC2 tile**，**这个 FC2 tile 就得等前面的数据 ready，会有额外开销、形成（空泡/等待 `(?)`）**（00:11:00–00:11:08）。
- 解法 **phase interleaving**，idea 很简单：**一开始多做几个 FC1 的 tile，然后 FC1/FC2 交叉**；**交叉的 FC1 与 FC2 tile 之间没有依赖**（00:11:13–00:11:22）。
- 于是就能**完美实现 FC1/FC2 各自的 epilogue 重叠**（00:11:27）。
- **实测数字：NVLink 利用率从 80% 多提升到 95% 以上**，因此判断**这个优化非常有效**（00:11:31–00:11:42）。
- **口头补充（幻灯片上没有的）**
- **"80% 多 → 95% 以上"是这一段唯一的实测数字**，幻灯片上没有（页面上只有 "Perfect overlap with L1 mainloop" 这类定性与 "Prologue state" 描述）。
- 补了**"为什么不能直接交叉"的原因**：FC1 写完 SMEM 要发 flag，flag 生效有延迟，FC2 tile 才能读到——这是 slide 上 "cutoff the L1/L2 dependency in steady state" 一句话背后真正的动机。
- 口播把 prologue 的用意讲清了：**"一开始多做几个 FC1 tile"就是在 prologue 阶段先把 L1 的活垫上**，从而在 steady state 里切断 L1→L2 的依赖（幻灯片只写 "consume some L1 tiles"）。
- 他还点了硬件"最初论文的设计意图"（第二阶段长、藏进 FC1），这是对页 1 那些时间轴图的一种回溯解释——说明现在这套相位安排是**有意对齐硬件原本的节拍**。
- **ASR 存疑**
- "公具的形式"（00:10:38）无法还原，只能推断为"**固定 / 成组 / 全同步的形式**"（`(?)`）。
- "格能我 m 人写到编里面"（00:10:47）疑为"**写到 SMEM 里面**"（`(?)`）；"不的更新" 疑为 "**flag 的更新**"（`(?)`）。
- "雷 ATC 的问题"（00:10:52）疑为"**（数据）依赖 / 延迟的问题**"（`(?)`）；"形成一些发火"（00:11:08）疑为"**形成一些空泡**"（`(?)`）。
- "各自的……美女 or"（00:11:27）无法还原，按上下文应为"**各自的 epilogue（与各自的内存序保证）**"（`(?)`）；"在 rwl 上" 无法还原（`(?)`）。
- 幻灯片上的 **"FC1/FC2 tile schedule matters"** 与 "RS"（Remote Store）这个缩写口播都没用，他直接说"第二阶段 / epilogue"。

第 15 页 | CuteDSL MegaMoE on Blackwell — Adaptive NVLink traffic control

> 拍摄 13:43:20 ｜ 音频 1331 段偏移 732 ｜ 讲者 孙方宇

- **口头要点**（00:11:42–00:12:37）
- 承接前面说过的判断：**NVLink 的流量可能会影响整个计算的有效效率**（00:11:42–00:11:53）。

- 所以又做了 **adaptive traffic control**（00:11:54，"adoptive 的 reading, traffic control"）。
- 场景：**你的 combine 流量已经不多了，但还有 dispatch 的（成）流量**（00:11:57–00:12:02）。
- 关键判断：**你并不需要 dispatch 一直维持那么高的（速率 / 带宽）**（00:12:02–00:12:04）。
- 问题：**如果 dispatch 太激进，在几个 block 期间把那个流量占满**（00:12:07），**NVLink 这边就会（堵住/阻塞），影响到一些底层的东西**（00:12:11–00:12:19）。
- 后果：**会产生（队头阻塞** `(?)`**），导致正常的（TMA 之类）被（卡住）**（00:12:22）。
- 做法：**做一个（动态）调度的尝试，让（发射）慢一点**（00:12:26–00:12:29）。
- 然后**观察什么时候我们的计算吞吐是最大的**（00:12:31），**达到那个点就调到最好的状态**（00:12:36）。
- **口头补充（幻灯片上没有的）**
- 幻灯片写的是一套具体机制（**测每个 chunk 的 NVLink 往返 → EMA → nanosleep 偏差**），口播把它压缩成一个**控制论式的直觉**："dispatch 不必一直满速，让发射慢一点，找计算吞吐最大的那个工作点"——机制细节（EMA、nanosleep、chunk 级测量）在录音里**一句都没出现**。
- 补了**适用前提**："combine 流量已经不多了"（即 Mode 2/3 之后 combine 已移出 epilogue warp，剩下的主要是 dispatch），这条前提幻灯片只写在 "Used in mode 2 and 3" 里。
- 点出了**dispatch 是"自己给自己制造"的干扰源**：占满带宽的不是 combine 而是 dispatch，所以可以从发射端限速——这是幻灯片图上看不出来的一层归因。
- **ASR 存疑**
- "周总之前说的"（00:11:42）疑为"**刚才说的 / 前面说的**"（`(?)`），也可能是对某位同事的称呼，无法确认。
- "他也会产生白开水"（00:12:22）疑为"**产生队头阻塞 / 堵塞**"（`(?)`），对应 OCR 的 "head-of-line blocks"。
- "你知 vn 可是就是图片二传出他就是发一些 m 些的体系"（00:12:11）完全不通，只能判断为"**NVLink 被占满后影响到更底层的机制**"（`(?)`）。
- "高级度的尝试"（00:12:26）疑为"**更精细的（调度 / 限速）尝试**"（`(?)`）；"正常的（?）要一些被个"无法还原（`(?)`）。
- **EMA、nanosleep、chunk 往返测量、** `t0 / t1 / w = t1 - t0` 这些幻灯片上的机制**全部未被口播**。
- 本窗口后段 00:12:37 起已是**第 16 页的低精度 combine**。

第 16 页 | MegaMoE on Blackwell — Support for low-precision combine data types

> 拍摄 13:44:21 | 音频 1331 段偏移 793 | 讲者 张毅

- **口头要点**（实际讲述时间为 00:12:37–00:13:05，落在第 15 页的音频文件里）
- 最后一项：**为了减少 combine 的流量，做了一些低精度 combine 的尝试**（00:12:37–00:12:46）。
- 思路：**原来要用 bf16 数据做 combine，现在用 MXFP8 或者用 NVFP4 的数据来做 combine**（00:12:46–00:12:53）。
- 做法是**观察性能会不会符合要求**（00:12:53）。

- 结果判断：**这样做了之后可以极大地减少 combine 部分的流量**（00:12:56–00:12:59），**对计算非常有效**（00:13:02），**并且也能提供比较好的性能**（00:13:03）。
- 随后收尾整个 Blackwell / MegaMoE 部分：**"这个就是我们 Blackwell 部分 MegaMoE 的讲解，接下来是张毅"**（00:13:05–00:13:09）。
- **口头补充（幻灯片上没有的）**
- 幻灯片上的前提是 **"Precision doesn't regress on some models"**（只在部分模型上精度不退），口播把这一条压缩成了"**观察性能会不会符合要求**"一句，**没有给任何模型名或精度数值**——听者拿不到"在哪类模型上能退、在哪些不能"的信息。
- 现场过渡（录音 00:13:05–00:14:05，已属张毅的开场，幻灯片可能已翻页）：**张毅接手，感谢"方宇"的介绍**（00:13:13），自报"我是张毅"，接下来讲（**OP / overlap 相关 `(?)`）的 MoE 优化**（00:13:20）。他说**整体流程和（Megakernel）基本上一样，分 4 个部分**（00:13:28–00:13:35）：**dispatch 先把远端 GPU 的数据拉到本地 → 等到（flag/barrier）开了以后开始做 FC1 计算 → FC1 的 epilogue 把 SwiGLU 和 FP8 cast 融合进去 → 算完把数据写到（SMEM/local `(?)`），并置一个信号量，FC2 相关的 CTA 去等**（00:13:38–00:14:02）。这段是下一张（张毅的）slide 的内容。
- **ASR 存疑**
- "s16 数据做防范" = **bf16 数据做 combine**；"MS 批 8" = **MXFP8**；"a 第四" = **NVFP4**；"防范" 对应 **combine**（`(?)`，全篇统一如此）。
- 幻灯片上的**精度格式表（bf16 / FP8 E4M3 + E8M0 + block 32 / NVFP4 E2M1 + bf16 amax + block 16）**、以及关键工程结论 **"bf16 是内核内 REDG 路径唯一能原子加的格式，所以低精度 combine 需要 megakernel 之后一个独立的 topK 归约 kernel"** —— 在录音里**完全没有出现**，这是本段最大的口述/投影片落差。
- 照片时刻（13:44:21 ⇒ 00:13:13）已是**张毅的下一页**；本页真正讲解在 00:12:37–00:13:05，错位约 45 秒。
- 张毅那段里 "op 相关的 mm 的优化"（00:13:20）无法确定是 **OP / operator** 还是 **overlap**（`(?)`）；"要到自这个俱乐卜开了以后"（00:13:43）疑为"**等这个 flag / barrier 开了以后**"（`(?)`）；"写到这个呃罗 go my 里面"（00:13:58）疑为"**写到 SMEM / local memory 里面**"（`(?)`）。

这一段最突出的特点是**只讲"为什么改"，不讲"怎么算出来"**：孙方宇把 16 页压成一条因果链——fusion 不免费，于是矩阵侧换大 tile、通信侧用 batch-commit 与 async UREDGR 替掉昂贵的 memory order、combine 侧用 Mode 2/3 与相位交织把它移出 epilogue warp。幻灯片上最有含量的反而被跳过：页 8 的 32×4 起点、LLM 协作与 BFS 搜索、4 轮 10 条指令、页 16 的精度表与后置 topK kernel，全无口播。他补的是判断与实测：FP8 下 combine 能藏进 L2 epilogue、FP4 藏不住，FC1→flag→FC2 的依赖，以及"NVLink 利用率 80% 多到 95% 以上"。语速快、术语半英半中，照片普遍比口播滞后十几到几十秒。

第 17 页 | Overall Architecture: One Kernel Does Everything

> 拍摄 13:45:05 ｜ 音频 1331 段偏移 837 ｜ 讲者 张毅

- **口头要点**

- 讲者顺着数据流讲一遍整条链：FC1 算完把结果写到**本地 GMEM**（ASR 作"罗go my / 俄罗my"），随后给出**一个信号量**。
- FC2 相关的 CTA 在该信号量上**等待**，等到之后把 FC1 的结果 load 出来，开始 FC2 的矩阵乘计算。
- FC2 算完，在 **epilogue** 里调用 **combine** 相关逻辑，把数据**写回原始的（source）rank**。
- 这一整块被设计成**一个 kernel**（ASR"噢新的科能"），内部用 **warp specialization**，不同 warpgroup 负责不同工作。
- 最要紧的是 **epilogue 与它的 WGMMA 融合在同一组 warp 里**（ASR"IP服务 + 他们既然的aw"）。
- 融合的理由讲者当场说了：WGMMA 最后计算的结果是**写在寄存器里**的，因此没法拆成独立的一组，所以整体上是融合的。
- 一个 warpgroup 一般以 **4 个 warp 为一个单位**；实际使用中只有 **1–2 组** epilogue，且它们的**寄存器用量很大**。
- **口头补充（幻灯片上没有的）**
- "为什么 epilogue 不能独立成一个 warpgroup"的因果解释：accumulator 落在寄存器 → 必须同一个 WG 内完成 WGMMA 与 epilogue，这是架构页上只画结构、不写理由的部分。
- 部署经验值：实际只用 **1–2 组** epilogue WG（对应幻灯片 18 页的"×1-2"）。
- **ASR 存疑**
- "罗go my / 俄罗my"≈local memory / GMEM；"漏得出来"≈load 出来；"抗办"≈combine；"科能"≈kernel；"司出来a选"≈warp specialization。
- "IP服务"≈epilogue（warpgroup）(?)、"既然的aw"≈其 WGMMA(?)——按上下文定为 epilogue WG 与其 WGMMA。
- 片段开头"啊融合进去，然后FC计算完以后"是被截断的前一句，本页真正的起点在上一段末尾。

第 18 页 | Warp Specialization

> 拍摄 13:46:14 | 音频 1331 段偏移 906 | 讲者 张毅

- **口头要点**
- 先讲 epilogue 之外的分工：剩下的 warp 里 **TMA-A / TMA-B** 分别去 load **token** 与 **expert 的权重**。
- **scheduler warp** 负责**任务的分配**；**dispatch warp** 负责去拉**远端 GPU 的数据**。
- **token-back** 是**可选的**（ASR"中代理"）：可以单独给它 **4 个 warp** 专门做 push，也可以不开、把这些 warp 复用来做数据搬运。
- 设计时要同时兼顾**训练场景与推理场景**，两者对（通用）GEMM 的用量差别很大。
- 不做 swapAB 时，**M 维度一般放 expert / channel**；token 数较大（如大于 64）时一般能把 **WGMMA 打满**。
- 推理场景 token 数很小，极端情况**只有 8 个 token**，而 **WGMMA 的 M 是 64 起步**的。
- **口头补充（幻灯片上没有的）**
- "训练 vs 推理"这条设计约束是口头补的：同一份 warp 分工要覆盖两种迥异的工况。

- token-back 的取舍说明：**单独 4 warp 专用** 或 **复用现有 warp**，两种都可。
- **ASR 存疑**
- "4个w铺/沃克"≈warp；"派出网们"≈dispatch warps；"中代理"≈token-back(?)、"腐败 / 白"≈push(?)，语义按"可选、4 个 warp"定为 token-back。
- "1~2组的o部"≈1–2 组 warpgroup；"6次"疑为 **64**(?)；"打出线开始后"疑为 MMA/WGMMA 指令(?)。
- **对应关系错位**：本页窗口从 `[00:15:02]` 起，"训练/推理差异、M 是 64 起步"这段（`[00:16:02]`–`[00:16:41]`）讲的其实是**第 19 页 swapAB 的动机**，比照片早了一页。

第 19 页 | The swapAB Variant: the Cure for Small Token Counts

> 拍摄 13:47:42 | 音频 1331 段偏移 994 | 讲者 张毅

- **口头要点**
- 承上：极端只有 8 个 token 时，**有 50 多个位置（约 56 行）根本没参与计算**，padding 浪费很严重 —— 这就是要换策略的原因。
- swapAB 的原理：用 WGMMA 计算时 **M 维度必须是 64**，但 **N 维度只要是 8 的倍数**（8 / 16 / 32…）即可。
- swapAB 之后 **M 放 channel、N 放 token**；token 数少时可调用 **N 较小的 MMA 指令**，整体计算效率得到提高。
- MegaMoE 的 kernel 提供**两种量化方式**。其一是 **per-tensor**：与常见做法一致，activation 一个 scalar、**weight 每个 expert 单独一个 scale**，FC1/FC2 的 accumulation 都同样处理。
- 另一种是 **DeepGEMM 式的 blockwise**：每个 expert 内按 **128×128** 的块做量化。
- **口头补充（幻灯片上没有的）**
- 把 padding 的实际浪费量说成"50 多个位置没做计算"，是图上"56 of 64 wasted"的口头版。
- 明确把两种量化方式**并列**讲（per-tensor 与 blockwise 都可选），而不是只讲 blockwise。
- **ASR 存疑**
- "sab / swpab"≈swapAB；"开测后 / 开摄措施"≈WGMMA(?)；"托 / 圈了"≈token；"6次"疑为 64(?)。
- "不探色 / 不探测"≈per-tensor；"d个GS"≈DeepGEMM；"一个MP3546"(?)、"1128×128"≈128×128。
- 本页窗口（`[00:16:32]`–`[00:18:11]`）前半是第 19 页内容，后半 `[00:17:34]` 起已经在讲**第 20 页**的量化对照，照片滞后约一句。

第 20 页 | FP8 Quantization: per_tensor vs. blockwise

> 拍摄 13:49:13 | 音频 1331 段偏移 1085 | 讲者 张毅

- **口头要点**
- blockwise 下 **FC1 与 DeepGEMM 一样用 1×128**（per-token × K128）的块。
- **FC2 没有采用 1×128**，而是 **1×64**。
- 限制来自 FC1 的形状：FC1 的 tile 一般是 **64×128**，其中 **128 是 K 维度**。

- 因为把 **SwiGLU** 融进了这一步，**做完 SwiGLU 后 128 的维度变成 64**，所以在该维度上做量化只能是 **1×64**。
- 为什么不用 **1×256**（即 64×256 的 tile）：**寄存器压力**会比较大。
- 为什么不拆成两个 64×128：那样**两个 warpgroup 之间需要一次同步操作**，会影响耗时。
- 讲完量化转入 kernel 优化：**第一个优化 cooperative warpgroup** —— 用两组 warpgroup，第一组处理 **0–127** 的 token、第二组处理 **128–255**，两者**并行执行、结果互不影响**。
- **口头补充（幻灯片上没有的）**
- 1×64 这个选择是**被 SwiGLU 折叠后的 K 维决定的**（滑片上只列三种粒度，不解释为什么选 K64）。
- 两条被否掉的备选方案及其代价：**1×256 → 寄存器压力**；**两个 64×128 → 跨 WG 同步**。
- **ASR 存疑**
- "MC1 / MCU / MC二"≈FC1 / FC2；"思维接入"≈SwiGLU；"方形/太l"≈tile。
- "第三都是46 / 一个MP3546"疑为 **FP8**(?)；"fdv信的话"为本句起头的杂音(?)。"x那个0到第一百二十七个圈了"≈token 0–127。

第 21 页 | Optimization 1: Cooperative Warp-group Scheduling

> 拍摄 13:50:50 | 音频 1331 段偏移 1182 | 讲者 张毅

- **口头要点**
- 两个 warpgroup **同时、按序地**做 MMA；MMA 算完进入 **epilogue** 做 **SwiGLU / 量化 / store**。
- 好处：**单个 warpgroup 发 MMA 时排布不满**，两组同时发射可以让 MMA 在 tensor core 上排得比较满。
- 因此 **MMA 的耗时占比提升**，相应地 **epilogue 暴露出来的耗时占比减小**。
- **第二个优化 ping-pong** 与 cooperative 比较相似，也是 CUTLASS 里比较典型的一种做法。
- 区别在于依赖关系：cooperative 的两组**没有依赖**、时间上同时做 MMA；**ping-pong 强制**它们按某种依赖顺序推进 —— 第一个 WG 先做 MMA，做完进 epilogue，epilogue 做完再回去做 MMA。
- **口头补充（幻灯片上没有的）**
- 把收益讲成"占比的此消彼长"（MMA 占比升、epilogue 暴露占比降），比滑片上的 trade-off 表述更直白。
- **ASR 存疑**
- "iplg / iplog / IPL"≈epilogue；"ma / m后a / WMA"≈MMA / WGMMA；"顾客/我顾客/我本部的"≈warpgroup。
- "叫做通通"疑为 **ping-pong** 的音译吞音(?)；"和vt有的上面at"疑为"和 CUTLASS 里的 pattern 比较相似"(?)。
- "所以全服人啊还有n"一句不可辨(?)——按上下文当作"epilogue 里做 SwiGLU、量化、store"。

第 22 页 | Optimization 2: Task-tile Ping-pong

> 拍摄 13:52:16 | 音频 1331 段偏移 1268 | 讲者 张毅

- **口头要点**
- ping-pong 里**第二个 WG 的时间顺序与第一个正好相反**：第一个做 MMA 时第二个做 epilogue，第二个做 epilogue 时第一个做 MMA。
- 效果是**把 epilogue 部分完全隐藏掉**。
- 实际做优化时，**两种策略（cooperative / ping-pong）会按具体配置各跑一遍实测**，用实测性能来挑更好的那个。
- 接着讲 **TMA multicast**：像 CUTLASS 里那样，两个 CTA **以一个 cluster 的形式同时执行**。
- 具体例子：沿 **M 维度放两个 CTA**，第一个处理 **64 个（K）**、第二个处理**另外 64 个 K**，而**它们调用的 expert 可能是同一个**。
- 做 TMA load 时两个 CTA 加载**不同的 tile、但相同的 expert 权重** —— 这正是可以多播的地方。
- **口头补充（幻灯片上没有的）**
- 经验判断：cooperative 与 ping-pong **不预设谁更好，按配置实测择优**。
- **ASR 存疑**
- "毛巾ps / 猫t卡 / ca"≈multicast；"艾特的权重"≈expert 权重；"64个头k"疑为 64 个 K 行/列(?)，按"各 64 个 K"理解。
- "通通"再次出现，仍按 ping-pong 理解(?)；"探后"≈tile/tensor core(?)。
- 本页窗口 `[00:22:00]` 起已经在讲**第 23 页**的 multicast，照片滞后。

第 23 页 | Optimization 3: Multi-CTA CGA + TMA Multicast

> 拍摄 13:53:39 ｜ 音频 1331 段偏移 1351 ｜ 讲者 张毅

- **口头要点**
- 有了 multicast，**CTA0 与 CTA1 不必各自重复 load 同一份权重，广播出来即可**。
- 收益：**降低数据流量、降低整卡功耗、提升整体效率**。
- 除了一个方向，也可以换另一个方向做多播（ASR"在电脑上"）：两个 CTA 做同样大小的量、但在**不同 expert 维度**上切分，此时可以多播 **activation**，让它们读同一份。
- 转到 epilogue 写回：epilogue 做完 **SwiGLU 与量化**之后，**数据还在寄存器里**，必须写到 global memory 才能给后面的 FC2 或外部使用。
- 写回有两条路：**直接 STG 写** —— 所有数据传输时间**完全暴露在 epilogue 里**；**或者做一个 pipeline 结构**（下一页展开）。
- **口头补充（幻灯片上没有的）**
- 把 multicast 的收益链讲全：**省流量** → **降功耗** → **提效率**（滑片只写"L2/DRAM traffic halved"）。
- 指出多播可作用在**两种 operand** 上（权重多播 / activation 多播），按切分方向不同而不同。
- **ASR 存疑**
- "漏的"≈load；"头分 / 思头"≈channel 分片(?)；"第零个cta…60个头分"疑为 64(?)。
- "在电脑上上去做p的"疑为"在另一个方向（列方向）上做"(?)；"做pass"≈multicast/broadcast(?)。
- "IPO / IP"≈epilogue；"揖入"≈SwiGLU（与后文一致）；"ST机"≈STG。

第 24 页 | Optimization 4: Mainloop & Store Pipelines

> 拍摄 13:55:20 ｜ 音频 1331 段偏移 1452 ｜ 讲者 张毅

- **口头要点**
- pipeline 存储的做法：先把寄存器里的数据写到 **shared memory**，再由**另一个 warp 专门负责**把 SMEM 的数据用 TMA 写到 GMEM。
- 把数据切成**一个个小片**：一部分继续做 寄存器→SMEM，另一部分并行做 SMEM→GMEM。
- 这样 GMEM 方向的读写**可以被 寄存器↔SMEM 的读写掩盖掉**，降低 epilogue 写回耗时的暴露。
- MegaMoE 里把 **SwiGLU 这一整块都 fuse 进 FC1**。
- FC1 先做 MMA；因为 **gate 与 up 放在同一个 MMA 里算**，做完必须**拆开（deinterleave）**，然后做**反量化**（per-tensor 与 blockwise 两种）。
- 反量化到 **FP32**、乘上量化 scale 之后，接下来就可以做 **SwiGLU**。
- **口头补充（幻灯片上没有的）**
- 直接 STG 与 pipeline 两条路的**取舍结论**（前者把传输时间全暴露，后者能掩盖），是上一页没说完的补充。
- **ASR 存疑**
- "拍卖的结构"≈pipeline(?)；"share业的 memory / 需要的my / 学m位 / 是有的m本r"≈shared memory；"我m / 罗go"≈global memory。
- "例子和AP"≈gate 和 up；"不探索/不探色"≈per-tensor；"此微转入 / 活动"≈SwiGLU(?)、"量化一率"≈量化 scale；"卖票"≈MegaMoE(?)。
- **对应范围缺失**：本时间窗（`[00:24:09]`–`[00:25:50]`）只讲 store path 与 epilogue 融合链，**听不到 mainloop 段落**（`k_pipe_mmas=1` 的 WGMMA 流水、TMA producer `try_acquire` peek、228 KB SMEM 动态填满、FC1/FC2 共用 4 KiB ring）的口头对应表述，可能这部分未被展开或落在相邻片段。

第 25 页 | Optimization 5: All-Register Epilogue Fusion Chain

> 拍摄 13:56:54 ｜ 音频 1331 段偏移 1546 ｜ 讲者 张毅

- **口头要点**
- SwiGLU 之后要看**是否需要吸收 top-k 的权重**：可以在这里直接乘掉，也可以留到最后单独 combine 的时候再乘。
- 然后做 **FP8 量化**；量化完数据在寄存器里，用前面说的**两阶段方式**：先写 SMEM，再通过 TMA 写回 GMEM，供后面的 FC2 使用。
- 特别强调 **gate 与 up 的 interleave 间隔**：它**取决于（该指令下）寄存器里数据的排布。**
- 上面那张图在 **PTX 文档**里能查到，是 **M=64、N 为 8 的倍数**时的**寄存器排布。**
- 图中红 / 蓝 / 绿框之间的**模式是重复的**；一般由 **4 个 warp** 分别负责计算，红、蓝、绿、紫各对应一个 warp。
- 设计原则：**尽量避免 warp 之间的数据拷贝，以及一个 warp 内不同线程之间的数据交换。**

- **口头补充（幻灯片上没有的）**
- 讲清图例怎么读：颜色框 = 不同 warp、图样重复；以及"这张图出自 PTX 文档"这条出处信息。
- 给出 interleave 间隔的**判据是寄存器排布**（滑片只给结论 interleave=8）。
- **ASR 存疑**
- "get和up p / I特利 / 一推特"≈gate/up 与 interleave；"偷的一个位置"≈top-k 的权重(?)；"客户k"≈combine(?)。
- "tma特里面寄存器的数据的同步情况"疑为"该指令里寄存器的排布情况"(?)；"第零个现场/县城"≈线程(thread)。
- 本页窗口 `[00:25:42]`–`[00:27:14]` 前半仍是第 24 页融合链的收尾，`[00:26:24]` 起才进入 interleave（即第 26 页内容）。

第 26 页 | Why Gate/Up Interleave = 8: the WGMMA Register Layout

> 拍摄 13:58:15 ｜ 音频 1331 段偏移 1627 ｜ 讲者 张毅

- **口头要点**
- 为什么必须避免数据交换：会牵涉**同步操作，影响性能**。
- 红色框里一个 warp 内的数据排布是 **8 行 8 列**，右边蓝色框同样也是 **8 行 8 列**，**整个图样以 8×8 的形式重复**。
- 所以做 gate / up 的 interleave，最好的方式就是**按这 8 行 8 列去排**。
- 排完之后：**第 0 行第 0 列**的数据由**第 0 个线程**处理；**第 0 行第 8 列**（蓝色框最上面那个）**也是第 0 个线程**处理，只是落在**不同的寄存器**里。
- 于是同一个线程**同时拿到 gate 和 up 的数据**，在**同一线程的不同寄存器里直接配对**就能计算，不需要 shuffle / SMEM。
- 接着讲**第六个优化点**：EP 场景实测**只开 1 个 dispatch warp 就能打满带宽**；原设计 4 个太多，把另外 **3 个置为 idle**，只用 1 个，顺带把寄存器省出来。
- **口头补充（幻灯片上没有的）**
- **实测结论**（1 个 dispatch warp 就够）来自实测，不是推导；以及"剩下 3 个 idle"的具体处理方式。
- 早期的 warp 预算方案：一开始把**前 4 个或前 8 个 warp 留给 epilogue**，**后续 4 个**分别给 **TMA-A、TMA-B** 和**一个 scheduler**，另外还有**一个 idle 的 warp** 可以干别的事情 **(?)**。
- **ASR 存疑**
- "沃克 / 县城 / 现场"≈warp / thread；"计术器 / 技术器"≈寄存器。
- "只有一个例子开过了"中的"例子"≈warp(?)、"开过"≈kernel(?)，按"1 个 warp 即可打满带宽"理解；"HDM"疑为 **NVLink / HBM** 带宽(?)，与第 27 页 BDP 的说法更吻合。
- "前4个或前8个是用做就是软件"疑为"用作 epilogue"(?)；"第六个优化点"之前的编号在本窗口内有过一次跳号（`[00:28:29]`）。
- **对应关系错位**：本页照片（13:58:15）对应的窗口里，`[00:28:29]` 之后讲的已是**第 27 页 Optimization 6**；反过来说，第 26 页的 8×8 图样讲解实际发生在第 25 页照片的时间窗内。

第 27 页 | Optimization 6: A Single Active Dispatch Warp

> 拍摄 14:00:17 ｜ 音频 1331 段偏移 1749 ｜ 讲者 张毅

- **口头要点**
- 接着上一条：**4 个 dispatch warp 按实测结果优化成 1 个**，warp 数量**直接砍掉 4 个**。
- 于是整个 kernel 就可以用 **8 个或 12 个 warp** 来做（可用 warp 预算变宽）。
- **口头补充（幻灯片上没有的）**
- 砍掉的**绝对数量（4 个）**与由此得到的 warp 总数（8 或 12），是滑片上"1 / 2 / 4 可配"之外的口头账。
- **ASR 存疑**
- "例子派出我"≈dispatch warp；"我们的数量"≈warp 数量(?)、"卧铺"≈warp。
- "I豆的"≈idle。
- **该页音频窗口极短**：照片 14:00:17 与下一页 14:00:37 只隔 20 秒，片段覆盖 `[00:29:05]`–`[00:29:36]` 约 28 秒，本条 optimization 的正文大多落在上一页窗口与缺口里，与照片严格对应的讲述量很少，**对应关系可能错位**。

第 28 页 | Optimization 7: Folding the Producer Warpgroup + Register Refit

> 拍摄 14:00:37 ｜ 音频 1331 段偏移 1769 ｜ 讲者 张毅

- **口头要点**
- 少 4 个 warp 的**最大好处是寄存器压力缓解很多**。
- 对于较大的 tile（如 **64×128**），用原来的方式 **epilogue 没法开两组**去做（ping-pong / cooperative 都开不起来）**(?)**。
- 把 dispatch 那 4 个 warp 省出来之后，就能**成功开启（双 WG 的）配置**，整体性能有一定提升 **(?)**。
- **第八个优化点**（`fc1_done` 发布提前）：原顺序是 FC1 做完 epilogue → **R2S** → **TMA store** → **等 scheduler 分配下一个任务**（这里有一段空闲等待）→ 任务分到之后**才**去发布"FC1 已完成"的信号量。
- **口头补充（幻灯片上没有的）**
- 把"折叠 producer WG"的收益落到**寄存器压力**上（而不是只报 geomean +15.8%）。
- 明确指出旧布局下**大 tile 无法启用双 WG** 这一具体受限场景。
- **ASR 存疑**
- "去开两桌户的去做平台或者靠微信"疑为"开两组 WG 去做 ping-pong 或者 cooperative"(?)；"和t和呃天空"不可辨(?)。
- "第般个月发点"≈第八个优化点；"C一杠"≈`fc1_done`；"r图s"≈R2S；"TS"≈TMA store；"信号量的外"≈release/发布(?)。
- 本页窗口 `[00:29:27]` 起的 Opt 6/7 收益描述与照片（14:00:37）大致同步，但 `[00:30:07]` 起已进入 Optimization 8（第 29 页内容）。

第 29 页 | Optimization 8: Early fc1_done Publish

> 拍摄 14:01:52 ｜ 音频 1331 段偏移 1844 ｜ 讲者 张毅

- **口头要点**
- 为什么要提前：一个 FC2 tile 的数据来自**很多个 FC1 的 tile**，只要**某一个 FC1 tile 发布晚了**，就会把整个 FC2 的开始时间往后推。
- 所以希望**被依赖的 FC1 tile 一算完就立刻 arrive 这个信号量**，让下游的 FC2 尽早开始。
- 具体改法就是把发布位置前移：**FC1 做完先 release 一次"FC1 已完成"，然后再去等下一个任务。**
- **第九个优化点**：blockwise FC2 用**两组 accumulator**。
- 先讲原来的 blockwise 内层流程：做 WGMMA（半边）→ **wait 这个 MMA** → 把寄存器数据取出来、乘上 **1×128 / 128×128** 对应的 scale → 放到**单独的 FP32 寄存器** → 再做 blockwise 特有的 **promotion**。
- **口头补充（幻灯片上没有的）**
- FC1→FC2 的**多对一依赖关系**（一个 FC2 tile 吃多个 FC1 tile）是提前发布的真正动机，滑片只写"依赖链变短"。
- **ASR 存疑**
- "wet"≈wait；"第三二的寄存器"疑为 **FP32** 寄存器(?)；"合模型"≈promotion(?)；"太l / 太度"≈tile。
- "1×128或者128128"≈1×128 与 128×128（blockwise 的 activation / weight 粒度）。
- 本页窗口 `[00:30:41]` 起讲 Opt 8，与照片（14:01:52）基本同步；`[00:31:39]` 起进入 Opt 9，属第 30 页内容。

第 30 页 | Optimization 9: Blockwise FC2 Dual-Accumulator Overlap

> 拍摄 14:03:22 ｜ 音频 1331 段偏移 1934 ｜ 讲者 张毅

- **口头要点**
- 这个 promotion 操作**与 WGMMA 不并行、完全顺序**，所以做 promotion 时 **tensor core 完全空闲在那里。**
- 做完再进入**下一个 tile 的 MMA**，然后再 wait WGMMA，再做 promotion —— 结果是一部分 **promotion 的耗时被暴露出来。**
- 改法：用**两组 acc（两组 WGMMA）**，让它们处理**不同的 N 维度**，从而**互相不依赖**；第一组发射完立刻发射第二组。
- 然后先做第一组的 promotion —— 它**只等最前面那个 MMA，与第二个无关**，因此可以和第二个 MMA 的耗时重叠。
- 于是做第一组 promotion 的时候，**第二组的 MMA 已经在执行**，整体性能提高。
- 限制条件：两组 acc 会占更多寄存器；**开启条件是 N=64 时寄存器数量足够**才开。
- **口头补充（幻灯片上没有的）**
- "tensor core 完全空闲"这一现象描述是口头给的。
- 具体的**开启门限（N=64、寄存器够）**比滑片上"≤64 regs + single-WG 或 ≥224 epi regs"的说法更偏工况化。

- **ASR 存疑**
- "广模式 / 和模型 / promot型"≈promotion；"探后 / 太阳的w推线"≈tile / MMA 序列(?)；"项目"疑为 SM/CTA(?)，第 31 页同处亦作此解。
- "当它n等于4的时候"疑为 **N=64**(?)；"两组的这个 later"≈两组 accumulator(?)。

第 31 页 | Optimization 10: Scheduler Groups and the fc1_done Spin

> 拍摄 14:05:46 ｜ 音频 1331 段偏移 2078 ｜ 讲者 张毅

- **口头要点**
- **第十个优化点**：调度时 **group 的大小可以调**；MegaMoE 实际调度不是"所有 expert 的 FC1 全做完再统一做 FC2"。
- 实际顺序是：**两个 expert 的 FC1 → 这两个 expert 的 FC2 → 再两个 expert 的 FC1 → 再它们的 FC2**，依次循环。
- 所以要有一个"**一次塞进几个 expert**"的 group 大小来把控。
- group 太小的问题：expert0 的 FC1 做完后 expert1 的 FC1 开始，但 **expert1 的 FC1 tile 很少**、很快做完，于是马上进入该 expert 的 FC2。
- 此时**它的 FC1 可能还占着 SM 没做完**，于是这个 **FC2 的开始时间被拖后**。
- 办法是把 **group 打得更大一点**，一次塞进更多 expert，让 FC2 的开始**往后推一些**；真正开始时已经有足够的 FC1 计算完。
- 这个调整**要按实际的 expert 数量与 token 数量动态决定**。
- **第十一个优化点**：epilogue 写回 global 有两条路 —— **先到 SMEM 再走 TMA**，或**直接从寄存器 STG 写回**；选哪条看具体配置。
- **口头补充（幻灯片上没有的）**
- 把"spin 空转"的**成因讲成一条时间线**（小 expert 的 FC1 太快做完 → FC2 提前到场 → 依赖未就绪），而滑片只给 25% 这个数。
- 明确 group 大小**没有唯一解，需按 expert 数与 token 数动态调**。
- **ASR 存疑**
- "SP / xp / xper / XP"≈expert；"Google / 顾客性质的一个大小"疑为 **group 的大小**(?)；"微"疑为"次/轮"(?)、"头"疑为 token(?)、"ac"疑为 FC1 tile(?)、"项目"疑为 SM(?)、"驻留"可辨。
- 该页窗口 `[00:34:37]`–`[00:36:44]`，**开头 34:22–34:38 有约 16 秒缺口**（第 30 页片段止于 34:19），Opt 10 的第一句（"不是所有 expert 的 FC1 做完..."）可能落在这个缺口里——本页首条要点部分依据缺口前的第 30 页尾部与后续描述推得 **(?)**。

第 32 页 | Optimization 11: 16-byte FC2 Output Stores

> 拍摄 14:08:02 ｜ 音频 1331 段偏移 2214 ｜ 讲者 张毅

- **口头要点**

- 直接 STG 写回的问题：每个线程拿到的元素**可能是不连续的**，比如第 1、第 8、第 9 个，写到 GMEM 只能发 `STG.16` 这种窄指令。
- 更好的方法：用 **shuffle** 把不连续的数据**重新排成连续**，让一个线程去拿 0,1,2,...,8 这样连续的八个元素。
- 这样就能调用**位宽更高的** `STG.128` 写回 global，**指令数更少、运行效率更高**。
- 最后看整体性能：**通信侧调用 NVIDIA 的（NVSHMEM）**，**compute 侧是 CUTLASS 的 FP8 实现(?)**。
- 两张表分别对应 **per-tensor** 与 **blockwise**；token 数从最左端（8 起）一路增大，**加速比不断提升**。
- token 数较小时加速比约 **3%**；token 数较大时，per-tensor 可以到 **24%**。
- 为什么小 token 时加速比不高：**实测发现 token 数少时通信量小、计算量也小，计算侧基本不饱和。**
- **口头补充（幻灯片上没有的）**
- 把 16 字节写回的价值讲成"**指令数更少**"（滑片讲的是 sector 填充率，两者是同一件事的两种说法）。
- 给出 **3% / 24%** 两个端点数字，并**当场解释了小 token 端为什么涨不上去**（通信与计算都不饱和）——滑片没有这条归因。
- **ASR 存疑**
- "销售"≈shuffle；"stg16 / ST及128"≈`STG.16` / `STG.128`；"县城"≈线程。
- "e蔻的"疑为 NVSHMEM / NVIDIA 的通信库(?)、"包括a的和s"不可辨(?)；"卡拉斯的"≈CUTLASS；"非q的排名"疑为 FP8(?)；"QQ的个版本"疑为 DeepGEMM 的某个版本(?)、"橙色这条线"对应另一条对比线。
- "后客的大概工艺"疑为"MegaMoE 的大概收益/性能"(?)；"从最下面最左边的8~37度8"中的上界不可辨，疑为 **32768**(?)；"加速比过高"按上下文应为"**不高**"。
- 该片段 `[00:37:31]` 之后已转入**全片性能总结**（不属于本页滑片内容），第 32 页照片与讲述基本同步。

张毅这一段是"照着数据流讲一遍、再逐条讲为什么这么改"的工程讲法：口语化、中英术语混读，习惯先报现象（"50 多个位置没计算""tensor core 完全空闲""等调度白等一段"）再给手段；每条优化都交代原做法、暴露的瓶颈、收益，并反复附加限制条件（寄存器够不够、N 是多少、一次塞几个 expert）。幻灯片没写而口头补上的主要是动机与踩坑：epilogue 必须与 WGMMA 融在同一 WG，是因为结果落在寄存器里拆不开；FC2 量化取 1×64 而非 1×256 或两个 64×128，是怕寄存器压力与跨 WG 同步；dispatch 只留 1 个是实测 1>2>4 而非推导；ping-pong 与 cooperative 要按配置各跑一遍择优；双 acc 的门限是 N=64 且寄存器够；group 大小要按 expert 数与 token 数动态调；小 token 端加速比只有 3%，是因为通信与计算都不饱和。

第 33 页 | Token Sweep Results

> 拍摄 14:09:40 | 音频 1331 段偏移 2312 | 讲者 张毅

- **口头要点**
- 先解释「token 数量较少的时候加速比为什么不高」（转写为"过高"，按上下文应为"不高"）：token 少时实际计算量少，kernel 基本不饱和，时间主要花在 HBM 的数据读写上，**两种配置其实都是 memory bound**，所以整体加速比不明显。

- token 数量多起来之后加速才明显，原因是这边做得比较好的**通信与计算融合**，以及 **FC1（"ipl"）与 SwiGLU（"思维专入"）的融合**，减少了 HBM 的读写量。
- 对 blockwise（"外资"）那一侧，加速比会更明显：**从小 token 时的约 3%，到 token = 32768 时达到 72%**。
- 收尾总结：设计了一个**融合 kernel**，把 FC1、FC2 以及 SwiGLU 等全部融进同一个 kernel；同时支持训练场景与推理场景两种用法；同时支持 per-tensor（"我探索"）与 blockwise 两种配置；此外还加了很多优化方式提升整体性能。
- 最后一句交回主持：「上面就是我们这边的分享，看看大家有没有什么问题」。
- **口头补充（幻灯片上没有的）**
- 讲者给的是**机理层面的归因**而不是读图：小 token → 通信/launch 占比高、GEMM 不饱和 → 两种实现都被 HBM 读写钉住；大 token → GEMM 占比上升，单内核融合才显出优势。
- 口头的 72% 与图注里 1.72x 的标注互相印证，可确认它属于 blockwise 一侧（Per-Tensor 侧口头没给到这么高的数）。
- 明确点出「一个 kernel 同时服务训练与推理」是本方案的设计目标，不只是性能对比。
- **ASR 存疑**
- 「为什么通过数量较少的时候加速比过高」——"通过数量"应为 token 数量，"过高"应为"不高"(?)。
- 「我们的通信是开始可办一的」整句语义不清(?)，只能推测为"通信占比更主导"。
- 「开始扣」≈ kernel(?)、「ipl」≈ FC1(?)、「思维专入」≈ SwiGLU(?)、「外资」≈ blockwise(?)、「19号的科」疑为"一整个/fused kernel"(?)。
- 「从 to等于80或者3%到脱等于三十七六八的时候72%」——数字不确定：可能是 **0.8 / 8 / 80** 中的某个，"3%" 也可能属于这个片段；「三十七六八」应为 **32768**(?)。

第 34 页 | Summary

> 拍摄 14:11:07 | 音频 1331 段偏移 2399 | 讲者 张毅

- **口头要点**
- （本页拍摄时刻的录音已不在上一场的正文里）主持人开始收场：请观众举手提问，「我们今天还有这个有奖问答小组」，然后宣布「第一场的分享先这样」，进入下一环节。
- 主持人感谢**张毅和石光宇**两位老师，并概括上一场：**聚焦在 MoE 的算子实现和通信计算融合**。
- 主持人抛出下一场的问题域：把视角放到完整训练系统——当模型规模与序列长度变化时，**执行路径、显存使用和并行策略都需要做相应调整**；如何结合模型特性、软硬件约束与实测性能做出更有效的优化决策，是大规模训练中的重要问题。
- 介绍下一场：以 DeepSeek-V4 在 NVIDIA GPU 上的训练为例，讲基于 Megatron 的优化实践；有请**白洪潇**老师。
- 白洪潇开场自我介绍：「我是来自 NVIDIA Megatron 团队的白洪潇」，本 talk 以 DeepSeek-V4 为例，介绍如何用 Megatron 做 MoE 模型的大规模训练及实践。

- 行业背景：近几年很多前沿开源模型不断被放出并快速迭代，迭代中一个非常明显的趋势是**参数量与上下文长度都在变得越来越大、越来越长**——参数量基本能到 **1T 甚至 2T 以上**，上下文窗口基本都在 **1M（"一兆"）左右。**
- **口头补充（幻灯片上没有的）**
- 上一场是**两位讲者**（张毅 + 石光宇），不是单人；现场还设了有奖问答。
- 本页 Summary 幻灯片上的四条内容（one persistent kernel / 两个变体 two quantization modes / 30+ 优化 + token-bucket heuristics 冻结 sweep winner）实际上对应的是 **page 33 窗口末尾 39:19–39:44 那段口头总结**，讲者在拍这张 summary 时已经讲完了。
- **ASR 存疑**
- **时间窗与页面对应不上**：本页音频窗口 39:56–42:07 全是串场词、讲者交接与新讲者开场，没有任何 Summary 正文，Summary 内容落在上一页窗口内。
- 「麦窗库 / 麦克窗 / Mt克 / ma克t」≈ Megatron(?)、「tckv4 / dpc个v4 / DP CK v4 / DC个v4」≈ DeepSeek-V4(?)、「一兆」≈ 1M。
- 「dpc个v4在nvgpu上面的训练」难以确认是 "NVIDIA GPU" 还是 "Blackwell GPU"(?)。

5.2 第 2 场（白洪潇）：大规模训练实践，第 35–52 页

口头补充了大量“为什么这样排序优化”的判断依据，以及问答中对精度与显存代价的追问。

第 35 页 | Two Challenges Define Training at Scale

> 拍摄 14:13:18 ｜ 音频 1331 段偏移 2530 ｜ 讲者 白洪潇（NVIDIA Megatron 团队）

- **口头要点**
- 这么大规模的模型在实际训练中会带来非常明显的挑战，主要有两方面。
- **第一方面是"装得下"**：如此大的模型要真正放到 GPU 上并成功训起来本身就不容易——参数量很大、上下文长度很长，使**常驻显存**部分占用很多；同时显存与模型结构还会带来并行策略上的挑战，以及模型结构、模型功能与各种优化 feature 之间的**兼容性组合**问题。
- **第二方面是"跑得更快"**：把它跑起来之后还有第二个问题——模型在结构和"赛"的选择上已经带来明显压力，在这个基础上还要做**状态与 offload 的优化**、运行时间与显存消耗之间的平衡，以及针对具体 workload 的优化，这些相对更复杂。
- 取材说明：后面这个 talk 主要基于 **DeepSeek-V4-Pro 的开源模型结构** + 在 **Megatron** 中的实现，实测量在 **NVIDIA Blackwell GPU** 上做。
- 承接上一段，做了一系列性能优化，先分成几点讲，第一点是整体的 **runtime overhead 以及相关的 memory trade-off**。
- **口头补充（幻灯片上没有的）**
- 讲者把「能训起来」和「训得快」明确拆成两条独立线索，并指出**两条都源自模型自身的规模与结构**，而不是纯工程问题。

- 「feature 之间的兼容性组合」是他从实际踩坑中体感到的复杂度，幻灯片只写了一个词 feature compatibility，口头把它展开成了"很多个优化功能两两之间的组合"。
- **ASR 存疑**
- 「非ature」≈ feature(?)、「work漏」≈ workload(?)、「状态oo尔态的优化」≈ 状态与 offload(?)、「专扣」≈ Megatron Core(?)、「不re报 / nvd不来报GPU」≈ Blackwell GPU(?)。
- 「它的呃赛的选择」——"赛"疑为 sequence/策略/配置的选择，无法确定(?)。

第 36 页 | One Playbook, Two Workloads

> 拍摄 14:15:51 | 音频 1331 段偏移 2683 | 讲者 白洪潇（NVIDIA Megatron 团队）

- **口头要点**
- 把这些优化组合成**一整个优化组合（playbook）**，应用到两个比较重要的 workload 上。
- 这些优化可以带来明显的性能提升，**但两个 workload 上的提升幅度不太一样**，后面会具体介绍。
- 转到模型 overview：左边这张图是从 DeepSeek 论文里拿出来的，整个模型基本还是一个比较标准的 Transformer 结构；**MoE 部分跟之前的结构比较类似，没有太大变化**。
- **attention 部分变化相对比较大**，因为它是一个**异构（"易购"）的 CSA 加 HCA 的结构**。
- 虽然说它比较复杂，但跟后面一些同类模型（如"4.1 之类"）比，它还算相对规则，**没有复杂的跨层状态**这类东西。
- 另一个变化是**在残差连接的地方还有一处做替换**（对应下一页图里的 Pre-Block Mixing / Residual Mixing / Post-Block Mixing），不过这块后面不再详细讨论。
- **口头补充（幻灯片上没有的）**
- 讲者主动给了一个横向参照：与后续版本/同类架构相比，DeepSeek-V4-Pro 的异构 attention **没有跨层状态**，这直接决定了它对并行与显存更友好——这是幻灯片上没有的判断。
- 讲者自己划了重点范围：Mixing/MTP 一带不展开，把时间留给 attention 与 MoE。
- **ASR 存疑**
- 本页幻灯片上的两个关键数字 **+59% @64K、+32% @1M** 在录音窗口（44:43–45:47）里**没有被念出来**，只有"提升明显、两个 workload 不一样"这种定性表述，数字以幻灯片为准(?)
- 「易购」≈ 异构(?)、「MAC都做替换」疑为 mixing（或 MTP）之类的组件替换(?)、「4.1之类的」具体指哪一代模型无法确认(?)。

第 37 页 | DeepSeek-V4-Pro Overview

> 拍摄 14:16:47 | 音频 1331 段偏移 2739 | 讲者 白洪潇（NVIDIA Megatron 团队）

- **口头要点**
- 整个模型大约 **1.6T 参数量**；对单个 token 来说，激活值大约 **49B**（转写为"49亿"）。
- 按 DeepSeek 官方的例子，预训练会分为几个阶段：先在 **4K、16K 以及较短的 64K** 上做 **dense** 的初步预训练，后面切换到 **64K 与 1M** 的 **sparse** 训练；中间还有一小段过渡阶段。

- 整个训练过程中**占比比较大的部分**是在 64K 到 1M 的 sparse 阶段，所以本 talk 的优化**主要集中在这两个配置（64K 与 1M）上。**
- 这条 workload 其实和我们**后训练**中使用的一些流程比较接近，所以这些内容也可以牵引到类似的后训练场景中。
- 前面提到的 4 个优化方向可以**和模型结构一一对应**：模型参数量很大 → 引入内存压力以及运行时的 overhead；接下来是里面比较大的两个组件——**attention** 和 **MoE**；最后是**长序列**带来的额外挑战。
- 长序列最主要的影响在**这里的 Indexer 部分**，因为 **Indexer 的计算量跟序列长度的平方成正比。**
- **口头补充（幻灯片上没有的）**
- 「这条 workload 与后训练接近、可迁移到后训练」是讲者补充的幻灯片外判断。
- Indexer 的**平方复杂度**是他用来解释"为什么长序列要单独作为一个优化方向"的机制说明，幻灯片只写了 causal long context 一句。
- **ASR 存疑**
- **「49亿」与幻灯片 `49B` 不符**：49 亿 = 4.9B，按幻灯片与量级应为 **49B**（约 490 亿）(?)
- 「s ps训练」≈ sparse 训练(?)、「中间还有一小段类似我妈的阶阶段」语义不清，疑为退火/中期训练阶段(?)、「拓客 / 拓格」≈ talk(?)、「单位的压力」疑为"内存的压力"(?)、「o尔看」≈ overhead(?)。

第 38 页 | Model Features Drive Optimizations

> 拍摄 14:19:29 ｜ 音频 1331 段偏移 2901 ｜ 讲者 白洪潇（NVIDIA Megatron 团队）

- **口头要点**
- 先做了若干技术组件的初步支持与相关算子开发，例如 CSA、HCA、DeepSeekMoE（转写为"dsc啦 hcdamac 等等"）等。
- 初步支持做完之后，就要考虑怎么把整个模型**完整地放到 GPU 上让它跑起来。**
- 第一个问题是：**怎么确定一个并行策略把它放进去**——因为我们知道单张 GPU 肯定没办法把整个模型跑起来。
- 做这一步时一般先用一些**简单的办法做估算**：模型的参数量是固定的；如果优化器确定，那**优化器这部分占用的显存也相对是一个固定值。**
- **口头补充（幻灯片上没有的）**
- 讲者给的是"**先粗估、再细调**"的方法论：把参数量、优化器状态当常数先算出一个下限，精度等因素先按固定值处理（下一页展开）。
- 他明确说注意力/混合精度这类因素在初步估算阶段可以先忽略，这是经验判断而非幻灯片内容。
- **ASR 存疑**
- 「像dsc啦 hcdamac等等」具体缩写无法确认，按上下文应为 CSA / HCA / DeepSeekMoE 一类组件(?)
- 「课上的一开发」≈ 算子开发(?)、「或者是他一些对支持」语义不清(?)。

第 39 页 | Choose the Parallel Layout（General Guidance on Parallel Tuning）

> 拍摄 14:20:05 | 音频 1331 段偏移 2937 | 讲者 白洪潇（NVIDIA Megatron 团队）

- **口头要点**
- 估算时，像精度这类因素可能会对这一部分有影响，但**初步估算时可以按一个固定值来考虑。**
- 用「一定要常驻显存的这部分」加上**显存大小**，同时考虑**为后面的 activation 与 buffer 预留的空间。**
- 这样基本就可以估算出**最少需要多少张 GPU** 才能把整个训练跑起来。
- **口头补充（幻灯片上没有的）**
- 讲者把估算顺序讲得很清楚：先算常驻静态部分 → 再看单卡容量 → **必须先扣掉 activation/buffer 的预留**，剩下的才决定卡数。这一步的"预留"意识是后面长上下文策略的前提。
- **ASR 存疑**
- 「为后面的一些I TV啦或者一些发货预留的这样一些空间」——「I TV」≈ activation(?)、「发货」疑为 buffer(?)
- 本页录音窗口与第 38 页大面积重叠（48:57–49:05 重复），是相邻两页同段录音切窗造成的。

第 40 页 | Choose the Parallel Layout（同页重拍）

> 拍摄 14:20:38 | 音频 1331 段偏移 2970 | 讲者 白洪潇（NVIDIA Megatron 团队）

- **口头要点**
- 基本确定需要多少张 GPU 之后，就把这些 GPU **分到一些通信组里面**，做并行的实现。
- 选并行策略有一些相对通用的方法（但不是绝对）：对 MoE 这样的模型，**优先倾向于用 EP 把 expert 拆开。**
- **如果 EP 不够，一般会选择 PP** 来做跨 domain 的拆分；**EP 和 PP 都不够时才考虑 TP**，因为 **TP 本身带来的 overhead 相对比前两个更大。**
- 考虑完静态部分之后，下一步是 **activation**——这部分跟**序列长度直接相关**；对 64K、1M 这种长上下文，显存压力就会比较大，所以一般通过 **CP（context parallel）在序列长度方向做切分**，使每个 rank 上处理的 token 数量不至于太多。
- 除了模型本身的特征，还要考虑**硬件限制**：硬件的显存以及通信带宽；一般来说会把 **EP、DP、CP 这种通信量比较大的组放在带宽比较大的域**内（例如 NVLink domain）。
- **口头补充（幻灯片上没有的）**
- 「先 EP、再 PP、最后 TP」的优先级 + 理由（TP overhead 更大）是讲者口头给的经验判断。
- 他把「通信量大的并行组要放进高带宽域」作为一条独立原则提出，比幻灯片上的 topology 一句更具体（口头点了 EP/DP/CP 三个例子）。
- **ASR 存疑**
- 「看莱斯开的」无法确认，疑为 context parallel 的序列切分（或有 sequence parallel 之意）(?)
- 「目前比如是说像一个国内」疑为"像 NVLink domain"(?)，句子被窗口截断。

- **幻灯片上的 256-GPU、PP4、EP64 三个数字在录音窗口内没有听到**——窗口在 51:38 被切断，下一页开头直接讲"到 1M 我们选 CP64"，中间那段（51:38–53:01）缺失(?)。

第 41 页 | Context Length Drives Memory Policy

> 拍摄 14:24:09 | 音频 1331 段偏移 3181 | 讲者 白洪潇（NVIDIA Megatron 团队）

- **口头要点**
- 到了 1M 的情况下，这里选择的是 **CP64**。
- 原因：**CP 如果再加得更大，CP 这个 group 的带宽会变得很低**，会带来非常明显的运行 overhead。
- 但这样带来的问题是：**每个 rank 要处理的 token 数量会更多，达到 16K**，是前面那个 case 的**大概 4 倍**，所以这里会带来很明显的压力。
- 因此这两种配置在初步测试时**会选择不同的显存（memory）策略**。
- 64K 的情况：用一些**轻量级的、小范围的 recompute 或者 offload** 来处理。
- 1M 的配置：基本上就**需要依赖全范围的 recompute**，使整个 activation 尽量不要长期驻留在显存里。
- 这就是初步设计整体运行方案时，对运行与策略的组合考虑。
- **口头补充（幻灯片上没有的）**
- 讲者把 CP 的选择讲成**「带宽 / overhead ↔ 每 rank token 数」的权衡**，而不是"切得越细越好"——这是幻灯片没有的推理链。
- 他口头的 16K 与"4 倍"隐含了 64K 侧是 **4,096 tokens/rank**（与幻灯片一致），但 4,096 这个数本身没念。
- **ASR 存疑**
- 「达到了16k」应指 **16,384 tokens/rank**(?)、「慢慢策略」≈ memory 策略(?)、「人个ute」≈ recompute(?)、「以兆配置」≈ 1M 配置(?)、「不同的慢慢的策略」重复转写噪音。
- **64K 侧的 CP16 与 4,096 tokens/rank 在录音中没有被念出**，仅有 1M 侧的 CP64/16K 与"4 倍"比值(?)。

第 42 页 | Context Length Drives Memory Policy（同页重拍）

> 拍摄 14:25:09 | 音频 1331 段偏移 3241 | 讲者 白洪潇（NVIDIA Megatron 团队）

- **口头要点**
- 在确定怎么把模型放到 GPU 上、并且成功逐步跑起来之后，**下一步会发现它其实跑的性能并不好**——这就回到了最开始提出的第二个问题：怎么让它跑得更快一些。
- 具体为什么跑得慢，**没有办法靠猜测给出答案**，一般要通过 **profile** 来确定问题出在什么地方。
- 常用工具：像 **NVIDIA Nsight Systems（"n家的 system"）** 以及其他 profiler，基本都可以用来做这样的工作。
- 这里截了**两张 Nsight Systems 的截图**做对照。
- 从截图里可以看到一个**非常明显的问题**（下一页展开，先看上面 64K 的情况）。

- **口头补充（幻灯片上没有的）**
- 讲者明确否定了"凭经验猜瓶颈"的做法，把 profile 确立为一切优化的前置步骤——这是后面 STEP 1/2/3 的共同方法论基础。
- 他还提到 profiler 不止一个，除 Nsight Systems 之外还有别的工具可选用。
- **ASR 存疑**
- 「n家的system啊太多是破坏了」≈ Nsight Systems 以及一些其它 profiler(?)、「两个分在system的截图」≈ 两张 Nsight Systems 截图(?)
- 本页与第 43 页录音高度重叠（55:05–55:18 同一段），页面文字偏少，窗口内没有对应具体讲稿。

第 43 页 | The Starting Points

> 拍摄 14:26:18 ｜ 音频 1331 段偏移 3310 ｜ 讲者 白洪潇（NVIDIA Megatron 团队）

- **口头要点**
- 看上面 **64K** 的情况：整个运行的时候会发现里面有**非常多的小块 kernel**，而且**整体的 GPU 利用率非常低。**
- 这些空闲的地方基本上是**集中、连续**出现的（一段一段的空隙）。
- 这种情况下对整体性能影响非常大，所以这是需要第一个去解决的问题。
- 对 **GPU idle** 这类问题（以及 CP 相关的 overhead），一般有两个常用的解决方法：一个是做 **kernel fusion（算子融合）**，另一个是用 **CUDA Graphs** 来减少 kernel launch 或 CPU 端的 overhead。
- kernel fusion 相对是一个比较直接的方法：**基本不会带来太多负面影响**，大部分情况都会有性能（以及配置选择上）的收益。
- **口头补充（幻灯片上没有的）**
- 他先把现象归因定性为「小块 kernel + 连续空洞 → GPU idle」，这是把 timeline 截图读成结论的过程，幻灯片只有两张图。
- 「融合基本没有负面影响」是他给的经验判断，同时预告了它的局限（下一页：复杂结构不好融）。
- 这一页的动机是后面 STEP 1（fusion / CUDA Graphs）的直接来源。
- **ASR 存疑**
- 「基本上都是集集中在挨着的状态」具体所指不明，疑为"连续集中的一段一段"(?)
- 「像 gpui豆啦或者是像CP ohead的这些问题」——「gpui豆」≈ GPU idle(?)，「CP ohead」疑为 kernel launch overhead(?)
- 幻灯片上的 **4K sparse proxy、先验证 Kernel fusions / packed sequence / CUDA Graphs、两段 profile 都在 150ms 窗口内** 这些点，录音窗口内**均未明确念出**(?)

第 44 页 | STEP 1: Runtime Overhead and Memory Trade-offs

> 拍摄 14:27:35 ｜ 音频 1331 段偏移 3387 ｜ 讲者 白洪潇（NVIDIA Megatron 团队）

- **口头要点**
- fusion 比较麻烦的地方：对复杂结构来说不太好融——例如 **MoE 部分里面有很多不同"赛"（不同形态）的 kernel，里面还有大 kernel、或者有些 scale 之类的操作**，不一定能轻松融起来。
- 所以实际执行时，还是会**有很多 kernel 留在那里**；想进一步优化这些 kernel 之间的 overhead，**CUDA Graphs 就是一个比较好的方案。**
- CUDA Graphs 在满足条件时基本是**开箱即用**的，它并不关心你里面具体跑的是什么，只不过使用时需要满足一些条件。
- **最主要条件：整个 capture 的范围之内必须是静态的**，不能在里面有动态的东西，否则会导致 capture 失败。
- 另一个**明显的负面问题**：启用后**显存增长会比较明显**（后面会再看）。
- 这一步的 fusion 主要涉及比较简单的部分：一些算子内部（例如 attention 相关的）在最开始实现时其实已经做了一些初步融合，但实测时发现里面还有很多小坑——比如 **THD 输入的小坑**、或者一些简单 attention 里 y 的计算、或 attention 相关的操作，于是又对这些操作做了一轮补充融合。
- 总结 CUDA Graphs 的定位：它是一个**选择性策略**——既可以带来非常明显的加速，但与此同时会使得**显存消耗非常明显地增加。**
- **口头补充（幻灯片上没有的）**
- 讲者把"CUDA Graphs 开箱即用 vs 条件苛刻"与"速度快 vs 吃显存"当成一对权衡来讲，并透露后面还会回到显存这件事。
- **THD（packed sequence）输入的小坑**、简单 attention 里 y 的计算等，是他在实测中踩到的具体细节，幻灯片上完全没有。
- 窗口末尾被截断在"因为前面其实已经介绍我在整个一兆的这样的一个 case 配…"，**1M 下对 CUDA Graphs 显存代价的取舍没讲完**（58:35–61:01 音频缺失）。
- **ASR 存疑**
- 「葡萄话 / 葡萄 / 不萄谱 / 不萄谱的带来」≈ CUDA Graphs(?)、「丢人 / 流人 / 修人 / 推人 / 救起来」≈ fusion(?)、「开ture失败」≈ capture 失败(?)
- 「thd有哪些小坑洞了」疑为 THD（packed sequence 布局）输入上的坑(?)、「完美相关的操作」疑为 attention 相关的操作(?)、「一些简单的案里面的y的计算」语义不清(?)
- 缺失段（58:35–61:01）里应该还讲了 **1M case 的显存增量与是否启用 CUDA Graphs**，此段无转写(?)。

第 45 页 | Where the Step Time Goes（Layer time decomposition）

> 拍摄 14:32:15 ｜ 音频 1331 段偏移 3667 ｜ 讲者 白洪潇（NVIDIA Megatron 团队）

- **口头要点**
- 先看 MoE：MoE 部分基本可以分为**两大块——MoE 计算**，以及 **MoE 带来的 EP 通信。**
- attention 部分里，占比较大的有**三个部分**：**蓝色是 Indexer 部分**，**紫色是里面的 sparse attention（"s8so"，疑指 CSA）部分**，**橙色是最后的 Group-O projection 部分。**

- 其他部分相对很难找出占比非常大的，所以**优化就集中在这几个部分**。
- 根据前面那个 profile，先来看 attention：attention 部分能看到里面有**三个非常明显的组件**，它们在整个模型里占比还是比较明显的。
- 因此做优化时就希望先看这几个组件怎么优化——**优化这些组件一般会带来更好的整体性能提升**。
- 首先做 sparse attention 部分：通过看它的实现之后，发现之前的实现**其实没有很好地利用 sparse 特性**。
- **口头补充（幻灯片上没有的）**
- 讲者是**用颜色讲图**（蓝 = Indexer、紫 = sparse attention/CSA、橙 = Group-O Proj），把堆叠柱状图直接翻译成优化优先级——这是幻灯片之外的信息增量。
- 「其他部分占比都不大」是他判断"只打这三个组件就够"的依据。
- 4K 在幻灯片上标为 sparse proxy 对照，口头没有再强调。
- **ASR 存疑**
- 「s8so」具体指 **CSA** 还是就是 sparse attention 本身，无法确认(?)
- 「group口o / group o 的一个 ption」≈ Group-O projection(?)、「通过看它的实践之后」疑为"它的实现"(?)、「这是实践」同(?)
- **幻灯片上 4K/64K/1M 三根柱子的具体百分比，录音里一个都没有念**，只有颜色与定性排序(?)。

第 46 页 | STEP 2: Optimize the Hybrid Attention

> 拍摄 14:33:36 | 音频 1331 段偏移 3748 | 讲者 白洪潇（NVIDIA Megatron 团队）

- **口头要点**
- **Sparse attention**：在 sparse 处理的时候，中间会有一些**无效的部分，用 -1 来填充**；实际执行时发现**这些无效部分会被额外处理**，从而带来 overhead。
- 优化做法：把它做一个**紧凑化（"精英化"）的排布**，这样执行过程中就**只会处理这部分有效的内容**，不会浪费时间处理额外的无效空白信息。
- **Indexer 第一个优化**：Indexer 其实是 QK 之间的一个乘法，做乘法时有一个 **causal mask**，也就是说**有一半元素其实是没有被用到的**；这种情况下就只处理那一半有效的部分，另一半无效的忽略掉，可以**避免很多无效的显存读写**。
- **Indexer 第二个优化**：QK 的计算量**跟序列长度的平方成正比**，在两个长序列配置下这一部分计算量会非常大，所以**用低精度来做优化会带来非常明显的收益**——把 QK 的实现做了量化：**输入之后先量化到 FP8，整个计算用 FP8 做，输出仍然还是 BF16**。
- **Group-O**：最初的实现用了按 group 拆分的做法，使得整个过程中**每一个 group 都是单独处理/单独 launch**，kernel 又多又碎。
- **口头补充（幻灯片上没有的）**
- 三条优化共享同一个思路：**把无效工作从 kernel 里彻底拿掉**（无效 slot、causal mask 的一半、碎片化的小 GEMM launch）——这是他口头反复强调的"只算有效部分"。

- **FP8 只作用在 QK 内部、输出回 BF16** 这一精度边界是明确的工程取舍，幻灯片只有 "Q·K quantized to FP8" 一句。
- 他明确指出 causal mask 导致"一半元素不用算"是 indexer 侧最大的一块白干，与幻灯片 `Skip the invalid part from causal mask` 对应但更具体。
- **ASR 存疑**
- 「s843神部分」≈ sparse attention 部分(?)、「精英化的一个排布」疑为"紧凑化/左对齐的重排"(?)、「马斯克」≈ mask(?)、「q别镜」≈ QK(?)
- 「最初实现的话它其实是利用了s萨」——「s萨」可能是 split/slice 之类实现方式，无法确认(?)
- 窗口末尾被截断在"每一个 group 口..."，**幻灯片上的 G=8、`G=8 small GEMMs` 与 batched GEMM 一次 launch** 在窗口内没有说完（64:36–66:35 音频缺失）(?)。

第 47 页 | STEP 2: Optimize the Hybrid Attention（效果页）

> 拍摄 14:37:43 ｜ 音频 1331 段偏移 3995 ｜ 讲者 白洪潇（NVIDIA Megatron 团队）

- **口头要点**
- 介绍完 attention 的部分之后，另一部分比较重要就是 **MoE**。
- MoE 这个结构出来已经很长时间了，而且一直在有一些更新，但**变化相对没有那么明显**，也就是说它有**很多优化其实已经相对比较成熟**。
- 计算部分：已经有**非常高效的 grouped GEMM（"服务何的进步"）**实现。
- 通信部分：也有很多高效的通信库，比如 **DeepEP 一类（"dpp 还会一批"）**。
- 也就是说对计算以及通信来说，**它们已经优化到相对比较好的状态**；想进一步优化的话**比较难，收益也比较有限**。
- **口头补充（幻灯片上没有的）**
- 讲者的判断顺序是：MoE 结构稳定 → 单点优化已成熟 → 于是**优化空间只能在"计算与通信之间"找**，这直接引出下一页的 EP 通信 overlap。
- 他把"grouped GEMM 已经很高效"当作 MoE 计算侧的基线，因此 STEP 3 不再碰计算内核本身。
- **ASR 存疑**
- **时间窗错位（重要）**：本页窗口 66:35–67:26 已经在讲 MoE 的引子，而幻灯片上的模块级加速 **25%–30%（Compact sparse attention）、25%–35%（Compact FP8 Indexer）、50%–60%（Group-O batched GEMM）** 与端到端 **+4.6%@64K、+9.0%@1M**，是在 **64:36–66:35 之间**念的，落在窗口前、未被抓到(?)。数字仅以幻灯片为准。
- 「服务何的进步」≈ grouped GEMM(?)、「dpp 还会一批 d个一批」疑为 DeepEP(?)、「收益会比较明显」从上下文疑为"收益比较有限/不明显"(?)。
- 幻灯片上 "The indexer accounts for a larger share of step time at 1M" 这句解释性观察，录音中未直接听到。

第 48 页 | STEP 3: Optimize the MoE

> 拍摄 14:38:26 ｜ 音频 1331 段偏移 4038 ｜ 讲者 白洪潇（NVIDIA Megatron 团队）

- **口头要点**
- 我们在前面看 profile 的时候，可以明显看到**整个 EP 通信占比还是挺大的。**
- 通信过程中**并没有实际去做有效的计算**，所以**这段时间很多时候可以认为是浪费掉的。**
- 因此**如果能把通信这部分时间隐藏起来，对 MoE 整个部分就会带来非常明显的收益。**
- 想隐藏 EP 通信，目前主要有**两个方式**。第一个是**跨 batch 的 EP overlap**：对不同的 microbatch，用**一个 microbatch 的通信去盖另一个 microbatch 的计算**;例如之前 **DeepSeek-V3 里提出的多 microbatch 流水（"多o派克"）**，或者 Megatron 里基于某种调度方式做的跨 EP overlap。
- 这种方式相对**比较适用于 EP 带宽较低（"在带宽情情况下"）的 case**；但它带来的问题是**显存（memory）开销会比较大**——做 overlap 的时候往往会有**更多的 microbatch 驻留在进程当中**，而且有些实现可能会使得**参数也留在线程/显存里。**
- 另一个方案就是前面介绍过的 **MoE megakernel** 方案：相对而言它是做了**细粒度（"细节度"）的 overlap**，把整个计算的东西**聚集在一起**；这个实现在领域内相对**更容易实现一些**，而且对显存**更友好。**
- **口头补充（幻灯片上没有的）**
- 讲者把两个方案讲成互补关系：**跨 batch 适合跨/低带宽 EP 场景、代价是显存；megakernel 适合域内细粒度重叠、更容易实现也更省显存**——这层取舍判断比幻灯片的两条列表更细。
- 「通信时间很多是纯浪费」是他给 STEP 3 的动机定性，也解释了为什么收益上限比 STEP 2 高。
- 他点出了显存变差的两个具体来源：**多 micro-batch 常驻** + **有些实现连参数都不释放。**
- **ASR 存疑**
- 「跨派去的一批o来p」≈ 跨 microbatch 的 EP overlap(?)、「多o派克」疑为 **DualPipe** 或多 microbatch pipeline(?)、「光方光l方闭APP」完全无法确认，疑为 Megatron 的 flexible/interleaved 调度(?)
- 「对人们那个总院来说」语义不清(?)、「细节度的o拉p」≈ 细粒度 overlap(?)、「慢慢包括看的会比较大一些」≈ memory 开销比较大(?)
- 幻灯片上的 **「MoE 部分里约 30%–50% 是 EP communication」** 与 **「block size 由 nsys report 里提取的 silicon time 决定」** 两句，录音中没有明确念出(?)
- 出处注 `The graph below is from the MoK(?) blog from Cursor.` 在口语中未提及，无法核对 MoK/MoE(?)。

这一段（33–48 页）的口头讲解最大特点是**"现象 → 机理 → 手段"三层推进，且几乎每一页都把幻灯片上的结论数字换成机制解释。**上一场收尾时，讲者不谈 1.35x 的峰值，而是先解释"小 token 为什么两种配置都被 HBM 读写钉住、都是 memory bound"，再补出幻灯片上没有的细粒度数字（从小 token 的约 3% 到 32768 tokens 的 72%），最后才回到"一个 kernel 融掉 FC1/FC2/SwiGLU，同时吃训练与推理"。白洪潇部分同样如此：他反复用"先粗估静态显存 → 预留 activation/buffer → 定 EP → PP → TP 的优先级 → 用 CP 切序列"这条工程顺序把 39–42 页的四张结构图串成一条可复现的决策链，并主动补上幻灯片外的经验判断

（TP overhead 比 EP/PP 大、CP 加大会掉带宽、异构 attention 没有跨层状态所以更友好、这条 workload 可迁移到后训练）。

第二个特点是**极其坦诚地讲代价与踩坑**：CUDA Graphs 他讲成"开箱即用但 capture 区间必须静态、显存增长明显"的选择性策略，融合则坦白"MoE 里 kernel 形态太多、有 scale 之类，融不动就还剩一堆 kernel"，还具体点了 THD 输入、简单 attention 里 y 计算这类小坑。

第三是**颜色与占比驱动的优化排序**：45 页他直接用蓝色 Indexer / 紫色 sparse attention / 橙色 Group-O 三色把堆叠柱状图翻译成 STEP 2 的三个抓手，并说明"其它部分找不到大占比，所以只打这三处"。

需要注意转写质量明显偏低（Megatron、CUDA Graphs、fusion、DeepSeek-V4、CSA、Indexer、Group-O 都严重音译失真），且 44→45、46→47 两处各缺约两分钟音频，导致 47 页的成绩数字与 48 页「30%–50% 是 EP 通信」等关键量化只能以幻灯片为准。

第 49 页 | STEP 3: Optimize the MoE

> 拍摄 14:41:59 ｜ 音频 1331 段偏移 4251 ｜ 讲者 白洪潇（NVIDIA Megatron 团队）

- **口头要点**
- MoK 这套方案可以在「比较少的额外开销」的情况下，带来非常明显的系统加速。
- 在 **DeepSeek-V4-Pro** 的设置上实测，整个方案能在 **MoE 部分**带来接近 **30%~40%** 的收益（投影片：30%–40% speedup on the MoE part）。
- 把这部分放回完整模型的测试里，**64K** 配置下能拿到 **14%** 的收益。
- 在 **1M** 上下文上只有大约 **4.6%**，明显偏低。
- 1M 偏低的原因「跟前面分享的部分类似」：这个 case 下 **MoE 本身的占比就偏小**；另外 1M 这个 case 的并行度(?)也比较小，所以再叠这些手段能带来的进一步收益没有 64K 那么明显。
- 他给这一段定位：前面这几个部分（STEP 1–3）是**相对通用**的部分，最后一部分才是专门针对长序列场景问题的。
- **口头补充（幻灯片上没有的）**
- 「性价比」式判断：小额额外开销换大幅加速，在他看来这是这套方案最值得强调的一点，而不是某个单点 kernel 数字。
- 收益随场景变化的经验：同一个优化在 64K 与 1M 的相对收益由「MoE 占比」与「并行度」两个因素共同决定，不能只看绝对加速比。
- 过渡铺垫：他提示观众，从前面几张投影片的图里已经能看出来，1M 这个 case 下 **Indexer** 部分占比比其他情况长非常多，这就是下一步要做更细 profile 的动机。
- **ASR 存疑**
- `mo一部门` = MoE 部分；`接近30%~40%的收入` 中的「收入」应为「收益」；`在一道上面` / `一照` = 1M（百万 token）上下文。
- `posoh度` 疑为「并行度」**(?)**；`这些丢人带来` 疑为「这些手段带来」**(?)**；`前面的那个符号里面` 疑为「前面那几张图/幻灯片」**(?)**。
- 本页窗口（01:10:51–01:12:38）确实在讲第 49 页，无错位。

第 50 页 | STEP 4: Balance CP Workloads

> 拍摄 14:43:38 ｜ 音频 1331 段偏移 4350 ｜ 讲者 白洪潇（NVIDIA Megatron 团队）

- **口头要点**
- 这一页是对长序列问题做更细的 profile：把不同 CP rank 的时间分别看一下，**这是一个 CP32 的 case**。
- 从 CP32 组里抽了几个 rank 出来对比：**绿色**部分是 CSA forward(?) 的时间，**灰色**部分是 Indexer 的时间。
- 可以看到 **Indexer 在不同 rank 上耗时差距非常明显**；但是把 Indexer 那部分减掉、只看 CSA，各 rank 的时间消耗差距并不大。
- 所以他们的优化**主要集中在 Indexer 部分**。
- 为什么会有这个问题：跟 CP 的切分方式有关。设计 DeepSeek-V4 的 CP 切分时，考虑到 sliding window、CSA 里的一些压缩算法等，为了简化这部分操作，采用**连续的 CP 切分**——CP4 就把整个序列切成 4 份，每个 rank 分一份。
- 但 Indexer 内部的计算涉及 causal mask(?)（「只有一半内容会被计算」），而且**这一半在不同 rank 里的占比还不一样**，于是靠前的 rank 实际做的计算少，越靠后计算量越大。
- **口头补充（幻灯片上没有的）**
- 这是一次标准的「先 profile 再优化」示范：他先指出「CSA 各 rank 差不多、Indexer 差很多」，把原因定位到**切分粒度**而不是 kernel 本身，然后才提出改法。
- 他明确说连续切分是为了**简化 CSA 的压缩/滑窗操作**才这么设计的——也就是说负载不均衡是设计取舍的副产品，而非实现 bug。
- 本页窗口内没有出现具体 ms 数字（投影片上是 53.1ms→18.4ms），讲者只用「差距非常明显 / 差距并不大」这类趋势判断。
- **ASR 存疑**
- `CSA和生部分` 疑为「CSA forward 部分」**(?)**；`eex` / `应该` / `一ex` 均为 **Indexer**；`PRO如马k` 疑为 **causal mask**(?)（或 position/mask 一类）。
- `CT r` / `CP32`：投影片是 CP32，讲者口中先出现「CP32 的一个 case」，后文又出现 CP4，属正常举例。
- `除理应该的部分` = 「除了 Indexer 的部分」。

第 51 页 | STEP 4 II: Balance CP Workloads（zigzag 布局）

> 拍摄 14:44:55 ｜ 音频 1331 段偏移 4427 ｜ 讲者 白洪潇（NVIDIA Megatron 团队）

- **口头要点**
- 比如 CP4 这种情况，**第 0 个 rank 的工作量可能只有第 3 个 rank 的约 1/7**。
- 于是做更精细化的切分，让每个 rank 处理的工作量尽量一致：在 Indexer 这里做了一次**内部的转换**，转成传统 CP 那种「细分/块切分」的方式。

- 具体做法：CP4 就把它**切成 8 份**；分配时用**间隔（strided）**的方式——从最前面最轻松的部分、以及最后面工作量最大的部分分别去取，这样每个 rank 分到的工作量基本一致（即投影片上的 zigzag 对角线配对）。
- 效果：在 **1M** 序列长度上能带来 **10% 以上**的收益，比较明显。
- 但在 **64K** 上：64K 虽然也算长序列，这个设置下**没有测到明显收益**。
- 原因：64K 下不均衡带来的 overhead 本身不严重，而做平衡算法本身又会**额外引入 overhead**，两者**相互抵消**，所以效果不明显。
- 收尾：他把前面提到的 **4 个优化**放到一起做对比（这句话一直说到下一页投影片出现）。
- **口头补充（幻灯片上没有的）**
- 这是「优化优先级取决于目标负载」最直接的实例：同一优化在 1M 有效、64K 无效，他给出的机制是「不均衡的收益」与「平衡算法自身开销」的量级对比，而不是简单说「长序列就该上」。
- 他用 **1/7** 这种量级描述极端不均衡（投影片 CP32 上是 53.1→18.4 约 3 倍），说明 CP4 这种粗切分下不均衡比 CP32 更极端。
- 他主动提醒观众：64K 也是长序列，所以不能一概而论——这是整场里少见的「给结论同时给失效边界」的讲法。
- **ASR 存疑**
- `第零个rank…可能1/7` 数字听感偏弱，**1/7 存疑**(?)。`内奥的转换` 疑为 internal transformation**(?)**；`代理细分` 疑为「块/条带细分」**(?)**；`CP44` 疑为 CP4**(?)**。
- `一兆` / `一照` = 1M；`10%以上的收益` 与投影片 12.2% 一致（口头取整）。

第 52 页 | Lessons for Scalable Training（KEY TAKEAWAY）

> 拍摄 14:49:28 | 音频 1331 段偏移 4700 | 讲者 白洪潇（NVIDIA Megatron 团队）

- **口头要点**
- 收尾话：欢迎大家试用（Megatron Core 的 DeepSeek-V4-Pro 参考配方），并「给我们提一些有用的建议」。
- 随后直接道谢、进入现场问答。
- 投影片正文（三条 key takeaway、Get Started with Megatron Core、配方文件名）在录音窗口内**没有被念到**，见「ASR 存疑」。
- **口头补充（幻灯片上没有的）**
- **【Q&A 1 | 部署与使用建议】** 观众问：站在 DeepSeek-V4-Pro 使用者的角度，结合这些优化点，在部署和使用时有什么建议？参数怎么设更合理、什么场景下优化效果更能体现？→ 白洪潇答：**推理部署他这边不是非常熟悉**，前面讲的内容**更偏训练**；推理方向的优化方向可能**完全不太一样**，「这样一些东西你可能没有办法直接给它迁移过去」。（问答结束后主持人发了龙虾公仔奖品。）
- **【Q&A 2 | 为什么按首尾配对】** 观众问：1M 下计算量随序列长度增长、越往后越多，所以「用首尾（两端）划分」是不是就按注意力/计算量随长度增长来判断的？→ 答：**是的**；并补充：这样平衡之后「理论上它是完全均衡的」。

- **【Q&A 2 追问 | 与其他上下文并行方案比较】** 观众提到 DCP 一类按 token 位置切分的方案（用于推理）。→ 答：主要问题是**训练和推理场景不太一样**，他不太好直接比较这些推理侧策略；但「如果是这种方案的话，它们的计算量确实就非常的好(?)」（原话含糊）。
- **【Q&A 3 | FP8 精度损失】** 观众指出：原始 DeepSeek-V4 是用 **FP16** 做 QK 的中间计算拿到 score 矩阵 S，而这次 Indexer 优化里**直接用了 FP8**，会带来精度损失，问后续训练有没有对造成的差值做补偿/量化。→ 答：如果是**非常长时间**这样练，他们**确实没有做过**；较短的测试里看，带来的性能影响「可能并不是特别明显」，当然也可能跟实际场景有关。DeepSeek 自己用什么精度，其公开配置里**没有提**，其 reference 里可能有注释暗示用了混合/BF16(?) 之类的做法，但**没有给出具体实现**，所以是「一个不确定的状态」；他猜测对方也用了类似策略，并且应该验证过「对精度带来的影响是比较有限的」。
- **【Q&A 4 | 显存换性能的阈值】** 观众问：STEP 1(?) 用的那个策略，在个人显存受限的消费级显卡上适用吗？怎么衡量「用显存换性能」的阈值？→ 答：**跟显卡种类没关系**，就是跟显存余量有关；具体要 case by case——比如开某个功能会带来几 GB 甚至十几 GB 的额外显存消耗，就看模型跑起来之后你的显存余量够不够；「当然你可以试一下给它开起来，看它是不是会 OOM 或出现其他问题」。
- **【主持人收尾】** 感谢白洪潇老师，说明会后还可以继续交流，并宣布「**接下来我们从训练转向模型的服务**」——这是他交棒给徐文杰的过渡句。
- **ASR 存疑**
- 本页窗口从 **01:18:20** 起已经是致谢与 Q&A；本页投影片正文（Lessons 三条 + Megatron Core 入口 + 配方文件名）落在 **01:15:55–01:18:17 的录音缺口**内，未被转写，故上文「口头要点」只到收尾话。
- `韩老师` 疑为口误/ASR 误识（本场讲者是白洪潇）**(?)**；`dpc可v斯PRO` / `C PRO` / `V似` / `维斯` 全部 = **DeepSeek-V4-Pro**。
- `古达Ros` 疑为 STEP 1 阶段某个策略/算子名，无法确定 **(?)**；`P均` 疑为「平衡」**(?)**；`经济损失` 疑为「精度损失」**(?)**；`京东上面` 疑为「精度上面」**(?)**；`在成成功跑起来` = 「模型能成功跑起来」。
- Q&A 后半段（Q&A 2 的答案与 Q&A 3、4、主持人收尾）超出本页音频窗口，取自同场录音的问答切片 `_build/audio/qa/qa2.txt`（对应 01:20:32–01:25:28）；本页窗口本身在 01:20:27 处停在一个问句中间。

5.3 第 3 场（徐文杰）：KV Cache 压缩，第 53–67 页

投影片给结论，口头给了选型顺序；问答部分（15:31 后 13 分钟）信息量最大。

第 53 页 | Motivation: KV Cache as a Serving Bottleneck（议程页）

> 拍摄 14:58:41 | 音频 1331 段偏移 5253 | 讲者 徐文杰

- **口头要点**
- 新讲者（徐文杰）用这一页交代本场结构：先说「各位观众能从我这次分享里面得到什么样的简短的结论」。
- 四个部分：**动机**（KV cache 为什么成为推理服务的瓶颈）→ **分类**（推理期 KV cache 压缩有哪些模式）→ **性能分析**（TurboQuant / KVTC / nvCOMP）→ **Takeaways**。
- 他没有在议程页上停留，紧接着就翻到动机页：「我们先来看最上面这个公式」。

- 公式讲解（在议程页窗口内已经开始）：分子里的 **2** 就是 **K 和 V** 两个分量；**context/序列长度** 是推理服务持续增长时需要缓存的 token 数量；**层数、head 数、head_dim(?) 这三个属于模型自有参数**，推理服务/模型一旦确定就不再改变。
- 剩下的一项是**他们存储 KV cache 所使用的数据精度**——这是公式里唯一可以动手的乘数。
- 由此得到一个「随着 context 增长而增长」的式子。
- **口头补充（幻灯片上没有的）**
- 他的叙事模板很固定：**先建公式（给定量的成本模型）→ 再定位瓶颈 → 再分类 → 再逐算法验证**，整场都在这条线上。
- 把「精度」单独拎出来讲，等于提前给全场定了调：压缩的本质就是降这个因子（在线量化）或降被缓存的 token 数（剪枝/丢弃）。
- 议程页只承担「预告结论」的功能，实际讲解时间几乎为零——这是学术型讲者的典型做法，与前半场白洪潇逐项讲优化收益的风格形成对比。
- **ASR 存疑**
- 本页窗口（01:27:33–01:28:26）**已进入第 54 页的公式讲解**，议程页本身的展开（四个章节的逐条说明）落在 01:25:28–01:27:33 之间，未被本页文件捕获；上文四节标题据投影片 OCR 补全，**未经录音确认**。
- `c个sn` 疑为 context sequence length / 序列 token 数**(?)**；`La尔stvh和还的两t选o` 疑为「L 是层数、还有两个是 head 数和 head dim」**(?)**；`他们在此` 疑为「位宽/bytes per element」**(?)**；`可v开始` / `qv开始` / `kv开始` = KV cache。

第 54 页 | KV Cache Is a Serving Bottleneck

> 拍摄 14:59:26 ｜ 音频 1331 段偏移 5298 ｜ 讲者 徐文杰

- **口头要点**
- 由公式可得：随着上下文增长、token 越来越多，**KV cache 的大小是线性增长**的。
- 在百万级（1M）上下文的场景下，这种线性增长会**为整个推理系统带来很多瓶颈。**
- 他挑出中间部分**三个比较典型的瓶颈**：
    - **瓶颈一：显存容量**。如果请求上下文特别长，它会直接导致推理服务的**活跃 batch size** 和**可支持的最大上下文长度**都变得很低，于是 GPU 上能做的计算量特别少，**GPU 利用率变低**。
    - **瓶颈二：HBM 带宽**。做 decode 的时候，**每一个请求的完整 KV cache 都要从 HBM 读到 SM 上**；KV 越长，数据搬运时间就越长，进而使 **TPOT** 这个关键指标的延迟增加。
    - **瓶颈三：系统 I/O**。当代的 KV cache 管理系统会把（压缩后的）KV 保存在**多级缓存**里——不只 HBM，也可能在 host 内存(?) 上、本地 SSD 上，甚至是远端存储上；而 KV 长度增加后，**KV 落在哪一级，哪一级的 I/O 就成为瓶颈**：可能是 PCIe，可能是 SSD 的 I/O，甚至可能是远端网卡。
- 结论：很直观地看，**如果有一种手段能降低 KV cache 的大小，就能显著改善推理引擎的吞吐等各方面指标**。
- **口头补充（幻灯片上没有的）**

- 他对「多级存储」的枚举很具体（HBM / host 内存 / 本地 SSD / 远端），这实际上提前解释了两页后「在线路径 vs offload 路径」的分工为什么存在。
- 三个瓶颈分别对应三个不同指标：容量→**batch/最大上下文**，带宽→**TPOT**，I/O→**PCIe/SSD/网卡**；他没有给具体带宽数字，全是因果链。
- 「降低 KV 大小」这句把全场三个算法统一成一个目标函数，后面的 TurboQuant / KVTC / nvCOMP 只是三种降法。
- **ASR 存疑**
- `必焦的用率` 疑为「GPU 利用率/显存利用率」**(?)**；`头盘越来越多`、`c增量型的增长` = context/序列长度增长；`活跃的概率三的大小` 疑为 **active batch size**(?)。
- `pp开始` / `PP开始` / `pv开始` / `kv开始` / `可v开始` 全部 = **KV cache**；`Htm` / `tbm` / `hbm` = HBM；`后sm位` 疑为 host/CPU 内存**(?)**；`铝墨头` 疑为 remote（远端）**(?)**；`电引擎` = 推理引擎。
- `本地的logo s在地上面` 疑为「本地 SSD（disk）上」**(?)**。

第 55 页 | KV Cache Compression Methods（training-time & inference-time)

> 拍摄 15:03:08 | 音频 1331 段偏移 5520 | 讲者 徐文杰

- **口头要点**
- （承接上一类方法）量化这么做的效果是：**存储内存下降、访存量下降、（反量化）计算量也下降。**
- **第三类叫「计算式稀疏」（compute sparsity）**：完整的 KV cache 依然存在存储系统里，运行时通过一些选择/排除算法**挑出 top-K 最重要的部分**，只让这部分参与计算。
- 它对系统的影响：**内存/存储占用基本不变**（因为存的还是完整数据），但**内存访问量显著下降**，**KV 上的计算量也随之下降。**
- 讲完分类后转到部署层面：把压缩算法部署到真实的推理引擎服务系统里会面临什么，**大致分两类——推理引擎内**与 **KV cache 管理后端**。
- 先讲引擎内：就是常见的 **vLLM / SGLang / TRT-LLM**(?) 一类推理引擎，把压缩算法集成到推理引擎的**在线计算路径**上。
- 通常做法是**以 op 的形式**：不管做成量化算子、压缩算子、反量化算子，它就是一个 op。
- 如果要做极致优化，就把压缩/解压算子**和 attention 或其他计算算子融合**，让两边互相 overlap，把压缩和解压的开销**藏起来**。
- **口头补充（幻灯片上没有的）**
- 他给的三条「影响面」（存储占用、访存量、计算量）是后面比较三种算法的统一尺子：量化三项全降、存储稀疏只降占用、计算稀疏只降访存与算力而**不降占用**。
- 「计算稀疏不省显存」这一点其实是在提醒听众：**想省显存在线只能靠量化，想省带宽可以靠 top-K 读**——这正是下一页两条集成路径的分野。
- 讲 op 与 kernel fusion 时用了「藏起来（overlap）开销」的说法，是全段的核心工程哲学：压缩的开销不是消掉，是被掩盖。
- **ASR 存疑**

- 本页窗口从 **01:31:59** 才开始，**第一类（训练期架构设计：MQA/GQA、MLA、Kimi Linear 等）与第二类（量化）的讲解**落在 **01:30:26–01:31:57 的录音缺口**内，未被转写；窗口内只剩量化那一类的「效果」小结。
- `计算式的输出` = 计算式稀疏（compute sparsity）；`vrm浪加trm` 疑为「vLLM、SGLang、TRT-LLM」**(?)**；`安排去计算` 疑为「反量化计算」**(?)**；`排除一些…选择的算法` 疑为「筛选/top-K 选择」**(?)**。
- `压缩算子反量化算子或者呃其压缩算子` 疑为「解压缩算子」的重复/误识 **(?)**。

第 56 页 | Where KV Compression Fits（两种集成路径）

> 拍摄 15:04:39 | 音频 1331 段偏移 5611 | 讲者 徐文杰

- **口头要点**
- 引擎内集成能带来的好处：**batch size 与最大上下文长度都可能得到改善**(?)；decode 时需要读的数据量减少，**TPOT 指标会下降**。
- 但这种部署方式**会带来很明显的代价**：首先，为了达到最佳效率，通常需要把解压算子写得很定制（手写 fused kernel），结果就是要**改动引擎内部、还要安全地实现**，这是一个**比较大的工程**；其次，需要**针对不同的推理引擎、不同的模型做特化**。
- 第二类：**KV cache 管理后端**——像 **LMCache**、HiCache(?)、Dynamo(?) 这些，它接在上层推理引擎下面，负责 KV 的存储。
- 压缩算法在这条路径上**以 offload 的形式存在**，优点三条：① 对上层推理引擎**完全透明**；② 能**显著降低存储开销**；③ 在整个系统 I/O 上**降低 I/O 流量**。
- 收束到下一步：「接下来我们会结合和我们客户/社区……」（转入具体算法与实测）。
- **口头补充（幻灯片上没有的）**
- 他把「要不要放进引擎」归结成两个成本的比较：**融合/特化的工程成本** vs **解压开销能否被省下的 KV 读取抵消**——这就是投影片底部那条 integration rule 的口头版。
- 「透明」是他强调的最大卖点：引擎不用改、压缩在下面做；代价是解压必须在 attention 之前**还原成标准 KV**，所以**不直接改善 TPOT**。
- 他两次提到「针对不同引擎/不同模型做特化」，说明这条路线的瓶颈不在算法而在**集成碎片化**。
- **ASR 存疑**
- `班赛可以达到分赛的支持` 疑为「batch size 可以达到更大的支持」**(?)**；`最大的上下的强度的冲突都会有新的的改善` 疑为「最大上下文长度也有改善」**(?)**；`参与性` 疑为「挑战/代价」**(?)**。
- `全做fsfs` 疑为「全部手写/做成 fused kernel（CUTLASS 一类）」**(?)**；`动同来想安全的实现` 疑为「改动引擎并安全实现」**(?)**。
- `lmlkv` = LMCache；`我们k` 疑为 HiCache**(?)**；`你m` 疑为 Dynamo/NIXL**(?)**；`ofro` = offload；句尾「我们会结合和我们客户」被截断。

第 57 页 | TurboQuant: Online KV Quantization

> 拍摄 15:07:42 | 音频 1331 段偏移 5794 | 讲者 徐文杰

- **口头要点**
- 第二步：对**旋转之后**的输入向量，预先准备一个**编码本（codebook）**；把每一个数值找到它在码本里**最近的位置**，把这个**索引**存下来——这个索引就是压缩后的数据。
- 之后还可以做一些算法上的补偿：比如用 **QJL** 的方式算一个**符号位**，作为恢复时数据的**校准**（inner-product correction）。
- 第三步是恢复：最上面黑色部分是压缩后的 KV；中间通过**查表（lookup）**把数据从压缩字典里取出来，再做**反旋转和反缩放**。
- 据 TurboQuant 论文报告，他们给出了一个**很诱人的 4.5x 压缩比**，在这个压缩比下**在绝大部分数据上都有很好的性能**。
- 但根据他们与社区伙伴的测试，**3.6x 左右可能是更「甜点」的数值**。
- 压缩后的数据怎么存：① 压缩后的**索引**；② **QJL 算出来的符号位**；③ 每个向量的**缩放/范数**一类元数据。
- layout 的 baseline 是 **vLLM 已开源的实现**：它把每个 token 按 **token 交错**的方式存，一个 token 要把 K codes、V codes、norm、scale、zero 这几个数据**交错打包成一段**；所以要访问第一个 token，得**跳 134 个 byte**。
- **口头补充（幻灯片上没有的）**
- 4.5x（论文）与 3.6x（社区实测甜点）的区分很有信息量：名义载荷压缩比不等于可用压缩比，**考虑精度后要往回收**——这是他全场少见的对自家论文数字的「打折」。
- 「134 byte 跳着访问」是为下一页 NCU 的 3.68B 做铺垫：先给出既有布局的物理形态，再指出访存效率的灾难。
- 他把「索引 / 符号位 / 逐向量元数据」三类字段讲得很清楚，因为这直接决定了下一页能做字段平面化（同一类的字段可以连成一片）。
- **ASR 存疑**
- 本页**第一步（随机旋转：raw vector → random rotation → 高维坐标更平滑）**的讲解落在 **01:35:31–01:36:29 的录音缺口**内，未被转写；本页窗口从「旋转后的输入向量」开始。
- `预编码一个编码本` = 预计算 codebook；`误` 疑为「码本」**(?)**；`QQ 1的国家选的方式` 疑为 QJL projection/sign**(?)**；`托` 疑为 lookup**(?)**。
- `以严肃的方式存储打成1个p` 疑为「interleaved 打成 1 个 page」**(?)**；`134个大b` = 134 bytes；`4.5倍`、`3.6倍` 听感清楚。

第 58 页 | TurboQuant: Layout Compressed KV for Decode

> 拍摄 15:10:01 | 音频 1331 段偏移 5933 | 讲者 徐文杰

- **口头要点**
- 现有（token 交错）布局的两个毛病：**访问不合并（no coalescing）**，**数据也没有对齐**——所以「光对齐其实也没用」。
- 用 **NCU** 测：每访问 **32B** 的数据，只有 **3.68B** 是有效的；**大部分数据都无效**，原因就是访问不合并 + 数据不对齐。

- 于是对数据做一次**转置**：把一个 page 里的 **16 个 token**（讲者说「也可能还能更多」）拿出来，**所有 K codes 连续存、V codes 连续存**，K norm / V scale / V zero 也各自连续存。
- 这样在保存/访问时就变得**特别友好**。
- 结果：在整个 **kernel 层面**，每 32B 客户单位的访问效率提升到 **29.75B——这一个布局改动就带来约 8 倍的访存效率提升**。
- 接着讲怎么把 TurboQuant 的解压计算融进 **FlashAttention**：以 FlashAttention 算子本身的实现作为整体框架，把 TurboQuant 特有的计算融进去。
- 数据读取路径（图最上面）：对每个上来的 Q，按**最多 16 个 head** 为一组做**旋转**，得到旋转后的 **FP16** 数据。
- 对 K/V 这些压缩数据，把 **4-bit codes + metadata** 读上来；整个数据流做**双缓冲**。
- **口头补充（幻灯片上没有的）**
- 这一页给了完整的「测量 → 归因 → 改布局 → 再测量」闭环，**3.68B → 29.75B** 是全场唯一的**纯访存层面**数字，也是「8 倍」这个说法的出处（29.75/3.68 ≈ 8.1）。
- 他特别强调「访问不合并」和「数据不对齐」是两个**独立**原因，所以只做对齐并不能解决问题，必须改成按字段平面化（等价于转置）才有效。
- 双缓冲的目的很明确：让 tile 之间的加载不停机（下一页展开）。
- **ASR 存疑**
- `用co来做一个测试` / `co` 疑为 **NCU（Nsight Compute）** 或 `ncu` **(?)**；`3.68b`、`29.75`、`8倍` 听感清楚；`客户单位` 疑为「32B load sector」**(?)**。
- `以可ps` / `可ps` 疑为 K codes**(?)**；`v扣的` 疑为 V codes**(?)**；`推动们VC料和比较的` 疑为 norms / scales / zeros**(?)**。
- `以10用的汉度…10多个h度` 疑为「最多 16 个 head」**(?)**；`扣0S的麦克data塔` 疑为「4-bit（低比特）codes 的 metadata」**(?)**；`数据的流畅会做一个双缓冲` = 数据流做双缓冲。

第 59 页 | TurboQuant: Tiled MMA Decode Attention on 4-bit KV

> 拍摄 15:11:54 ｜ 音频 1331 段偏移 6046 ｜ 讲者 徐文杰

- **口头要点**
- 双缓冲的具体含义：做**第 i 个 tile** 的计算时，同时去 **load 第 i+1 个 tile**，用的是**异步数据加载接口**。
- 计算路径：把序列按 **64 个 token 一个 tile** 做切分，也就是所谓的 **KV split tile**；**每一个 CTA 处理其中一个 split**。
- tile 内部：K codes 从双缓冲里拉上来后，因为 **K 矩阵是沿 token 维度切分的、每个 token 的数据量很少**，所以**直接从寄存器里解出来**即可。
- 然后用**旋转后的 Q** 与 **K 的转置**做计算——这是标准 FlashAttention 的做法。
- 再对结果做一次**缩放/反量化的校正**，这是他明确点名为「**TurboQuant 额外引入的东西**」。
- 接着是标准的 **online softmax / max** 计算。
- V 侧必须经过 **shared memory**：因为 **V 矩阵是沿另一个维度（head dim）切分**的，切分后这个维度的数据需要在**不同的 tile 之间复用**，所以要通过 shared memory 从寄存器转出来。

- 最后完成 **P×V 的 MMA 计算**，输出一个 tile 的结果（跨 KV split 的 FP32 运行状态再合并成 BF16 输出）。
- **口头补充（幻灯片上没有的）**
- 两处「为什么这么做」很有价值：**K 走寄存器**是因为按 token 切、每 token 数据少；**V 必须落 shared memory** 是因为按 head dim 切、要在 tile 间复用——这解释了同一条 attention 里 K 和 V 为什么走两条不同的数据通路。
- 额外的 scale 校正被点名为压缩带来的**在线开销**，也就是「加速不是白来的」那部分成本。
- 整页他按「内存路径 → 计算路径」两条线讲，而不是照图从左到右讲，是工程讲者的典型顺序。
- **ASR 存疑**
- `第I加一个跳的漏` = 第 i+1 个 tile 的 load；`64个头看为一个票` = 64 个 token 为一个 tile；`kv实力的票e` = KV split tile。
- `每一个CPA来处理其中一个具体的PE` 疑为「每一个 CTA 处理其中一个 split」**(?)**；`重器` = 寄存器；`政策化的教做` 疑为「反量化的 scale 校正」**(?)**。
- `使用12个单位` 含义不明 **(?)**；`罗开始` 疑为 shared memory**(?)**；`太要的结果` 疑为「tile 的输出结果」**(?)**；`发线的那个要求` = FlashAttention 的做法。

第 60 页 | TurboQuant: Decode Performance

> 拍摄 15:15:29 | 音频 1331 段偏移 6261 | 讲者 徐文杰

- **口头要点**
- 结果：基本上在**各种架构上**都能获得 **1.25x 到 4.8x** 的 decode 吞吐提升。
- 对比对象是 **vLLM 原始的 baseline 实现**。
- 相对（蓝色那条 / **FlashAttention**）也能拿到大约 **1.5x** 的吞吐提升(?)。
- 一个很有意思的现象：当上下文增长到 **16K 往上**，**FlashAttention 会 OOM**，这时候 TurboQuant 的优势就体现出来了——它**不会 OOM、可以支持更长的上下文**，从而给系统带来**更大的容量**。
- TurboQuant 的优化实现**已经开源**，在 **vLLM 的 PR #53060** 上。
- 社区里有小伙伴拿这个 PR 在自己的**生产环境**上做了测试，得到了非常好的加速比；感兴趣可以把这个 PR 拉下来真实体验。
- 接着转入第二个算法：**KVTC**。
- **口头补充（幻灯片上没有的）**
- 「16K 以上 FA 会 OOM」是他解释收益曲线拐点的关键：长上下文下的加速**不只来自位宽更小**，还来自「**基线根本没跑起来**」（投影片上多处标 FA OOM = capacity-skipped）。
- 他对社区复现是邀请口吻（「拉下来真实体验」），隐含自家数字还需要外部验证；也提到投影片底部的严谨性声明（跨 GPU 的对比只作方向性参考）。
- KVTC 的出发点在这一页末尾给出：**层（layer）、head、以及维度这三个维度之间都存在信息**，所以压缩时应该**综合考虑这三个维度**，而不是只看一维。
- **ASR 存疑**

- `1.25倍到4.8倍` 听感清楚（投影片峰值 4.84x 吻合）；`大概在1.5倍` 与投影片「相对 FA 长上下文 3.7x–4.8x」不一致，疑为指短上下文或口误 **(?)**。
- `回来参数和om` 疑为「FlashAttention 会 OOM」**(?)**；`通过框不会o` 疑为「TurboQuant 不会 OOM」**(?)**；`非常性感的加速比` 疑为「非常惊喜/非常香」**(?)**（也可能是讲者原话的玩笑说法）。
- `微量维测题目研究出来的` 疑为「（我们）NVIDIA Research / 我们团队研究出来的」**(?)**；`那么HD`、`那么1尔斯还的两个选` 疑为 layer / head / head_dim 三个维度**(?)**。
- 本页窗口内已在讲 KVTC 的动机（01:45:28 起），属正常串讲，非错位。

第 61 页 | KVTC: Compression for Storage and Transfer

> 拍摄 15:17:21 ｜ 音频 1331 段偏移 6373 ｜ 讲者 徐文杰

- **口头要点**
- 算法分三步：① **拼接**——把需要压缩的三个维度上的数据点**拼成一个连续的维度**；② **PCA 降维**——本质上就是一次普通的矩阵乘法，把一个很大的输入做**主成分分析**、把维度降下来；③ **动态精度分配**——根据主成分分析的结果判断每个数值的重要性：重要的用**更高精度（比如 FP8）**存储，重要性稍差的用 **INT4 / INT2** 存储，最不重要的**可以直接抛弃**（作为可选方式）。
- 最后还可以在压缩后的数据上再用一个**无损压缩 nvCOMP** 做**二次压缩**。
- 中间那层图展示的是 **KVTC 如何集成到 KV cache 计算的数据流**中：KVTC 需要做 PCA，这是一个**很大很重的矩阵乘法**，所以**很难直接用在基于 HBM 的在线路径上**，他们把它**放在 offload 路径上支撑**。
- 运行时路径：把压缩后的 KV **从 CPU 加载到 HBM** → 用 KVTC 的**反量化算法在 SM 上解压** → 把解压后的数据**写回到 HBM** → 需要做 attention 计算时再从 **HBM 加载到 SM**。
- 论文报告：可以获得 **8x 到 64x** 的压缩比。**为什么说是「动态」**——因为**主成分想丢多少就可以丢多少**。
- 论文报告在 **16x 压缩率**下，**绝大部分经典数据集上的精度下降都小于 1 个点。**
- 同时他们和一些**很重要的客户**合作，客户在某些场景下做了内部测试，反馈在 **8x 压缩率**上（也就是原始 16 bit 的数据）……
- **口头补充（幻灯片上没有的）**
- 「为什么不能在线用」讲得很实在：不是算法不行，而是 **PCA 这个矩阵乘太重**、且 KVTC 的组织粒度（**cross layer/head**）与 attention 的取用粒度（**per layer/head**）**不匹配**，所以只能退到 offload 路径。
- 「动态压缩比」的定义很关键：8x–64x 不是固定档位，而是靠**丢主成分的个数**连续调节——这解释了为什么同时存在 8x（客户实测）与 16x（论文精度结论）两个数字。
- 二次无损压缩（nvCOMP）在这里被说成「可以再套一层」，为后面第三个算法（nvCOMP）埋了线：有损之后还能接无损。
- **ASR 存疑**
- `汇入一个…拼成一个连续的维度` = concat；`本质上就是一个普通的乘法` = 矩阵乘；`动态规划优化的方式` 与投影片 `DP-planned mixed precision` 吻合。

- `fp8` / `I特2` / `I nnt4` = FP8 / INT2 / INT4； `n微抗谱` 疑为 **nvCOMP**（或 nvCOMP-ANS / DEFLATE）**(?)**； `kbtc` / `kptc` / `pvpc` / `pcpc` / `PP开始` 全部 = **KVTC** / KV cache。
- `落实多到sbm上面` 疑为「写回到 HBM 上面」**(?)**； `吊顶小于一个点` = 精度下降小于 1 个点；句尾「原始是 16 比特的数据」之后被窗口截断（下一句在 062 文件开头）。

第 62 页 | KVTC: Integration and Pipelined Decompression

> 拍摄 15:20:28 | 音频 1331 段偏移 6560 | 讲者 徐文杰

- **口头要点**
- （承接 8x = 16bit → 约 2bit）每张 GPU 上需要解码的份额其实是一样的，所以 **KVTC 的解压计算可以放到 8 张卡上并行做。**
- 然后**补一个 AllGather 算子**，把 8 张卡的数据汇总起来。
- 于是很直观地形成一条**三层流水线**：① **PCIe 的 host-to-device** 拷贝；② **GPU 计算**——既包含**受带宽限制**的反量化/按 DP 分配精度的反量化算子，也包含**计算密集**的矩阵乘法；③ **NCCL 通信**（AllGather）。
- 经过这样的 **TP 优化 + 三级流水线编排**之后，KVTC 的解压相比原先能获得 **4~5 倍**的速度提升。
- 集成架构（下面那张图）：**最底层是 KVTC 算法**（三步 + 计算），对上提供一个 **codec 接口**；**中间层是 KV cache 管理层**——他们在 **LMCache**（以及 HiCache(?)）这类后端上做了**实际的优化**，并且正在对 HiCache(?) 做进一步优化。
- **上层推理应用只需要通过 GPU connector 连到中间的 KV cache 管理层，就能直接使用压缩能力。**
- 接着进入 KVTC 的性能测试：模型 **GLM-5.1**(?)、**8 张卡 TP=8**、用的是 **vLLM + LMCache** 的一套组合、**8x 的 KV cache 压缩率**、**4.7x 的综合压缩率**。
- **口头补充（幻灯片上没有的）**
- 「每张卡要解码的量是一样的，所以解压可以并行」是他把通信/计算重叠讲清楚的前提：**先确认切分均等**，再用 AllGather 把完整 M 补回来（投影片上的 M-sharded TP-GEMM + AllGather）。
- 三级流水线（拷贝 / 计算 / 通信）是他描述工程实现时的固定说法，「4~5 倍」就是这条流水线的总收益。
- 他提到**正在优化 HiCache**，说明这套 codec API 已经不止一个后端落地，且是「边讲边做」的状态。
- **ASR 存疑**
- `一个o盖的算子` / `o的算子` 疑为 **AllGather** 算子**(?)**； `3层的流水线` 、 `4~5倍的提升` 听感清楚。
- `P PC PC的居民的计算` 疑为「KVTC 解压的计算」**(?)**； `缓存受限的地方` 疑为「受带宽/缓存限制的部分」**(?)**； `反样化算子` = 反量化算子。
- `在安开始这一层以及感m没这一层` 疑为「在 LMCache 这一层以及 HiCache 这一层」**(?)**； `海开的这个管理的服端` 疑为「HiCache 的管理后端」**(?)**； `GPU connect` = GPU connector。
- `需要0.12 18的模型` 疑为模型名（GLM-5.1 / GLM-4.5 之类）**(?)**； `4.7倍的综合压缩应该还有3次` 句尾被窗口截断， `还有3次` 疑为「还有 3 张图」**(?)**。

第 63 页 | KVTC: Single-Request TTFT

> 拍摄 15:23:01 | 音频 1331 段偏移 6713 | 讲者 徐文杰

- **口头要点**
- 这一页几乎没停留：拍摄时刻他只剩半句「……上下来移动」，TTFT 曲线本身的讲解（Prefill 与 PCIe 传输叠加、各上下文下的降幅）没有落进本页文件。
- 随后他直接转到**真实吞吐（end-to-end）测试**：「接下来我们看一下相同（场景）下，我们在真实吞吐测试下会出现什么样的一些性能表现」。
- 他说明呈现方式：**图很多、有三张图，我只想带出三个观点。**
- **第一个观点**：不集成 KV cache 压缩算法时，随着横轴 **host 内存可用容量下降**，**KV cache 命中率会直接下降。**
- **口头补充（幻灯片上没有的）**
- 第一个观点的定量版：命中率**从 80% 下降到了不到 20%**；命中率下降的后果是**发生重计算（recompute）**，反映到系统上就是**吞吐降低——从原先接近 140 降到 50。**
- 这实际上就是本页之后那张图的 takeaway 的口头版：**内存变小 → 驱逐（eviction）→ 命中率下降 → 重计算 → 吞吐断崖。**
- 投影片字面内容（TTFT 由 843→773ms、591→558ms、407→377ms、204→182ms，降幅 5.5%–10.8%；setup 为 GLM-5.1 FP8 / 8×Hopper / PCIe Gen5 / TP=8 / vLLM+LMCache / CR 4.7x）中，**setup 部分**在上页窗口末尾被提到（8x KV cache 压缩、4.7x 综合压缩），**柱值与降幅未被念到。**
- **ASR 存疑**
- `上下来移动` 是断句残片，疑与「随上下文增长 TTFT 往下走」有关 **(?)**。
- **拍页/翻页错位**：本页照片拍摄于 15:23:01，但窗口内讲者讲的已经是**第 64 页的三张图**（命中率 80%→20%、吞吐 140→50），说明该页投影片在他讲话时被快速跳过或翻页早于快门。
- `横轴的这个后s，单位的可用的POS单位` 疑为「横轴的 host memory 容量（GiB）」**(?)**；`80%`、`不到20%`、`140`、`50` 听感清楚。

第 64 页 | KVTC: Throughput Under Memory Pressure

> 拍摄 15:23:45 | 音频 1331 段偏移 6757 | 讲者 徐文杰

- **口头要点**
- **第二个观点（右图，讲者说「大图往往都是最重要的」）**：随着 host 内存下降，比如降到 **96GB** 的时候，**上面那条绿线（KVTC）始终维持很高的吞吐。**
- 这种情况下它相比**低命中率的非压缩场景**可以获得 **2.49 倍**的吞吐提升。
- **第三个观点**：如果 host 内存比较充足（讲者说推理引擎生产里部署场景很多），在**多种并发场景下（并发从 1 一直到 16）**，吞吐**始终在正负 3% 之间波动**，是一个很小的波动范围。
- 「这就是 KVTC 在性能上的表现」——本段 KVTC 讲完。
- 接着介绍**第三个压缩方法：nvCOMP**。

- **nvCOMP 是什么**：它是 **NVIDIA 的一个通用、专门面向 GPU 的（KV cache）压缩与解压库**，里面有一些**极致优化的经典压缩算法。**
- **第一，它是无损的**——数据原来是什么样，压缩完再恢复以后还是什么样。
- **第二，它是通用的**——理论上可以压所有的数据；他举例说我们平时的一些压缩包，底层可能就是用这类算法做的。
- 里面集成了很多算法，他们**一个个验证以后发现 ANS 这个算法对 KV cache 是最友好的。**
- **口头补充（幻灯片上没有的）**
- 「并发 1→16 吞吐只波动 ±3%」是他对「压缩是不是白拿的」这一质疑的回答：**内存充足时开销可忽略**，等于几乎免费；这也对应投影片上「Low overhead — Capacity sufficient (400 GiB)」那张表（+3.6% ~ −2.1%）。
- **无损 + 通用 + ANS** 三点把 nvCOMP 定位成「**兜底方案**」：不改模型、不改精度语义，靠通用无损压缩拿收益——与前两个有损算法形成互补菜单。
- 讲者在这里明显在收束三个算法的定位：TurboQuant（有损、在线、省 HBM 读取）、KVTC（有损、面向存储/传输）、nvCOMP（无损、通用、末级二次压缩）。
- **ASR 存疑**
- `96GB`、`2.49倍`、`正负3%`、`1一直到16`、`8张卡` 听感清楚。
- `NV抗和` / `NV抗的` 疑为 **nvCOMP**(?)；`ans这个算法` = ANS**(?)**；`因为他不是nvd亚的一个通用的专门的一个GPU的PP开始的压缩和解压力库` 中的否定词疑为 ASR 误加，语义应为「**它是** NVIDIA 的一个通用的、专门面向 GPU 的 KV cache 压缩解压库」**(?)**。
- `后程` = host；`后程m瑞` / `hosts m位` / `host m位` = host memory；`qv条` 疑为「KV cache」**(?)**；`从一一直到16` = 并发 1 到 16；`绿色的线`（投影片为绿线 KVTC，OCR 未标颜色）**(?)**。

这一段横跨两位讲者、两种风格。白洪潇（49–52）是「收益账本式」：每页都落到数字（MoE 30%–40%、64K 14%、1M 4.6%、1M 12.2%），而且每给一个收益必跟一句失效条件——1M 下 MoE 占比小、64K 下不均衡收益与平衡开销相互抵消；问答里也直说「推理部署我不熟悉」「长时间这样练确实没做过」，不装懂。徐文杰（53 起）是「模型先行」：先用 KV 大小公式建成本模型，再拆容量 / 带宽 / IO 三重瓶颈，然后把压缩分类、把部署分成在线路径与 offload 路径，最后用 TurboQuant、KVTC、nvCOMP 在同一坐标系里定位。他总先讲「为什么这么做」（K 走寄存器因按 token 切、V 落 shared memory 因按 head 维切要跨 tile 复用），再讲数字，且数字都带限定（4.5x 是论文名义值、3.6x 才是社区实测甜点）。两人共同点：主动暴露失效边界，不宣称普适。

第 65 页 | nvCOMP-ANS: Lossless KV Cache Transfer

> 拍摄 15:25:32 | 音频 1331 段偏移 6864 | 讲者 徐文杰

- **口头要点**
- nvCOMP 这类库里集成了很多压缩算法，他们**逐个验证**后认为 **ANS 这个算法对 KV cache 是最友好的**，于是选它。
- 因为它是**无损**的，讲者的态度是"不做白不做，反正是免费午餐"，于是把它集成进了 **FlexKV**（KV cache 管理后端）(?)。

- 搬移路径讲得很具体：原始 KV 存在 **GPU** 上（BF16 / FP16 / FP8 一类精度 (?)），用 **nvCOMP 的 ANS** 在 **GPU 上做压缩**，压缩后数据走 **PCIe** 传到 **CPU 侧的管理后端**，后端再借助 KV cache 管理把数据放到 **SSD** 上，也可以放到内存的任意位置 (?)。
- 在 FlexKV 上做了**好几层**的集成，并做了**性能分析和验证**；支持了 **MHA / GQA / MLA / DSA** 这一类注意力算法 (?)，以及 **TP 和 PP** 这类并行方式。
- 压缩率口头给了数：**BF16 KV 平均 1.4x**，**FP8 KV 平均 1.1x**——并明确对比"这跟 TurboQuant、KVTC 动辄 **8 倍、16 倍甚至 32 倍**的压缩率不太一样"。
- 之所以敢用这么低的压缩率，是因为**它是无损的**；讲者由此给出判断：**对精度极度敏感的场景，这种方式值得尝试。**
- **口头补充（幻灯片上没有的）**
- 选型过程的细节：不是直接照搬某个默认算法，而是"里面集成了很多算法，我们一个个验证以后发现 ANS 对 KV cache 最友好"。
- "免费午餐"是这一段的贯穿说法（页 65 开头一次、页 66 结尾又一次"免费的午餐谁不想尝一尝"），用来降低听众的试用门槛。
- 三级搬运链的**工程落点**讲得比图上更具体：GPU 侧只做编解码，真正把数据接到 SSD / 内存的是 CPU 侧的 FlexKV 管理后端。
- **ASR 存疑**
- "pv开始 / kp开 / pvpc / pptc / PP开始" 等反复出现的词，按上下文应为 **KV cache**；"pptc" 应为 **KVTC**；"n为抗 / n微看 / NV com" 应为 **nvCOMP**（各标 `(?)`）。
- "Stv / spv / 格stv" 疑为 **FlexKV**（页 65–66 的幻灯片与 PR 都指向 FlexKV，标 `(?)`）。
- "它可能是21较6还比16.18的" 疑为"可能是 **FP8 / FP16 / BF16** 的"（`(?)`）。
- "支持了安全些样行apsa" 疑为"支持 **MHA / GQA / MLA / DSA** 这些注意力算法"（`(?)`）。
- "也可以发到与m特的任意啊任意的程序的想法" 明显错译，只能推断为"也可以放到内存的任意位置/程序里"（`(?)`）。

第 66 页 | nvCOMP-ANS: Compression and Serving Results

> 拍摄 15:27:40 | 音频 1331 段偏移 6992 | 讲者 徐文杰

- **口头要点**
- 测试配置**和 KVTC 那一页一样**（讲者原话"测试的配置和 PPT 那个是一样的"）：提到 **GLM-5.1、8 卡、BF16 KV**（`t78` 疑为 FP8，与 OCR 页面的 **GLM-5.1-FP8 / BF16 KV / 8x Hopper** 对应，`(?)`）。
- **吞吐：不管并发怎么变，吞吐基本不变**——讲者明确说"这个现象和 KVTC 的结论是一样的"，对应幻灯片 **Throughput unchanged**。
- **TTFT：在低并发下收益最明显**。讲者给的数是"**c = 1、2 的时候，TTFT 的 P90 能达到接近 20% 的提升**"（幻灯片为 c = 2 处 **+18.7%**，P90 组另有 **+8.2%** 均值口径）。
- **随着并发数增加，带来的提升会逐渐下降**——和幻灯片高并发处出现负值柱一致。
- 结论句很干脆：**低并发场景用 nvCOMP 做压缩，能在吞吐不变的情况下提高 TTFT 这个指标。**

- 开源信息：这是 **FlexKV 的 PR #183**（`(?)`），**已经进入 FlexKV 主线**，讲者邀请大家"拖下来实际测一下"。
- **口头补充（幻灯片上没有的）**
- 讲者特别补了一句原因层面的解释：压缩率不高（1.4x / 1.1x）仍值得试，因为**无损**；这话把页 65 的数据和页 66 的收益连起来。
- "毕竟是**无损**的嘛，就是**免费的午餐**谁不想尝一尝"——这是第二次用同一措辞，明显是有意的记忆点。
- 说明收益口径时要小心：口头只报了"接近 20%"在低并发，没报页面上那些 1%~8% 的小幅度与负值，**听众容易把 20% 当成通用收益**。
- **ASR 存疑**
- "ttmt他的啤酒零" 疑为"**TTFT 的 P90**"（`(?)`）；"12的时候" 疑为"**c = 1、2 的时候**"（`(?)`）。
- "虽然5.1把卡都会丢u啊或是噢啊 t78" 只能推断为"**GLM-5.1、八卡、BF16 / FP8 KV**"（`(?)`）。
- "stv的PR和183" 应为 "**FlexKV 的 PR #18x**"；OCR 页面记作 `pull/18a`，录音听作 **183**，两者取录音（`(?)`）。
- "吞吐如咽的情况下来提高我们提前技这个指标" = "**吞吐不变的情况下提高 TTFT 这个指标**"。

第 67 页 | Takeaways

> 拍摄 15:29:43 | 音频 1331 段偏移 7115 | 讲者 徐文杰

- **口头要点**
- 讲者先用一句自嘲过渡："**整个 PPT 念了 30 分钟**，其实想跟大家抛出来的观点就是**三个**。"
- **观点一（压缩跑在哪一层，决定提升什么）**：KV cache 压缩算法被集成到**推理引擎的每一个层面**，带来的提升效果**完全不一样**。如果集成进推理引擎内部、把它和算子做**融合**，那就可以**降低 HBM 上的压力、降低 HBM 存储压力，提高 decode 时的吞吐**。
- **观点二（集成到 cache 管理后端）**：如果 KV cache 压缩集成到 **KV cache 管理后端**，那它更关注的是**改善整个推理服务系统的 IO**，同时**降低数据传输**、降低**后端存储（SSD）上的数据开销**。
- **第三个说法（不同算法改善不同指标）**：不同 KV cache 压缩算法带来的性能提升**完全不一样**，能改善的指标也不一样——**优化后的 TurboQuant 算子**提升的是**长上下文下的吞吐**；**KVTC** 是在**保证吞吐不变**的前提下降**内存压力**；**nvCOMP** 是在**低并发**场景下降 **TTFT**。
- **观点三（怎么选）**：选 KV cache 压缩算法时必须考虑**推理引擎的部署条件**和**真实业务场景**，并且要有**自己的核心指标**——到底关注的是**推理输出的精度质量**，还是 **TTFT / TPOT / 吞吐**，甚至 **KV cache 命中率**，以及集成压缩算法到整个推理系统时**需要付出的成本**。
- **口头补充（幻灯片上没有的）**
- 口播把幻灯片的三条要点**重新组织成了"三个观点"**：第一个讲"集成位置"、第二个讲"算法差异"、第三个讲"选型方法论"，其中"算法差异"这条在幻灯片上是被放在 **The methods delivered different gains** 里的案例，讲者把它抬成了独立观点。
- 讲者明确**补上了"成本项"**：幻灯片只说 "after codec and integration costs"，口播展开为"**集成到整个推理系统时需要做的（成本）**"，并留了话尾（录音在此处截断）。

- 这一段本质是**方法论总结**而非结论复述——"你需要有一个自己的核心指标"是讲者给听众的选型建议。
- **ASR 存疑**
- "pptc" = **KVTC**；"n为抗体" = **nvCOMP**；"通框图的算子" = "**TurboQuant 的算子**"（`(?)`）。
- "降低hm上面的那个压力 / 降低hm为存储压力" = "**降低 HBM 的压力**"；"提高后的时候的吞苦" = "**提高 decode 时的吞吐**"。
- "或者说 pdfttbut或者吞吐" 疑为 "**TTFT、TPOT 或者吞吐**"（`(?)`）。
- 该时间窗最后一句"还甚至说我们在集成 KV cache 压缩算法到整个推理系统时候需要做的"在录音里**没有说完**（音频在 01:59:59 处结束），后半句缺失。

5.4 第 4 场（卢祥龙）：Dynamo 与智能体推理，第 68–96 页

口头补充了不少落地经验（版本要求、缓存分层建议、三级存储的取舍）。

第 68 页 | What Dynamo can do today

> 拍摄 15:45:57 | 音频 1531 段偏移 889 | 讲者 卢祥龙（NVIDIA 解决方案与工程）

> 讲者切换：**卢祥龙（NVIDIA 解决方案与工程团队）**，分享《Dynamo 为智能体推理基础设施加速》。

- **口头要点**
- 开门见山给两个"影响特别大"的问题：**这么高并发的请求，每个请求应该路由到哪一个 worker**，因为要同时管理**负载均衡**和**cache 命中**。
- 第二个是 **KV 的多级/分层态势管理**：现在有 **L1 → L3** 一直到 **CPU memory → SSD** 的**多级存储管理**，用来**提高全局的 KV 复用比重**。
- 还提供了很多其他重要组件：**非常多的容错机制**，前端、以及 worker 那一批（`外的一批`）都可以通过某些方式做**容错**（`(?)`）。
- 做了很多 **simulation（数字孪生式仿真）**：给定**一个模型、一个 SKU、一个 GPU**，要用什么样的**并行方式**、什么样的**策略**才能取得好的收益；真机测试会占用非常多的 GPU 资源，所以想用**高保真的 simulation——用大部分 CPU 加很少的 GPU** 来做到仿真。
- 今年有**三四个月**时间转向 **harness 式的 agentic 整体编排**：去年还是 **PD 调度**的概念，现在有了 **agent 调度**的概念，"是一个完全新的时代"，因此引入了很多相关组件。
- 引出第一部分：**KV Router 和 Flash Indexer**——"为什么我们要一个 router？我们用 **round-robin** 不好吗？"答案是**轮询解决不了**同时保证 **worker 间负载均衡** 与 **让每个请求命中 KV 最高**这两个使命，**只有精确的 KV Router 能做到**：对每个 worker 的 **KV 命中有一个 score**，**负载均衡有一个 load score**，再做**加权打分**。
- **口头补充（幻灯片上没有的）**
- 幻灯片是"能力清单"（各种 disaggregation、容错、全局 KV 管理、Grove、数字孪生、RL rollout），口播则按**问题驱动**重讲：先说"高并发下请求该去哪个 worker"和"KV 多级存储"这两个最痛的点，再补容错、仿真、agentic 编排。

- **仿真（simulation）的动机是算力成本**：真机试配置"会占用非常多的 GPU 资源"，所以要用"大部分 CPU + 很少 GPU"的高保真仿真先做投影——这是幻灯片上"Digital twin / high-fidelity projection"背后的工程理由。
- **对 round-robin 的否定**是现场最值得记的判断：负载均衡与缓存命中在纯轮询下**不可兼得**，因此必须是"两个 score 加权"的 KV-aware routing。
- 明确点名当前处于"**agent 调度**时代"，并把自己团队近三四个月的投入方向说了出来（agentic 编排 + 相关组件）。
- **ASR 存疑**
- "l一到l三cpmr到SSD多级的一个存储的管理" 疑为 "**L1 → L3、CPU memory → SSD 的多级存储管理**"（`（?）`）。
- "我们都可以通过你搜这样的一些方式去做它的容错" 中"你搜"疑为某个具体机制名（可能是快照/弹性扩缩一类），无法确认（`（?）`）。
- "simulation o / c么l选o / simulation" 明显是 **simulation** 的多次错译；"一个sk u一个GPU" = "**一个 SKU、一个 GPU**"。
- "哈斯space斯的这种a整体的编排" 疑为 "**harness 式的 agentic 整体编排**"（`（?）`）。
- "我们一会大夫一点到这个 kvv尔老和flash 感这个索引" = "一会会讲到 **KV Router 和 Flash Indexer**"；"老 / rooter" 一律为 **router**；"lo score" 为 **load score**。
- 该页 OCR 的社区数据（**~8.3K stars / 244K+ pip installs / 9,000+ PRs**）在这个时间窗里**没有被口播**。

第 69 页 | KVRouter 在生产环境中被广泛验证 / FlashIndexer 是支撑准确路由的基石

> 拍摄 15:49:10 | 音频 1531 段偏移 1082 | 讲者 卢祥龙（NVIDIA 解决方案与工程）

- **口头要点**
- **索引的作用**：它可以告诉每一个请求"**我现在所有的 worker 上有哪些是跟你是命中的**"，而且**实时性非常好**。
- **两路信息汇合**：负载与命中这两路信息**同时到 router 的进程里面**，router 再按前面说的方式**算加权打分**，问题就解决了。
- 顺势转入索引本体："我们来说一下这个索引，我们的索引叫做 **Flash Indexer**"。
- **口头补充（幻灯片上没有的）**
- 口播在这一页强调的是**信号的实时性与汇合点（router 进程）**，属于对第 70 页那张"两类信号 → Cost Function"图的机制性解释。
- 该时间窗**没有朗读**本页 OCR 上的两个客户案例数字（**Baseten 2X TTFT / 1.6X TPS、Meituan 2x TTFT / +20% KV cache hit**），案例数据是"挂在页面上"而没有展开讲的。
- **ASR 存疑**
- "那某服装店这的CP的memory"、"某服装店" 反复出现，疑为某个进程/服务名（如 standalone indexer 进程）的误译，无法确认（`（?）`）。

- "然后老出来去算刚才我们算的一个方式，解决掉了去了" 句尾残缺，"解决掉了"后面的宾语缺失（`(?)`）。
- 本页两个案例（Baseten / 美团）与页脚两条链接**在录音中缺席**，属于"照片时刻讲者已翻页/未逐条念"的情况。

第 70 页 | 请求进入后，KV Router 用两类信号为 Worker 打分

> 拍摄 15:49:29 ｜ 音频 1531 段偏移 1101 ｜ 讲者 卢祥龙（NVIDIA 解决方案与工程）

> 注意错位：本时间窗（00:18:23–00:19:20）讲者已经在讲 **Flash Indexer 的数据结构**，而非本页的 cost function 图。

- **口头要点**
- Flash Indexer "说得有点抽象，那它其实是一种**数据结构**"——可以理解为**一棵并发的 radix tree**（`c卡人的r tree`，`(?)`）。
- 它**存在某个进程的 CPU memory 的堆里边**；随着 **KV block 数量增大**，它的**占用会变大，可能是几百兆到个别 G**。
- 它里面**存下所有 worker 现在的负载情况**，以及**每一个 worker 上 KV block 的存储情况**——"所以你请求来了你才能去做比对"。
- 所以它本质上**是一张"位置到 worker"的地图**：请求进来后，先把 prompt **切成 blocks**。
- 右边是一个 **hash map**（`action ma`）：可以看到**在第一个位置**上有 `0`（即 `(0, h0)` 这一项），后面靠扫描这个表来判断命中。
- **口头补充（幻灯片上没有的）**
- 两条幻灯片上没有的量化信息：**索引常驻在 CPU 内存堆里**，**占用从几百 MB 到个别 GB**（随 KV block 数量增长）——这解释了为什么它必须做得足够省内存、足够快。
- 讲者用"**账本/地图**"两个比喻反复解释同一个结构：对外是"位置 → worker 的地图"，对内是"记录所有 worker 状态的账本"。
- 数据结构形态被口头点名为"并发 radix tree"，比页面上只画 `(position, block_hash) → worker 集合` 更接近实现。
- **ASR 存疑**
- `c卡人的r tree` 疑为 "**concurrent radix tree**"（与页 73/76 演进梯队的 Concurrent Radix 对应，`(?)`）。
- "action ma" 应为 "**hash map**"；"位置到我的地图" 应为 "**位置到 worker 的地图**"。
- "它里面就会存下所有w rker现在的负载情况" 与幻灯片页 70 的"两类信号"口径一致，但**本页图上的 Cost = Prefill + Decode + （可选）活跃请求成本公式没有被口播**。

第 71 页 | Flash Indexer 在链路里做什么

拍摄 15:50:23 ｜ 音频 1531 段偏移 1155 ｜ 讲者 卢祥龙（NVIDIA 解决方案与工程）

- **口头要点**
- 请求被切成 blocks 后，**看哪些 worker 在同一个位置上有同一段 KV**；讲者点出关键前提：**做前缀缓存时，它跟位置是有很大关系的。**
- 规则讲得很明确：**你前面连续的所有 block 都命中，你这一段的前缀才能被复用**；"你不能（跳过）不同的位置去（命中）"。
- 于是**扫描这张表**：**第 0 个位置有 worker A、C、D 都满足**；**到请求的第二个 block，只有 A、C 满足**；**再往后只有 C 满足。**
- 结论：**只有 Worker C 针对你这个请求的 KV 命中是最高的。**
- **口头补充（幻灯片上没有的）**
- 讲者把"为什么必须看位置"讲成了**前缀缓存的定义性约束**（连续命中才算复用），比页面上的"同一 block_hash 可能出现在不同位置"更口语、更像经验判断。
- 逐步扫描 A/C/D → A/C → C 的推导过程是**现场演算**，页面上只给了结果（4 / 2 / 0），口播把每一步都念了出来，便于听众跟上。
- **ASR 存疑**
- "切成boss" = "**切成 blocks**"；"KB" 在本文多处实为 **KV**；"第零个位置有worker、ACD它都满足" = "**position 0 上 Worker A、C、D 都满足**"。
- "但是到了request始的第一个啊第二个呃第二个开始log" 断句混乱，只能推断为"到第二个 block 就只有 A、C"（`(?)`）。

第 72 页 | 索引里存的是一张"位置到 worker"的地图

拍摄 15:51:12 ｜ 音频 1531 段偏移 1204 ｜ 讲者 卢祥龙（NVIDIA 解决方案与工程）

- **口头要点**
- 这个索引**经过了前后六七次迭代**（"六七次"，与幻灯片 Gen01→Gen06 对应）；它"其实就是一个**账本**，去记录上面的 KV 有多少状态"。
- **最早**的实现是"**naive 的 … dict**"（`拿衣服的 nty，n的一个date`，`(?)`）：**每一个 worker 都有一个 position + block hash 的账本。**
- 于是**每个请求进来**，都要**一个一个 worker 去遍历这个 hash map**，看到底有哪些 block 是命中的。
- 复杂度当场给出：**O(W × D)——W 是 worker 的数量**，**D 是每个 worker 上这个 hash map 的深度。**
- 后来想"能不能有更好的数据结构"，于是考虑 **radix tree**：它能解决**位置**问题——**父节点一定是要命中的**，对应的语义就是**你前面这些 block 都命中之后，才看子节点。**
- **口头补充（幻灯片上没有的）**
- 讲者把复杂度讲得比页面更细：**W 是 worker 数、D 是每个 worker 账本的深度**，并强调慢的根源是"**每个请求都要逐个 worker 遍历**"。

- "**六七次迭代**"是口头给的代数（页面按 Gen01/06 → Gen06/06 记 6 代），说明这条数据结构线是长期打磨出来的。
- 讲者把 radix tree 的价值解释成一条**语义规则**（"父节点必须命中"）而不是数据结构技巧，这个说法很便于讲清 prefix 连续性。
- **ASR 存疑**
- "拿衣服的啊 nty, n的一个date" 疑为 "**naive 的 nested dict**"（与页 73 第一代 **Naive Nested Dict** 对应，`（?）`）。
- "啊 ow乘b" 应为 "**O(W × D)**"；"b就是你每个w上的这个还是m的一个深度" = "**D 是每个 worker 上 hash map 的深度**"。
- "要不要用redis, tree / rd才能解决一个问题" 中的 "redis tree"、"rd" 均为 **radix tree** 的错译（`（?）`）。

第 73 页 | GEN01/06 THROUGHPUT — 第一代：Naive Nested Dict

> 拍摄 15:52:09 ｜ 音频 1531 段偏移 1261 ｜ 讲者 卢祥龙（NVIDIA 解决方案与工程）

> 注意错位：本页照片为**第一代**，但本时间窗（00:21:01–00:22:04）讲者已经在讲 **radix tree** 和 **positional index + jump search**，口播比投影片快约 1–2 页。

- **口头要点**
- **radix tree 的语义**：**父节点一定要命中**，然后才看**子节点上的命中**——"这个也符合我们 **prefix cache** 的算法"。
- 举例说明路径：**先在 h1 这个节点**，**再到 h2**，最后才能选到 **W1、W2 这两个 worker**（"还是前面三个点都命中"）。
- 接着讲再快一步的方向：**用了 positional index 加（jump search）这种方式**。
- 给出适用场景的判断：**这种方式比较适合现在这种 agentic coding 的场景**——"大家如果看 **AgentX** 的话，就知道它的**理论命中是 97%，非常非常高**"。
- 97% 的含义被立刻翻译成工程结论：**意味着 worker 上的复用可能非常高**，所以**不必一个一个 block 去查**，改用**跳跃**的方式。
- **口头补充（幻灯片上没有的）**
- **97% 这个数字**在本页第一次出场（页面第 73 页没有该数值），并在页 79 被再次用来论证"理论命中 vs 实际命中"的落差。
- 讲者把数据结构演进的动机始终挂在**业务形态**上：因为是 agentic coding 这种高复用场景，才值得为"快速知道谁命中最多"付出设计成本。
- 口播对第一代的"为什么慢/为什么不准"**没有重复讲**（幻灯片上写得很细：O(W×D)、local_hash 碰撞），说明讲者对已翻过的页面略讲。
- **ASR 存疑**

- "rd才能解决一个问题" = "**radix tree 才能解决一个问题**"；"这个也符合我们这个啊 PP为us的一个算法" = "**符合 prefix cache 的算法**"（`(?)`）。
- "啊 h1ne这个一中，然后再到ht然后最后他才能选到w一w二" 只能推断为"先 h1、再 h2，最后选到 W1、W2 两个 worker"（`(?)`）。
- "所以我们就避免了一个一个的开始说去相关运用我们做一个跳跃的方式" 断句混乱，语义应为"避免逐个 block 查询，改为跳跃"（`(?)`）。
- 本页幻灯片上的 **385K block ops/s**、**Two hashes（local_hash + sequence_hash）**、**match depth 2/1/0** 在本时间窗**均未被口播**。

第 74 页 | GEN04/06 THROUGHPUT — 第四代：Radix Tree

> 拍摄 15:53:04 | 音频 1531 段偏移 1316 | 讲者 卢祥龙（NVIDIA 解决方案与工程）

> 注意错位：本页照片为**第四代 Radix Tree**，本时间窗（00:21:56–00:22:39）讲者在讲 **jump search 的具体跳法**与"这套结构有什么用"。

- **口头要点**
- 因为 **agentic 场景理论命中高达 97%**，worker 上的**复用非常高**，所以"**避免一个一个 block 去查**"。
- **跳跃的具体做法**：**先跳到 128 个 block、再跳到 256 个 block**，对**每一个 worker** 检查它**前面这些 block 是不是全部命中**。
- **判断规则**：**如果全部命中，就再往后跳**；**如果没有命中，就往后回退**这段区间去找（`(?)`）。
- 由此"引进了这样的一个（数据）结构"。
- 然后讲"这东西到底有什么用"：**各大公司现在都在管理非常大的集群——几千个、甚至上万个节点**去做 serving（`几千个节点上万个节点去做你的s…`）。
- **口头补充（幻灯片上没有的）**
- **跳跃步长 128 / 256 block** 是口播给的实现细节（页 75 的 OCR 只列了横轴 checkpoint 刻度），并且"**全命中继续跳、未命中回退**"这条判据被讲成了可操作的规则。
- 讲者把优化动机和业务规模直接挂钩：**上万节点级 serving 集群**才是这套索引存在的理由。
- 页面上的 **4M block ops/s**、`dual-key design: local_hash for traversal, seq_hash for events` 在本窗口**没有被口播**。
- **ASR 存疑**
- "我再往后推这个控制"、"我再往后推这个" 疑为 "**再往后（回退/回扫）这一段区间**"（对应页 75 的 lookback，`(?)`）。
- "比如说几千个几千个节点上万个节点去做你的s" 句子在录音里**被截断**（`s` 后无下文，可能是 serving / S 服务），`(?)`。
- "我们眼睛这数据结构有什么用" = "**我们这个数据结构有什么用**"。

第 75 页 | GEN06/06 THROUGHPUT — 第六代：Positional Indexer + Jump Search

> 拍摄 15:53:41 ｜ 音频 1531 段偏移 1353 ｜ 讲者 卢祥龙（NVIDIA 解决方案与工程）

- **口头要点**
- 在**几千到上万个节点**的集群里，**router 要实时掌控**：**所有 worker 的配置、负载情况、增删（上线/下线）**，以及**每个请求的运行情况**。
- **这个数据量非常大**，"它的 KV（数据）里面是非常大的"——所以索引必须扛住。
- 需求因此分成两半：**第一，每秒处理尽可能多的（事件）**——worker 发过来的**增删事件**要**马上消化掉**，把信息**增加到（索引）上**，**对树做更新和扩展**。
- **第二，请求进来时要找出哪一个 worker 命中最高**——"这个也是你（索引/radix）的一个查询过程"。
- 这两类动作被统称为 **index operations**，"我们希望让这个 operations 每秒能够处理越多越好"。
- **口头补充（幻灯片上没有的）**
- 讲者把"索引要处理什么"讲成了一个**双负载模型**：**写路径 = 高频 KV events 的实时 ingest + 树更新扩展**，**读路径 = 每个请求的前缀命中查询**；这个拆分是页面上"事件怎么处理 / 查询怎么走"的工程化重述。
- 强调了**实时性要求**：事件"**马上**把它消化掉"，因为状态旧就会路由错。
- "每个 worker 的配置、负载、增删、每个请求的运行情况"——比页面的 `Store/Remove events` 清单多了**配置**与**请求运行时状态**两项。
- **ASR 存疑**
- "我们的老能够实时的掌控你所有w克的配置" 中"老" = **router**；"w克 / w rker / wrk" = **worker**。
- "就是你的w rker发送发送过来这个增三的这样的一些里原子" 疑为 "**worker 发过来的增删（store/remove）事件/操作**"（`(?)`）。
- "然后还有你每个星球它的运动情况" 中"每个星球"无法确认，疑为"**每个请求**"（`(?)`）。

第 76 页 | 性能数据（Achieved vs. Offered Throughput）

> 拍摄 15:54:30 ｜ 音频 1531 段偏移 1402 ｜ 讲者 卢祥龙（NVIDIA 解决方案与工程）

- **口头要点**
- 上一页的两类动作（**事件 ingest + 命中查询**）都属于 **index operations**，希望**每秒处理越多越好**。
- 经过数据结构迭代，现在**可以处理 170 million operations per second**（"170million 个 operations per second"，对应页面 **170M block ops/s**）。
- 工程结论：**处理这种上万个 instance 的集群，这个索引不会成为瓶颈**——"**索引的查询和更新不会成为瓶颈**"。
- 转向 standalone 动机：**有些用户有自己的调度策略**，不想用他们的调度方式——"**因为我们的调度方式也只是我们自己的理解**"。
- 但强调"**这个 map 这个思维很重要**"，于是把它**打包成了一个独立的 KV Indexer 系统**，提供**查询 API**，让用户**自己的自定义 router** 去查**当前时刻哪些 worker 上有哪些 KV 还在**。

- 数据流：**左边 worker 不停地把 KV（事件）通过 ZMQ 消息队列发到独立的 indexer 进程**，indexer 再**通过 `POST /query` 这样的 API 对外发布，自定义 router 就通过这个 API 查到想要的 KV 命中情况。**
- **口头补充（幻灯片上没有的）**
- **170M ops/s 与"上万 instance 集群不成为瓶颈"**是现场给出的量化判断（页面只给曲线与 440x/42x）。
- **拆成 standalone 服务的产品动机**被讲得很直白：用户想保留自己的调度策略，所以只卖"索引能力"，不卖 router。
- **传输机制点名了 ZMQ 消息队列**（页面写作 ZMQ PUB/SUB），并明确了对外是 `POST /query` **这类 HTTP API**、使用者是"**自定义 router**"。
- 页面上的 **440x / 42x 倍数、24-core Arrow Lake 285K、Mooncake FAST'25 trace、16,384 GPU blocks、24 线程**在本时间窗**均未被口播**。
- **ASR 存疑**
- "170million个啊 R operations per second" ＝ "**170 million index operations per second**"。
- "上万的in fe的集群" 疑为 "**上万个 inference instance 的集群**"；后文"这个应该性索引的查询" ＝ "**这个索引的查询**"。
- "他不想用我们的po OS方式 / 我们他POS方式" 疑为 "**调度（policy/scheduler）方式**"（`(?)`）。
- "把它打包成了一个独立的k个inex词的一个系统" ＝ "**独立的 KV indexer 系统**"；"这样q消息队列" ＝ "**ZMQ 消息队列**"。

第 77 页 | Standalone KV Indexer: 定位

> 拍摄 15:56:14 ｜ 音频 1531 段偏移 1506 ｜ 讲者 卢祥龙（NVIDIA 解决方案与工程）

- **口头要点**
- 讲者给前面这一大段定性："上面其实是**有点像是上个时代的东西**，就是**去年我们聊的这种 request 级别的 scheduling**。"
- 但立刻补一句价值判断："**但是这个非常重要，这是基础。**"
- **到了今年，会去研究更多 agentic 结构。**
- **给 agent 下的定义**：如果给一个简单定义，就是"**（一个 harness）加上一个很聪明的模型**"，可以做 **web（search）等等这些事情**（`(?)`）。
- **agent 的范式**：**顺序地在 GPU 上跑 LLM**，然后**到 CPU 上做 tool call（突破）**，**循环直到完成所有任务**。
- 目标体验：**跟（大家平时）用（的工具）是一样的体验**，即用户侧无感。
- **口头补充（幻灯片上没有的）**
- 这一页口播的核心不是"Standalone Indexer 是什么"（页面的三栏对照没念），而是**时代划分**：**request 级调度 ＝ 去年/基础**，**agentic 编排 ＝ 今年/新**。
- "**非常重要，这是基础**"是讲者对前面 router/indexer 工作的自我定位——即复杂调度不能跳过 KV 索引这一层。

- **agent = harness + 聪明模型** 的定义，以及 **GPU 推理 ↔ CPU tool call 交替循环** 的范式描述，是页面（第 77 页三栏文字）上没有的内容。

- **ASR 存疑**

- "这种阿里斯加上一个一个很聪明的模型" 中"阿里斯"疑为 "**harness**"（页 68 亦出现"哈斯space斯"、页 78 出现"一监督人"，疑为同一词，`(?)`）。
- "然后它的范式就是啊顺序的llm后GPU上的，然后CPU上面的突破" 中"突破"疑为 "**tool call**"（`(?)`）。
- "就跟大家用，然后的就是一样的一个体验" 中"然后的"疑为某个 agent 工具名（如 Claude Code 一类），无法确认（`(?)`）。

第 78 页 | Standalone KV Indexer: 架构

> 拍摄 15:56:53 ｜ 音频 1531 段偏移 1545 ｜ 讲者 卢祥龙（NVIDIA 解决方案与工程）

- **口头要点**
- 观察：**很多公司在自己研发这么一套（agentic）基础设施**，去**节省成本**，或者做**定制化**，**在自己的内部业务上产生效果**。
- 转入基础话题："先从一些 **basics** 讲起"——**agentic 时代的 SLA 和去年相比发生了很大变化。**
- **传统 inference** 只看 **TTFT、TPOT、（请求）端到端 latency、QPS** 这些技术指标。
- **agentic 时代的要求变了**，第一点：**更关注"（轨迹）的端到端"问题**——"我问你一个问题，我期待你在**最短时间内完成整个任务、给我最终答案**"，这个很重要。
- 第二点：**time to first answer token**——模型要**思考很长时间、做很长时间的（推理）**才给答案，所以希望它**在思考、吐字之前的思考时间尽可能短。**
- 第三点：**系统吐字的速率（吞吐）越高越好**，因为"**这个跟成本直接挂钩**"。
- **口头补充（幻灯片上没有的）**
- 讲者把"trajectory E2E latency"翻译成了**用户视角的一句话**（"期待你在最短时间内完成整个任务、给我最终答案"），这是页面对照表里读不出来的动机。
- **time to first answer token 为什么重要**被讲清了：agent 有"长时间思考/工具调用"阶段，用户在这段时间是"什么都看不到"的，所以要压这段。
- **吞吐与成本的直接挂钩**被明确点出（"跟成本直接挂钩"），为页 79 的 KV 命中率—算力成本论证做铺垫。
- 页面的 **Standalone Indexer 架构图（ZMQ SUB / HTTP API / `(model, tenant) -> Radix Tree`）在本时间窗没有被口播**——讲者已翻到 SLA 话题。
- **ASR 存疑**
- "它想做到的事情啊就跟大家用，然后的就是一样的一个体验" = "**跟（常用工具）一样的体验**"（`(?)`）。
- "那在一个传统的一f里面" = "**在一个传统的 inference 里面**"；"快安全ls啊香港PPS" 疑为 "**端到端 latency、QPS**"（`(?)`）。

- "我们更关注这个垂直的安全的问题" 中"垂直的"疑为 "**trajectory（轨迹）**"（与页 79 对照表的 Trajectory E2E latency 对应，`(?)`）。
- "t to first answer talk" = "**time to first answer token**"。
- "然后像一些社区，呢比如说AA阵特" 中 "AA阵特" 疑为 **Artificial Analysis（AA）的 AgentPerf**（与页 79 **AA-AgentPerf** 对应，`(?)`）。

第 79 页 | SLAs of New Agentic Era

> 拍摄 15:58:12 ｜ 音频 1531 段偏移 1624 ｜ 讲者 卢祥龙（NVIDIA 解决方案与工程）

- **口头要点**
- 社区侧的评测指标：**AA（Artificial Analysis）** 认为应看"**SLO 下最大并发的 agent 数量**"，作为一项测评指标。
- 第五个指标：**全局 KV cache 命中率**。
- 关键落差："**AgentX 的理论命中可能是 97%**，但你这套实际的 **inference infra 软件**能不能让系统**实际达到 97%？不一定。所以（命中率）肯定是越高越好。**"
- **给了一个换算例子**：**KV cache 命中从 95% 掉到 90%**，换句话说**你需要两倍的 prefill 计算算力**去做（补算）——"**所以它的成本节省是非常恐怖的**"。
- **研究方向**：主要针对 **agentic 的 trace**；介绍 **AgentX trace**——一个**开源数据集**，大家按规范提交，里面有 **393 条真实的（Claude Code (?)) 对话轨迹**，一共 **98,000 个请求**。
- 输出形式：提交的结果是**一条 Pareto 曲线**，**纵轴是 TPS / 每 GPU 吞吐（输出吞吐）**，**横轴一般是你这个交互速率（TPS 的种类）**。
- 下面还给了**请求的输入长度分布**：**从 32K 到 256K、到 512K 不等**，"但我们可以看到它是**非常长**的"。
- **口头补充（幻灯片上没有的）**
- **95% → 90% 就等于两倍 prefill 算力**是本场最锋利的一个量化论证：命中率每掉 5 个点，成本就翻倍级别变化。
- 讲者区分了"**理论命中 97%**"和"**实际 infra 能不能达到 97%**"，并把这当成自己这套软件要证明的事。
- **benchmark 的口径说明**：AgentX trace 是**开源、按规范提交、输出 Pareto 曲线**，纵轴 **每 GPU 吞吐**、横轴**交互速率**——这是一套可复现的公开对比方法。
- 页面上的**具体分布百分比（<32K 2%、32K–128K 21%、128K–256K 27%、256K–512K 30%、>512K 20%）与三个大数字（393 / 98,827 / 1M）在本窗口只被部分口播**（393 与 98,000 念了，分布只念了区间）。另：本页 OCR 的 97% 与口播的 97% 一致。
- **ASR 存疑**
- "全局的一个KB开始的基图人" = "**全局 KV cache 命中率**"（`(?)`）。
- "你的so下最大的并发的a阵子30的数量" = "**SLO 下最大并发的 agent 数量**"（`(?)`）。
- "但是他最后的输出呢你的提交呢就是一个拜托的曲线" = "**Pareto 曲线**"；"纵轴的话就是你的TPS或是GP你的吞吐输出存吐" = "**TPS 或 per-GPU 输出吞吐**"。
- "然后大家如果去看那个上面s下面拿拿s斯的提交" 无法辨明，疑为某个提交仓库/规范地址（`(?)`）。

- "393条实际的扣子的实际的对话的一个轨迹" 中"扣子"疑为 **Claude Code**（与 OCR 页 80 的"Claude Code 轨迹"一致，`(?)`）。
- "pass 到 256...到 512k" 处数字有停顿，"32K→256K→512K"的完整区间只能大致还原（`(?)`）。

第 80 页 | AgentX Trace

> 拍摄 16:00:11 ｜ 音频 1531 段偏移 1743 ｜ 讲者 卢祥龙（NVIDIA 解决方案与工程）

- **口头要点**
- 要**真正做 benchmark**，就**需要满足这样一些要求、跟它对齐**（`需要完成跟它对齐`）。
- "刚才说了那么多条，那我就**拿出一条来看看它长什么样子、有什么特征**"。
- **看一条 session**：这条 session **一共花了 60 轮**才完成。
- **每一轮分成三个部分**：**一个是 GPU 上的，一个是 CPU 上的**；口头用颜色标注——**绿色基本上是 GPU 上的（推理）**、**蓝色是 CPU 上的（I/O (?))**、**黄色是 CPU 上的 tool call**（`(?)`）。
- **tool call 其实可以做很多事情**：比如 **web search**、比如**代码编译**，"这些其实都会占用 CPU **非常大的核数和耗时**"。
- 由此可以看到**每一轮从 0 到 60 轮，不同阶段的（时间）分布情况**。
- **第二个特征**在下面这张图：**基本上所有这种 agent 型（的负载）…**（录音在此处结束）。
- **口头补充（幻灯片上没有的）**
- **"一条 session 60 轮"**是口语给出的真实轨迹样本规模（页面只给分布比例，没有轮数）。
- **单轮的三段结构 + 颜色语义**（GPU 推理 / CPU I/O / CPU tool call）是讲者对着图现场解释的，OCR 只记下存在三色分段。
- 明确指出 **tool call 的 CPU 开销（核数与耗时）非常大**，并举例 **web search、代码编译**——这解释了为什么 agentic 负载是"CPU+GPU 混合"问题，而不只是 GPU 推理问题。
- 最后"第二个特征"**没讲完**（录音在 00:30:22 结束，音频片段到此截止），后半段论述缺失。
- **ASR 存疑**
- "这一条ST审他一共花了60轮" 中 "ST审" 疑为 "**session**"（`(?)`）。
- 颜色与阶段的对应存在错译风险："绿色的话就是你基本上的目料" 疑为"**绿色是 GPU 上的（model/推理）**"，"蓝色的话就是CPU上的以后" 疑为"**蓝色是 CPU 上的 I/O**"，"黄色的话就是CPU上的q高的" 疑为"**黄色是 CPU 上的 tool call**"（均标 `(?)`）。
- "它是需要满足这样一些要求" 中的要求清单**未被口播展开**；"下面这一行"（页面的实验方法标签：固定 seed、官方 harness 回放、每 lane 10 条、25–75% 切入、N lane、1 小时 warmup）在录音中**几乎未提**。
- 页面上的 **p99 近 900K、1,048,576 上下文窗口**在本窗口**未被口播**。

这一段最突出的特点是**重判断、轻图表**：投影片做的是结构化论证（对照表、曲线、六代演进阶梯），口播却按"问题—为什么不这么办—怎么办"推进，benchmark 参数与曲线细节多半略过，转而把每个技术选择翻译成"值不值得"——1.4x 压缩率靠"无损、免费午餐"立住，round-robin 因负载均衡与命中"两头不可兼

得"被否掉，索引拆成独立服务则用"用户想保留自己的调度策略"解释。口播优先抛数字与倍数（1.4x/1.1x、接近 20%、97%、95%→90% 等于两倍 prefill、170M ops/s），且与照片常有一两页错位。语气直白、带自嘲与劝试，术语音译错字极多，但不妨碍论证链条，并反复以"去年 request 级调度、今年 agentic 编排"划时代。

第 81 页 | Single AgentX Session Characteristics（单会话特征：单步时延分解 + ISL 增长）

> 拍摄 16:01:22 ｜ 音频 1531 段偏移 1814 ｜ 讲者 卢祥龙（NVIDIA 解决方案与工程）

- **口头要点**
- [30:14] 承接上一张图：讲的是"不同阶段的分布"，即每一步时延被拆成几段的构成情况。
- [30:16] "第二个特征呢，就是下面这个图"——几乎所有 Agent 型会话的 ISL 都是**线性单调递增**的，最早起步时大概只有 400 多个 token（投影片上是 448）。
- [30:31] 举具体流程解释为什么会涨：你让 agent 拉一个 git 仓库、解决里面的一个 issue，它就会不断去思考、去调工具。
- [30:46] 每轮的 response 和 tool（"突破"≈tool call）的 response 都会被**拼接进下一轮输入**，所以每一轮 ISL 越来越长。
- [30:57] 实测最后一直增长到 **219K**（投影片：219K，每步新增 p50 约 2K）。
- [31:02] 结论原话："**高 KV 复用**和 **ISL 很长且不断增长**，是 agentic agent 最重要的两个特性。"
- [31:14] 顺着这两个特征，转到"从 Dynamo 的角度看，未来这套真正的（软件）架构会长什么样"，并给出三层结构（详见第 82 页）。
- **口头补充（幻灯片上没有的）**
- 讲者给的是因果链而非曲线形态：思考 → 调工具 → 工具结果回填 → 下一轮输入变长，所以 ISL 单调增长是**任务形态决定的必然结果**，不是某个实现的偶然现象。
- "两个最重要的特性"这一结论是整段后续工作（KV 容量、命中率、调度）的出发点，讲者用它把问题定义收口。
- 时间窗内**没有**展开讲投影片上第 42 步"间隔 46.7 小时后恢复、187K 全量重算"这个标注 `(?)`。
- **ASR 存疑**
- "asl 都是线性啊单独递增"、"下一轮的 SL"：均为 **ISL**；"400 多个头"疑为 "400 多个 token"（投影片 448）`(?)`。
- "你跟他的一个 p 子"疑为 **prompt** `(?)`；"拉下这个给他的微口"疑为 "拉下（clone）这个 git 仓库" `(?)`；"里面的一个群"疑为 "一个 issue" `(?)`。
- "康开的到"疑为 "concatenate / 堆到" `(?)`；"高 KB 互用"疑为 "高 **KV 复用**" `(?)`；"a 阵子 agent 子"疑为 "agentic agent 的" `(?)`。

第 82 页 | 3 Layer Software Stack of Agentic Inference（Dynamo 三层软件栈 + agent hints）

> 拍摄 16:03:15 ｜ 音频 1531 段偏移 1927 ｜ 讲者 卢祥龙（NVIDIA 解决方案与工程）

- **口头要点**
- [31:21] 未来这套架构"主要是 3 层"；最上面是 **Layer 0**：各家自己定制化的开源 agent，客户也会有 Claude Code、Codex 这样一整套东西。
- [31:35] 中间层是**调度/编排**：会有一个前端（frontend）、一个 router，以及多级 KV 管理和 **KV 的 P2P 传输**。
- [31:51] 最下面一层是真正在 GPU 上干活的部分，要靠高效引擎执行：vLLM、SGLang（"希浪"）这些大家耳熟能详的引擎。
- [32:06] 讲者点名"中间这个 **agent hint** 比较有意思"：把一些 agent 侧已知的信息，**通过请求体传递**到 Dynamo 这一层。
- [32:21] 传什么？第一是"哪些 KV 前缀重要、哪些不重要"。举例：**`<think>` block 不重要**——想完了就丢掉了。
- [32:37] 再一个：**subagent 的子上下文不重要**——我只需要它的结果回到主 agent 里去做下一步，不需要保留它自己的历史。
- [32:52] 第二类信息是**优先级**：不同 session 有不同优先级（比如 VIP 用户），需要在 harness 这一层把这些信息传下来。
- **口头补充（幻灯片上没有的）**
- 讲者是从"agent 语义"反过来推"缓存策略"：因为 `<think>` 用完即弃、subagent 只回传结果，这些 KV 天然是"可牺牲"的，所以应该由 harness 显式告诉推理栈，而不是让缓存算法盲猜。
- 语气上把 agent hints 定位成**新增的接口约定/协议层工作**（"传递到我们的（Dynamo）边来"），是 harness 与推理栈之间要一起定的事，而不是 Dynamo 单方面能做的优化。
- 投影片上的 FlashIndexer `170M ops/s`、Thompson Sampling routing、`X-Dynamo-Session-ID` 等文字，在这一时间窗内**没有逐个念** `(?)`。
- **ASR 存疑**
- "啊的话是 20 层" = **Layer 0**；"o 客户合同话"疑为 "Claude Code / Codex 这样" `(?)`；"我们 o 编程"疑为 "orchestrator（编排）" `(?)`。
- "希浪这样明天这样" — "希浪" = **SGLang**；"明天"疑为 **vLLM / TensorRT-LLM** 之一，无法确定 `(?)`。
- "PP 开始重要 / 可 p 开始" = **KV cache（前缀）**；"think I 头" = **`<think>` block** `(?)`；"agent 三 agent 的这些小的子"疑为 "subagent 的子上下文" `(?)`。

第 83 页 | AgentX Dashboard（基准测试面板：吞吐 vs 单用户速度前沿）

> 拍摄 16:05:11 | 音频 1531 段偏移 2043 | 讲者 卢祥龙（NVIDIA 解决方案与工程）

- **口头要点**
- [33:10] 这是"我们的一个内部实验的提交"：用了 **4 个节点、32 张 GPU**，模型选 **GLM-5.2**，用来做大规模（"批发"）的 KV cache 实验 `(?)`。
- [33:27] 提交遵循的是三层（"三米" `(?)`）那套要求；随着 **concurrency 不断增大**，曲线从右往左形成一条**偏向右上方的前沿曲线**。

- [33:47] 做研究要做的就是"把这条曲线不断往右上角推"，推得越远性能越好。
- [33:56] 横轴是 **tok/s/user（TPS of user，每秒每用户读多少字）**，纵轴是 **output throughput**（"众轴…GPS 的 GP"）。
- [34:06] 软件栈：**Dynamo + SGLang + HiCache/多级缓存**这样一套一体化方案 `(?)`。
- **口头补充（幻灯片上没有的）**
- 讲者明确把这张"吞吐–单用户速度"权衡图当作**优化目标的坐标系**：不是单点指标，而是一条要整体外推的前沿（frontier），这决定了后面所有实验都是"同一张图上多条曲线"的对比方式。
- 报了硬件规模（4 节点/32 GPU）与模型（GLM-5.2），并强调这是**内部实验提交**，不是第三方数字。
- Dashboard 的 `Performance / Trends / Capacity / Runs` 页签、`log x / log y`、`COMPARE HISTORY` 等界面元素在时间窗内**没有逐一讲** `(?)`。
- **ASR 存疑**
- "demo 的 demo 发量"疑为 "Dynamo 的…发量/部署" `(?)`；"还开始加工"疑为 "HiCache（还开始≈HiCache）" `(?)`；"国际化成的一个西"疑为 "一体化的一个（方案）" `(?)`。
- "遵循啊三米的这样一个要求"：**"三米"不明**，疑为 "三层（three-tier）" 或某基准命名 `(?)`。

第 84 页 | Observability – Dynamo Provider Plugin（trace 可观测性）

> 拍摄 16:06:27 | 音频 1531 段偏移 2119 | 讲者 卢祥龙（NVIDIA 解决方案与工程）

> 说明：本页照片（16:06:27）对应的时间窗 `[35:17]-[36:40]` 讲者已经在讲第 85 页（1 GB 能存多少 token）。真正描述这张 trace 界面的是上一时间窗 `[34:16]-[35:05]`（082/083 文件），因此要点按该段归入本页。

- **口头要点**
- [34:16] Dynamo 提供了**agent 级的可观测性**能力。
- [34:27] 图上是 8 个（"s18 个"）对话/会话，每个对话里有一些 LLM 调用——就是图上的**蓝色**那部分。
- [34:39] 还有一些 **tool**（"to"）调用，是图上**棕色、绿色**的块，对应在 CPU 侧的工具执行。
- [34:44] 通过这些工具，可以很清晰地看到**每个时刻在做什么**、tool 的执行情况、以及**有没有空泡/空档**（"空号"`(?)`）。
- [34:57] 讲者评价它"有点像 agent 时代的 **strace**"之类的东西 `(?)`。
- **口头补充（幻灯片上没有的）**
- 讲者把可观测性的价值落在两件事上：看清**时间轴上的构成**（LLM 段 vs tool 段）与**空泡**，这正好对应第 81 页"tool/用户间隔占比最大"的结论，是同一问题的诊断工具。
- 用"agent 时代的 strace"这种类比帮助听众定位工具性质（系统级调用轨迹），而不是当监控大盘介绍。
- 界面里的 `swebench_p18taskpl-batch-…` 批次名、`Qwen3.5-27B (2149 tok)`、`tool:bash` span 名等细节在录音里**都没有念到** `(?)`。
- **ASR 存疑**

- "8 个 s18 个对话"疑为 "8 个（session）对话" `(?)`；"有没有一些空号"疑为 "空泡 / idle gap" `(?)`。
- "一个 a 选时代的一个啊 ssss"：**无法确认具体类比词**，按语义记为 "agent 时代的 strace" `(?)`。
- 本页时间窗 `[35:17]` 起内容属第 85 页，见下节。

第 85 页 | How many tokens fit in 1 GB（55.9 KB/token 与 TP8 / DEP 对比）

> 拍摄 16:07:40 ｜ 音频 1531 段偏移 2192 ｜ 讲者 卢祥龙（NVIDIA 解决方案与工程）

- **口头要点**
- [35:13] "第一个（想聊的细节）就是我们一个 GB 的存储，它能存多少 token？"——这是所有容量结论的起点。
- [35:22] 以 **GLM-5.2** 为例，它是 **MLA + DSA** 的结构；实测每个 token 占用 **55.9 KB** 显存。
- [35:34] 其中 **MLA 占 43.9（口语"44 点"）**，**DSA 占 9.8**。
- [35:37] 第二个问题：用 PD 分离之后要选运行策略（TP 还是 DEP），**"1 GB 能存多少 token"本身就是一个很重要的选择维度**。
- [36:06] 用 **TP** 的话，权重（"天水"≈TP）是切分的，但**每张卡都各留一份 KV cache**，等于有"惩罚性的复制"；1 GB 只能存 **2.2K**，所以会更快被挤出、落到 CPU/多级平台上。
- [36:32] 对比 **DEP** 的运行方式，1 GB 能存 **17.9K**——因为每个 DP rank 有独立的计算，请求报到独立的卡上，**没有复制的额外开销**，看起来是更好的方案。
- **口头补充（幻灯片上没有的）**
- 讲者把"每 token 多少 KB"直接升级成**部署选型维度**：同一个模型、同样的卡，TP 与 DEP 的差异不是微调而是数量级（2.2K vs 17.9K per GB），这是他后面所有 DEP 结论的前提。
- 解释了 TP 吃亏的原因：切分权重但不切 KV，等于 8 卡各存一份 → "惩罚的购置税"（ASR 原词）这一说法很形象。
- 57KB/token 与 55.9KB/token 两个数在后面几页混用（第 87 页写 57KB），时间窗内没有专门澄清差值来源 `(?)`。
- **ASR 存疑**
- "一个基地的存储" = "一个 GB 的存储"；"能存多少克斯" = "多少 token"；"每个头会占用 55.9 KB"疑为"每个 **token** 占用 55.9 KB" `(?)`。
- "mla 占 44 点，sa 占 9.8" = MLA 43.9 / DSA 9.8；"天水是切分的" = "**TP** 是切分的" `(?)`。
- "惩罚的一开始惩罚的一个复值税"疑为"惩罚性的（KV）复制"+"购置税"口语比喻 `(?)`。

第 86 页 | Goal 1: actual KV hit rate vs theoretical max（容量决定命中率）

> 拍摄 16:10:18 ｜ 音频 1531 段偏移 2350 ｜ 讲者 卢祥龙（NVIDIA 解决方案与工程）

- **口头要点**
- [36:51] 接着上页放大到部署规模：每个节点能存多少、4 个节点整套能存多少。
- [37:01] TP8（"GP"）能存 **5.0M**（"5 个命令"），而 DEP 能存 **30.6M**，差距非常大。

- [37:10] 但注意"它其实不是一个乘 8 的关系"——DEP 除了 expert 部分不切分，其它权重还是要切分，额外权重占用更多，还有稠密层的开销，**比 TP8 多占将近 100（GB）** `(?)`，所以留给 KV 的空间少一点。
- [39:06] 换算到单会话：一个 session 峰值可能还有 **300K** 的 ISL，这么多 KV 很可能存不下。
- [39:15] 存不下的去处：往 **L2 CPU** 放（local CPU memory），或者放到 **L3**（Mooncake）一个非常大的池子里；**再放不下就只能重算了**——这在 agentic 高并发下非常容易出现，是"非常不愿意看到"的事。
- [39:39] 初始设置（作为基准）：**GPU 4 节点共 1710 GB**；**G2 是 host CPU memory + local，4 节点共 2048 GB**；**G3 共享 CPU 池共 1024 GB（1 TiB）**。
- [40:07] 右侧对比图：**绿色是实际命中率，淡紫色是理论命中率**；并发 **64、96** 时实际能跟上理论，到了 **c=128 就跟不上**。
- [40:33] 原因不在算法而在**容量**：G3 有点小，它**只能存下 41 分钟**内产生的 KV；41 分钟之前的历史 KV 就会被从 L3 里挤出去。
- [41:03] 结果：实际命中率从 **95.8% 掉到 91.9%**，意味着约 **4%** 的 KV 要重算，会消耗很多算力。
- **口头补充（幻灯片上没有的）**
- 讲者先给"空间不够怎么办"的三级逃生顺序（G1 → L2 → L3 → 重算），再说"重算是最不愿看到的结果"，把后面的所有工作定义为"消灭重算"。
- 归因链条完整且是**容量归因**：不是路由算法差 → c128 时工作集溢出 G2 → G3 只装得下 41 分钟写入 → 老历史被逐出 → 8.2%/4% 重算 → 命中率掉 3.9 pt。这为"到底多级存储要设置多大"这一问句做了铺垫。
- 41 分钟这个数被反复强调（"他只能存 41 分钟"），是讲者给出的**可操作的容量标尺**。
- 投影片上 `TTFT p95 290 s` 与 `L3 hit 1.2%` 在这一时间窗内没有念到 `(?)`。
- **ASR 存疑**
- "GGP...就能存 5 个命令" = "TP8 能存 5.0M"；"30.6 个米令" = "30.6M token" `(?)`。
- "将近 100 分字"：单位不明（疑为 GB）`(?)`；"老 logo 的 cpup memory and loading"疑为 "local CPU memory + local disk/HBM" `(?)`。
- "I 三你 mk 一个非常大的一个尺寸" = "L3 的 Mooncake（一个非常大的池子）"；"实际的 PPT 突" = "实际命中率（hit rate）"；"4% 的 k 个开" = "4% 的 KV（cache）" `(?)`。

第 87 页 | KV capacity for diff session cc（KV 容量公式与分级取舍）

> 拍摄 16:13:59 | 音频 1531 段偏移 2571 | 讲者 卢祥龙（NVIDIA 解决方案与工程）

> 注意：本页时间窗 `[42:49]` 起，前一段 `[41:14]`–`[42:49]` 录音缺失，`KV to keep = sessions × context × headroom` 公式与表中 cc=16/32/64/128 的逐行数字**没有听到**。

- **口头要点**
- [42:49] 讨论另一种"加容量"的做法：把**所有 CPU 都给到一个非常大的 L3（Mooncake）池子**里。

- [42:53] 这样做有好处：这部分的 KV 是**所有节点、所有 DP rank 可见**的，基本上不太可能出现（前缀）不命中的情况。
- [43:07] 但也有缺点（"推倒"≈代价/短板 `(?)`）：
- [43:13] 如果走 **L2**，本地或异地 L2→L1 的搬运会比较快；而走 **L3**，这部分 KV 很可能命中在**别的节点的 CPU** 上，需要有一个类似 **master 进程**的查询过程，这个过程本身有开销。
- [43:36] 其次，其它节点 CPU → 你的节点 CPU 的 **RDMA 传输也有开销**，所以在速度和"来电性"（延迟/局域性）上会慢一点——**两种方式各有优劣。**
- [43:45] 第二个问题：**decode 的 concurrency 比较高**，它决定了实际输出吞吐；这就和 TP8 与 DEP8 的选择结合起来。
- [44:05] TP8 一个节点的容量比 DEP 少将近 **8 倍**，能承载的并发（"30o"`(?)`）比 DEP 少很多，会在高并发下**极大限制 output 吞吐**。
- [44:30] 左下角那张图能看出来：灰线（TP8）到了 **cc=32 之后出现雪崩效应**。
- **口头补充（幻灯片上没有的）**
- 讲者给的取舍很实用：**L3 大池子换"几乎不 miss"，但付出"跨节点查询 + RDMA 传输"的延迟代价**；L2 则换"搬运快"但容量有限。这是一个明确的 latency vs hit-rate 权衡，不是单纯"越大越好"。
- "decode concurrency 高 → 决定实际输出吞吐"这句把容量问题接到吞吐问题上，是第 88 页"Goal 2"的引子。
- 页面上 `staging only` / `buffer_only · no host cache mode · today` 两种模式和 cc=16→128 的 KV to keep 增长（7.2M→57.6M token）在时间窗内**没有讲** `(?)`。
- **ASR 存疑**
- 开头"等 0,但是我会把所有的 CPU"疑为 "G2 等 0（不用 G2），把所有 CPU 给到 L3" `(?)`——前半句在录音缺口中。
- "有一些推倒"疑为"有一些代价/短板"；"它的来电性上会慢一点"疑为"延迟/局部性" `(?)`；"能承载的 30o 的并发"疑为"能承载的 **300（或 33）**并发" `(?)`。
- "第一批 8" = **DEP8**；"附近的一个版本"疑为"单个节点/一个版本" `(?)`。

第 88 页 | Goal 2: decode concurrency must keep up（雪崩成因与 hard affinity 的代价）

> 拍摄 16:16:25 ｜ 音频 1531 段偏移 2717 ｜ 讲者 卢祥龙（NVIDIA 解决方案与工程）

- **口头要点**
- [45:14] 紧接着解释雪崩：会有很多请求需要做 **recompute（"rp"）**。
- [45:26] 于是会看到 **P（prefill）节点一直满、一直满载运行**，但它其实只在做 recompute，**没有真正在做有用的活**。
- [45:38] 结果是：**input 的吞吐基本没有变化，但 output 吞吐降得非常厉害**——算力就白白浪费了。
- [45:46] 所以这是 decode 卡点与运行方式选择（TP8 vs DEP）的一个关键问题；DEP 就没有这个问题，它**是一个线性的增长**。
- [46:08] 但还有很多工作要在上层做（harness / 上层调度这一层）。

- [46:18] 举例最简单的一种方式就是 **hard affinity（"哈的听听"，讲者口误成"CT 赛事"`(?)`）**：为了拿到高 KV 命中，第一轮先做 **KV-aware（"kbv 尔"）**，把 4 个节点先用（KV 感知）摊平。
- [46:40] 第二轮怎么办？后续的这些请求都还是打到原来这个路由上——这样 KV 命中一定是最高的，**但它会造成某些 rank （"股票"≈rank/worker）过热**。
- [46:54] 过热原因：session 的轮次差异很大，**有的 10 轮就结束了，有的 200 轮才结束**，一直钉死在同一个 worker 上就会过热。
- [47:11] 第二个原因：每轮的 ISL 不一样，**平均 ISL 方差特别大**，就会造成任务维度上的负载不均。
- **口头补充（幻灯片上没有的）**
- 讲者用"P 节点满载但只在重算"这个现象解释"为什么高并发时 input 吞吐不变、output 掉得厉害"——把**算力被浪费**讲得很具体，是这一页最有说服力的部分。
- 对 hard affinity 的评价是**双向的**：命中率最高（好），但换来 rank 过热与队头阻塞（坏），并且过热有两个独立机制（轮次长度差异 + 每轮 ISL 方差），不是单一参数能调好的。
- 投影片上的具体数字（TP8 `1.26M tokens/node`、`ceiling 33 requests in flight`、DEP `7.6M (6x)`、`scales to 100+ sessions`）时间窗内只讲了"少近 8 倍 / 线性增长"，**没有逐条念数字** `(?)`。
- **ASR 存疑**
- "rp/kp 开始" = recompute / KV cache；"input 子的思路" = "input 吞吐"；"out 的会降得非常厉害" = output 吞吐 `(?)`。
- "哈的听听就是 CT 赛事"：**"CT 赛事" 不明**，按上下文记为 hard affinity（讲者自己随后解释为首轮 KV-aware + 后续钉住原路由）`(?)`。
- "股票（"股票"）的过热" = "rank / worker 过热"；"题目的下的负载"疑为"任务维度上的负载不均" `(?)`。

第 89 页 | Hard affinity 模式下 DEP 的负载均衡问题（32 个 dp rank 的热力图）

> 拍摄 16:18:21 | 音频 1531 段偏移 2833 | 讲者 卢祥龙（NVIDIA 解决方案与工程）

- **口头要点**
- [47:21] 指着热力图：左边一共 **32 个 dp rank**，**红色那些就是过热的 rank**——这对系统来说比较不好。
- [47:33] 回到路由策略：讲者说自己之前也提过一些想法，但现有做法有问题。
- [47:45] 问题在于现有 KV-aware 路由**不知道这些请求属于同一个 session**。
- [47:51] 所以第一步是让路由**先建立 session 的概念**：把这些请求的 **session ID** 传下去（"我们把这些 s 选 ID...首先要让系统建立起 session 的概念"）。
- [48:02] 有了这个信息之后，最简单的就是做 hard affinity（"印章"≈绑定 `(?)`，即前面说的 sticky 亲和）。
- [48:14] 但要解决过热问题，于是提出 **soft affinity**：router 只做**建议**，请求仍然在你这个 worker 上一直执行，**直到**（压力/队列条件满足）时再考虑 reroute。
- **口头补充（幻灯片上没有的）**

- 讲者把"传 session ID"当成**前置条件**而不是优化项：路由必须先知道"这些请求是一个会话"，才谈得上亲和与否——这一步是和 harness 层（第 82 页 `X-Dynamo-Session-ID`）配套的。
- soft affinity 的定位很清楚：**把绑定从"硬约束"降级成"建议"**，仅在 worker 过载/排队恶化时才失效，从而在命中率与均衡之间取得中间态。
- 热力图的量化指标（`Minutes at ≥90% (of 61)`、`red = ≥90% full`、`c64 · 3600 s` 时间窗）在时间窗内只讲了"红色=过热"，**具体分钟数没有念** `(?)`。
- **ASR 存疑**
- "一共有 32,在左边有一共 32 个 dpr" = 32 个 dp rank；"这些红色的话就是这些 dpr 它是过热的" 清晰。
- "在老体策略上"疑为"在路由策略上"；"有没尔这是我们之前聊过的"疑为"KV-aware 这是我们之前聊过的" `(?)`；"啊印章，连他的冰"疑为"硬绑（sticky）/hard affinity" `(?)`。
- `[48:30]-[48:40]` 一段（"谢谢谢谢谢谢"、"包括你们不是这个请求队列的时候"）严重失真，**soft affinity 的触发条件具体是什么无法确认** `(?)`。

第 90 页 | Goal 3: Agentic Routing Strategy（KV-aware / Hard / Soft Affinity 与 KVCR）

> 拍摄 16:19:43 | 音频 1531 段偏移 2915 | 讲者 卢祥龙（NVIDIA 解决方案与工程）

- **口头要点**
- [48:43] "那我过热了以后那怎么办？"——把这个请求**绕到下一轮**（reroute）到（8 个里）**负担最轻的一个** rank。
- [48:55] 同时还要把之前已经算好的那部分 **KV cache 同步过去**——因为肯定不希望请求过去了还要重算这部分。
- [49:10] 这就是最近提出的 **KVCR** 概念（讲者口误为 "PPC" `(?)`）：**路由换 rank + 前缀 KV 跟着搬。**
- [49:14] 也可以选更简单的做法：我们有一个 **G3 池子**，这些 CPU 的内存互相可见、**互相能 RDMA P2P 传输**，把它们**注册到同一个池子里。**
- [49:31] 这样在请求被绕到其它 rank 之后，它对应的 KV 也能**非常快地被拉取到**——这是第二种做法。
- [49:42] 然后第四个（Goal 4）：要有一个很好的**准入机制**（"阿童一选"≈admission `(?)`）。
- [49:54] 它要解决刚才 TP8 在**高并发下 output 雪崩**的问题：请求在不断进来，不断被打断、退回队列、再重新（prefill/recompute）。
- [50:13] 想法：**存储已经打满、已经用了 95% 的时候，就不要再往里塞新请求了**；能不能 **pause** 一些请求、让它们等一等；等存储空间缓过来，再把它们（resume）送进来。 [50:40] 那怎么决定哪些请求该被暂停？
- **口头补充（幻灯片上没有的）**
- 讲者对"负载均衡 vs 前缀复用"给的**两条落地路径**很具体：① KVCR = reroute 时把前缀 KV 一起搬（NIXL P2P）；② 共享 G3 池 = 让所有 CPU 通过 RDMA 互见，reroute 后直接从池里拉。后者工程更轻，前者更省一次跨节点拉取。

- 准入的思想被讲成"**在容量已满时靠暂停换取不雪崩**"，而不是拒绝服务；95% 这个门限是口头明确给出的。
- 投影片上的三列策略对比表（每轮打分 vs 首轮定终身 vs 建议+过载值）与 `FlashIndexer` / `BinaryHeap` 等实现细节时间窗内**未逐条展开** `(?)`。
- **ASR 存疑**
- "绕到下一轮这这个 8 个客" = reroute 到（8 个 rank 中的）最轻的一个 `(?)`；"让上算我们的这些 PP 开始同步" = "已算好的 KV cache 同步" `(?)`。
- "PPC / Pvcr" = **KVCR** `(?)`；"阿童一选工作"疑为 "admission 准入" `(?)`；"TP 包含卡瑞 t8" = "TP8" `(?)`。
- "POS 暂停一些请求" = pause；"进入安全"疑为"（重新）admit 准入" `(?)`。

第 91 页 | ThunderAgent 的暂停对象选择（打分决定谁先让位）

> 拍摄 16:22:06 ｜ 音频 1531 段偏移 3058 ｜ 讲者 卢祥龙（NVIDIA 解决方案与工程）

- **口头要点**
- [50:57] 先停哪一类？如果它的 LLM 调用**不在 GPU 上**、而是在 CPU 侧（在等 tool），那么这部分历史 KV 现在**根本用不到**，就没必要留着 → 可以把它 **pause 进队列**。
- [51:13] 但正在（GPU 上）decode/生成的**不能停**——因为它正在做生成，停了就白算。
- [51:24] 第二个维度是**每个 session 已积累的历史长度**：有的 40K、有的 120K、有的（在做 tool 的）只有 30K。
- [51:33] 那肯定先停这个只有 **30K** 的：它又在做 tool（"突破"），历史又最短，**停它代价最低**。
- [51:41] 所以会**不断对所有 session 打分**，看"停掉它的代价是不是最低"，然后**一次性把该停的全部停掉**。
- [51:54] 再在**合适的机会**把它们送（resume）回来。
- **口头补充（幻灯片上没有的）**
- 讲者给的打分直觉是"**重算代价最小者先牺牲**"，并落到两个可观测维度：**是否在等 tool（当前不使用 KV）** 与 **上下文长度（重算要多少 token）**。
- 强调是"一次性批量停"而非逐个停，说明实现上按 tick 批量决策（与第 92 页 5 s tick、`0.80/0.85/0.95` 门限一致，但时间窗内**没有念这三个门限数值** `(?)`）。
- "打得起来再送回来"体现了 pause/resume 是**可逆的租借**，不是丢弃。
- **ASR 存疑**
- "不 GPU 上的 lm 口你头影 CP 上"疑为"（会话的）LLM 调用不在 GPU 上，而是在 CPU 上（等 tool）" `(?)`；"你如果在用个 CP 上去报的话"疑为"跑在 CPU 侧" `(?)`。
- "把你 ps 掉了，对列去好了" = pause 掉、退回队列；"我肯定不能停你我不能停止你，因为你正在做保先生"疑为"正在做生成（decode）" `(?)`。
- "历史只有 30 天" = "30K（token）"；"被创作的测试"疑为"benchmark 测试" `(?)`。

第 92 页 | ThunderAgent 实测效果（对 LRU/SLRU 的对比与 45/77 次进出）

> 拍摄 16:23:00 ｜ 音频 1531 段偏移 3112 ｜ 讲者 卢祥龙（NVIDIA 解决方案与工程）

- **口头要点**
- [51:58] 做了一些 benchmark 测试：**在比较高的并发下**，当存储空间产生"挤压式"的雪崩效应时启用这套准入。
- [52:14] 它比 SGLang 内置的 **LRU 或 SLRU** 这类策略**效果明显更好**。
- [52:21] 无论是 **TTFT** 还是 **TPOT**（"T 两 t" `(?)`）都会有很好的效果，**主要是避免雪崩**。
- [52:33] 讲者口头确认"这代表它效果很好"，并给出实测：**在一个小时的测试中，准入机制 pause 了 45 次，resume 了 77 次**（"pass 45 次，然后送 77（次）" `(?)`）。
- [52:51] 再说准入与 KV offload 结合的好处：通过对 session 打分，其实已经得到一个**很抽象的结论**——下一轮调度时**哪些 session 重要、哪些不重要**。
- **口头补充（幻灯片上没有的）**
- 45 次 pause / 77 次 resume 是**一小时窗口内的控制回路工作量**，说明门限触发频繁但可恢复（resume 次数约为 pause 的 1.7 倍），这是讲者用来证明"不会抖成死锁"的实测证据。
- 讲者把"打分"的产物定义成**跨模块的中间结论**（重要/不重要标签），从而自然接到下一页"把不重要的下放到 G2/G3"。
- 投影片第 93 页那组收益数字（e2e −51%、TTFT p50 −87%、TPOT 减半、吞吐 ×1.6）在这一时间窗内**没有念** `(?)`。
- **ASR 存疑**
- "在安全安的来 NC 啊 T 两 t 啊 t"疑为 "TTFT 啊、TPOT 啊" `(?)`；"他会 pass 45 次，然后会是送 771 度"疑为 "pause 45 次、resume 77 次"（"771 度"极可能含混）`(?)`。
- "s 上的这种内置的 lu 或者 sru" = SGLang 内置的 **LRU / SLRU**；"比较高的他 r 下" = "比较高的（并发）cc 下" `(?)`。
- 本页标题（ThunderAgent · how it decides）与音频内容不完全对应：录音讲的是**实测结果**，`S = 1/c + 1{ACTING}·2^(-t)` 打分公式与 `wait > 1800 s → forced resume` **没有听到** `(?)`。

第 93 页 | ThunderAgent Benchmarks（SWE-Bench Lite 上的收益）

> 拍摄 16:24:12 ｜ 音频 1531 段偏移 3184 ｜ 讲者 卢祥龙（NVIDIA 解决方案与工程）

- **口头要点**
- [53:03] 顺着上页"哪些 session 重要/不重要"的结论继续：可以把**不重要的 session 预先下放到 G2 或 G3**（更多层级）。
- [53:19] 并**提前（prefetch）往下放**；同时把**很重要的 session 往 GPU（G1）提**上去。
- [53:27] 这就是同时使用（准入 + offload）的做法。
- [53:30] 最后：这套**准入机制和 KV offload** 在 **RL 的 rollout 阶段**也有应用。
- **口头补充（幻灯片上没有的）**

- 讲者描述的其实是**双向流动**：不重要的往下"预先"搬（避免被动逐出），重要的往上提；这解释了第 94 页那句"ThunderAgent 自己不搬 KV，只是改变谁变冷"。
- 增益数字在这一时间窗内只到"有应用"，具体百分比在下一窗口才出现（见第 94 页 `2.5 倍`）。
- 第 93 页投影片的四大指标柱状图（TTFT p50 / TPOT p50 / gen 吞吐 / e2e）与 `B·TA / A·LRU / SLRU` 三组对照，录音里**没有逐个念数值** `(?)`。
- **ASR 存疑**
- "把这些不重要的赛事去往 g 二或者 G3 的情况" = "session 下放到 G2/G3"；"一些顶是吧"疑为"一些（提前）预取" `(?)`。
- "kp 的老师"疑为"**KV offload**（offload 被听成"老师"）" `(?)`。
- 时间窗偏短（仅约 50 秒），第 93 页页面上的全部数值标签均未在录音中出现 `(?)`。

第 94 页 | ThunderAgent + KV offload（三层之间谁冷谁热）

> 拍摄 16:24:54 ｜ 音频 1531 段偏移 3226 ｜ 讲者 卢祥龙（NVIDIA 解决方案与工程）

- **口头要点**
- [53:45] 我们把 **Dynamo 作为 VeRL 的 rollout 后端**，并**已经做了一个投入/落地**（"已经进行了一个投入"`(?)`）。
- [53:53] 可以看到：在比较高的并发下，**KV offload 加上准入**，相比他们**几个月之前**的实现，大约有 **2.5 倍**的速度提升。
- [54:11] 这对 RL 的 rollout 非常好，因为真正的 RL rollout **长得很像 fanout**（"粉丝"`(?)`）。
- [54:20] 它其实占了整个 rollout **约 70% 的时间**（"占了整个 ro 的 70% 的时间"`(?)`）。
- [54:25] 它又很像 fanout，主要追求的目标是**吞吐**，而不是 TPOT 这类单请求时延指标——所以这种编排策略能在 rollout 上取得好效果。
- **口头补充（幻灯片上没有的）**
- 讲者用"**RL rollout 像 fanout、占 70% 时间、追吞吐不追 TPOT**"来解释为什么这套"准入 + offload"在推理服务里是避免雪崩、在 rollout 里却是**直接的吞吐收益**——同一机制两种价值，这是本页最有信息量的判断。
- 2.5 倍是**与自家几个月前实现**对比的口径（不是与第三方 baseline），讲者同时说明已投入实际使用 `(?)`。
- 投影片的 program table（`A decoding → G1 95%`、`B waiting tool → G2 60%`、`C offloaded`、`G3 一份共享副本`）与结论句 `TA never moves KV · it only changes who is cold` 时间窗内**没有逐条念** `(?)`。
- **ASR 存疑**
- "把 demo 作为 world 的一个 r 后端" = "把 **Dynamo** 作为 **VeRL** 的 **rollout** 后端" `(?)`；"已经进行了一个投入"疑为"已经投入（生产/内部使用）" `(?)`。
- "司机第三会有 2.5 倍左右速度的提升"：**"司机第三"不明**，按上下文指"相比几个月前的实现" `(?)`。

- "他长得非常像粉丝" = "像 **fanout**" `(?)`；"占了整个 ro 的 70%"疑为"占整个 rollout 的 70%" `(?)`；"主要的追求目标是思路" = "吞吐" `(?)`。
- 本页照片（16:24:54）时间窗内容实际已进入第 95 页（RL rollout）的讲解，故本页"热点"是 RL 应用而非三层示意 `(?)`。

第 95 页 | ThunderAgent in RL rollout – Dynamo VeRL Integration（+16.5% / +6.8% 与仿真在研）

> 拍摄 16:26:04 ｜ 音频 1531 段偏移 3296 ｜ 讲者 卢祥龙（NVIDIA 解决方案与工程）

- **口头要点**
- [54:39] 最后是资源与未来工作：**左边是相关的 PR**（"我们写一些...报告" `(?)`），对应刚才讲的这些知识点的细节，可以去翻。
- [54:56] 右边是未来要做什么。第一个工作刚开始：就是刚才讲的**仿真（simulation）**，我们叫 **Agentic Infra 的仿真**（"aic 的 a 型" `(?)`），感兴趣可以看一下。
- [55:08] 就是整套这种编排可以比较多维度地达到"安全执行"（`(?)`），做一个**闭环的、高保真的仿真**。
- [55:17] 它可以指导你：在**运行配置**、**KV 多层级（offload）操作的策略**、以及**存储空间大小**（"数字空间大小" `(?)`）上给出一些建议。
- [55:31] 第二个方向是在**策略上的进一步改进**：刚才提到的 soft affinity 加上 **KVCR**（P2P 传输与高绑定结合的算法）。
- **口头补充（幻灯片上没有的）**
- 讲者把仿真定位成**部署前的"设计工具"**：不解决线上调度，而是在你选配置、选分级策略、定各层容量之前给建议——这是对第 86/87 页"容量该设多大"那个问句的正面回答路径。
- 策略方向上，讲者把 soft affinity 与 KVCR 明确配成一对（建议式绑定 + KV 跟着搬），作为 hard affinity 过热问题的后续解法。
- 投影片上的曲线与数字（三条曲线 `Default LB` / `Dynamo Python TA` / `Dynamo native Session Aware`、`+16.5%` / `+6.8%`、`Default LB collapse`、`cc 32→512`、`Qwen3-Coder-30B-A3B`、`8x Hopper`）在这一时间窗内**均未念到** `(?)`。
- **ASR 存疑**
- "c 闻 aion / aic 的 a 型"：**无法确认全称**，按上下文记为 "Agentic Infra 的 simulation" `(?)`。
- "我整套这种编排可以比较多位达到我的安全执行"疑为"多维度地达到…（安全/可靠执行）" `(?)`；"数字空间大小"疑为"**存储空间大小**" `(?)`。
- "骚作的冰体" = **soft affinity**；"Pvcr / ppppp 传输" = **KVCR / P2P 传输** `(?)`。

第 96 页 | Assets & Future Work（收尾页 + 现场问答）

> 拍摄 16:26:59 ｜ 音频 1531 段偏移 3351 ｜ 讲者 卢祥龙（NVIDIA 解决方案与工程）

- **口头要点**

- [55:49] 关于**三级存储（KV 分层缓存）**：它在**高并发下很有用**，但在**中低并发没有什么用**——因为压力不够，根本不会"打爆"（不会 OOM/雪崩）。
- [55:58] 所以我们在想：在**中等压力**下，这种准入策略能不能有更好的优化。
- [56:07] 最后一个方向是**可编程的 KV block hints**：不同的 session、不同的 block，**优先级不同、重要程度不同**，而我的存储空间是有限的。
- [56:13] 所以我会把**不重要的往下层放，或者提早丢弃**；**重要的往上提**（往 GPU/G1 提）。
- [56:29] 这些是我们目前看到的未来几个工作。
- [56:35] 收尾："Ok，那我今天的分享可能就到这里，非常感谢大家"，并说"还有两分钟"，进入提问。
- [56:42] **（Q&A / 收尾）** 现场第一个问题（提问人语速快、录音失真）：指向前面某张 PPT / 某个"产品"相关的指标（"合会"`(?)`、"目标二"`(?)`），追问"这个问题大概是怎么解决的？"
- [57:18] **（Q&A / 收尾）** 讲者回答：**（soft affinity）加上 KVCR**，是目前最有可能（解决）的方案 `(?)`。
- [57:39] **（Q&A / 收尾）** 讲者："看看还有没有其他的问题。"
- [57:49] **（Q&A / 收尾）** 第二个问题：关于"单个 worker / 会话（'志愿'`(?)`）如何通过现有（系统）接入"的问题，讲到一半转成"你们要方便（的话）加微信"——**录音在此结束（57:59）**。
- **口头补充（幻灯片上没有的）**
- **（Q&A / 收尾）** 三级存储的适用边界是讲者主动交底的一条经验：**它是高并发的工具，中低并发下没有价值**（不会触发逐出/雪崩），这直接解释了为什么"更先进的准入"要针对中等并发区间（对应投影片 `ThunderAgent v2 · deal with moderate cc`）。
- **（Q&A / 收尾）** 可编程 block hints 的动机是"**存储空间有限 → 按 block 重要性做分层与丢弃**"，与第 82 页 harness 传 hint 的思路首尾呼应（第 82 页透传"哪些前缀重要"，这里变成 router 下发 `Share/Prefetch/Demote/Pin/Retain`）。
- **（Q&A / 收尾）** 现场互动到此为止：问答只剩约两分钟，第一个问题讲者用"soft affinity + KVCR"一句话作答，第二个问题未及展开就转为线下沟通（加微信）。**任务说明中提到的 57:49–61:00 更长问答/分会场收尾，在本段录音中只捕捉到开头两条**，录音文件止于 57:59 `(?)`。
- **（Q&A / 收尾）** 投影片上的 Docs 链接（Flash Indexer Digest、ThunderAgent Program Scheduler User Guide）、三个 PR（`#11447` / `#11787` / `#11616`）、`verl-recipe` 与 `Soft affinity mode + KVCR`、`SGLang RFC #27574` 等条目，讲者只笼统说了"左边是 PR，可以去翻"，**未逐条念** `(?)`。
- **ASR 存疑**
- "三类阵"按上下文判断为 **三级存储（KV 分层缓存）** `(?)`；"根本就不会 p 死" = "不会打爆 / OOM" `(?)`。
- "这可可编程这种 PP 报嗯思" = "可编程的 **KV block hints**" `(?)`；"不同三选预报"疑为"不同 session 的 block" `(?)`。
- Q&A 段（`[56:42]`–`[57:32]`）几乎不可辨：`"合会"、"最后一个 p"、"目标二"、"再往前冲 200"` 等词**均无法可靠还原**，提问所指的具体图表/指标不明，标 `(?)`。
- "那个超的 p 体呃加上 PPC" 疑为"**soft affinity 加上 KVCR**"（也可能是 hard affinity），标 `(?)`；"最多都是认可"疑为"目前最有可能（的方案）" `(?)`。

- "单个的那个沃克的或者志愿的东西" = "单个 worker 或者 session" `(?)`；"如何通过那个现有的那个 MM"后接"加微信"，**接入方式的答案没有录到** `(?)`。

这一段（81–96 页，约 30:14–57:59）最大的特点是**"容量归因"式的工程叙事**：讲者少念数字表格，多用一条因果链把结论讲通——ISL 从 448 涨到 219K 定义问题，55.9 KB/token 把"1 GB 装多少 token"变成 TP8（2.2K、ceiling 33）与 DEP（17.9K、6x）的选型判据，G3 只装得下 41 分钟写入、命中率 95.8%→91.9% 说明瓶颈在容量而非算法。其次是口语比喻密集（"惩罚的购置税""打爆""雪崩""想完就丢"），并常用现场观察替代指标推导："P 节点满载却只在重算、input 不变而 output 掉"。三是坦诚交底取舍：hard affinity 命中最高但会过热、L3 大池子换来几乎不 miss 却付出跨节点查询与 RDMA 延迟、三级存储在低并发无用。收尾极快，问答两分钟即转线下。

六、对《内容整理与总结分析》的修订清单

录音带来的是"讲者视角"的一手信息，因此对前一份只凭投影片整理的文档提出以下修订与增补。**凡不影响原有技术结论的，只做增补；影响表述准确性的，明确标为"修订"。**

6.1 需要修订的表述

#	原文档位置	原表述	依录音改为
1	摘要、第二章	"五段主题"	**"4 场分享 / 5 位讲者"**（第 1 场为孙方宇、张毅接力，第 2 场白洪潇，第 3 场徐文杰，第 4 场卢祥龙）
2	第二章 2.1	把第 68–78 页（KV Router / Flash Indexer）与第 79–96 页（智能体）当作两个部分	两者同属**第 4 场**（Dynamo 主题）；内部只是从"路由与索引"过渡到"智能体会话调度"，用一页 `SLAs of New Agentic Era` 转折
3	第五章 5.2 判断二	"训练侧的结论与推理侧的结论是互通的…本质是同一个命题在两种负载上的两次求解"	概念相通成立，但讲者明确说过 **"推理方向可能完全不一样，这些东西没法直接迁移过去"**（第 2 场问答）。应改为："思路可迁移，**实现与参数不可平移**"
4	3.1.4 数据段	只给 geomean 1.16× / 峰值 1.35×（per-tensor）、1.30× / 1.72×（blockwise）	补充讲者**口头给出的档位数**："小 token 档位约 **3%**、大 token 档位约 **24%**、blockwise 在 32768 档 **72%**"，并说明原因是"小 token 时通信与 launch 主导、两种配置都是 memory-bound"
5	3.3.1	"压缩没有直接的 TPOT 收益"	补充问答中的量化解释：**解压算子占用 SM、与推理争用**，实测"解压带宽约 50 GB/s、解压很快，所以占用时间短"，但**并发越高越不稳定**（吞吐波动约 **−5.6% ~ +5%**）
6	1.1 资料来源	未提录音	增加"同一现场另有 4 段录音（约 4.9 小时）及讯飞转写结果"作为资料来源与交叉验证手段
7	附录 D	文件清单	增补录音转写文件与本次新增的讲稿整理文档

6.2 口头补充的事实（原文档未收录）

1. **会议身份**：这是 **NVIDIA 开发者日（Developer Day）中国站**的分会场，主题为"**大语言模型训练与推理**"；分会场共 4 场分享，收尾语明确为此。原文档只能凭页脚 NVIDIA 标识与 `gtc26-*` 链接推测，现已由录音确认。
2. **讲者名单**：孙方宇、张毅、白洪潇（NVIDIA Megatron 团队）、徐文杰、卢祥龙（NVIDIA 解决方案与工程）。三人单位未在录音中明确说出，故不推断。
3. **CUDA Graphs 的代价**：开启会带来"几 G 甚至十几 G"的额外显存消耗，与显卡型号无关，取决于剩余显存余量；可能 OOM（第 2 场问答）。
4. **FP8 Indexer 精度的验证范围**：讲者承认"非常长时间的训练确实没有做过"，短期测试性能影响不明显；DeepSeek 官方参考实现"没说用什么精度"，属不确定状态（第 2 场问答）。

5. **KV 压缩选型三步法**：① **先看精度**（KVTC 计算重但精度更高，8× 的 KVTC 精度优于 3.6× 的 TurboQuant）；② 再看算法形态（存储型 / 计算型、能否与 attention 融合）；③ 最后看优化目标（HBM 压力 / 内存容量压力 / 系统 I/O 压力）（第 3 场问答）。
6. **甜点区**：TurboQuant 极限约 **4.5×**、甜点约 **3.6×**；KVTC **8×–64×**、**8×** 较合适；压缩的"最性感场景"是**超长上下文**（第 3 场问答）。
7. **KVTC 的集成细节**：集成在 KV 缓存管理后端，映射变换完全动态完成，**不需要模型改动**；团队尚未开源，仅可定向提供（第 3 场问答）。
8. **Dynamo 相关策略的版本门槛**：开源项目里"有对应设置，但比较新"，需更新到 **1.5 / 1.5.1 以上**才能开启（第 4 场问答）。
9. **KV 分层建议**：与其纠结各种淘汰策略，"比较好的做法是把本地 L2（CPU 缓存）设得很小甚至 0，用一个很大的共享 CPU/SSD 池，所有节点互相可见，这样**不会有缓存不命中**"（第 4 场问答）。
10. **跨硬件可移植性**：KV 压缩的收益曲线在 NVIDIA GPU 上基本一致；非 NVIDIA 硬件需按数据访问/合并特性重做，且对**数据对齐**敏感（第 3 场问答）。

6.3 只出现在口头、投影片完全没有的内容

- 第 1 场开场用约 1.5 分钟"复习 MoE 是什么"：传统 MoE 推理中 FC1/FC2 与通信使用不同资源但被**分开调度**，所以没有重叠；后续多卡并行只做到**有限重叠**；因为 FC1 远大于 FC2 而 dispatch/combine 的量级与之不匹配，**单条流水线式的融合并不高效**，需要"用一个 scheduler、把数据切成更小单位、做更细粒度流水"——这正是 MegaMoE 的出发点。
- 第 1 场口头确认了 `W4A4/W8A8/W16A16` 与 `NVFP4/MXFP8`、`FP16` 之外还提到"子通道"精度与 **NVL72、基于 NVLink 的 RDMA** 方向的探索（与投影片的 support matrix 呼应，但更具体）。
- 第 4 场提到调度策略在 **RL rollout 阶段**也可复用（与投影片第 95 页的 VeRL 集成呼应）。
- 各场主持人用语确认了场次编号（"今天我们 4 场分享已经结束"）。

附录 A：上午场纪要

（据 `录音机-20261016-0945.wav`、`录音机-20261016-1103.wav` 与**补充的 4 段 4K 现场视频** `VID_20261016_105447/110133/110211/110316.mp4` 的转写整理；**无对应投影片照片**，故仅作背景参考。）

素材覆盖已更新：原先 09:45–10:39 与 11:03–12:10 两段录音之间存在约 **24 分钟**空档；本次补充的 4 段视频覆盖 **10:54:48–11:07:01**（互有重叠），因此空档缩小为 **10:39:37–10:54:48（约 15 分钟）**，且 **11:03–11:07 与录音二重叠**、可互为校对。其余限制：转写只在**段落级**给出起止时间（正文 `[分:秒]` 有按比例估算成分，标 `≈` 者更不确定）；人名存在多种写法（"刘明玉/刘明宇""安迪/An Patel""吴自华/吴子华""张家驹/张家军""朱立根/朱立登""姚勉/杨敏"），本纪要取较可能的一种并保留异写。

A.1 补录段：主论坛英文演讲 10:54:48–11:07:01（据新补视频）

这段正是此前缺失的"Brian（Bryan Catanzaro，NVIDIA ADLR 副总裁）"演讲的开头与主体。讲者自述"多年前曾在百度工作"，以"AI 是当下最大的计算挑战"开场，主旨是把**计算转化为智能**，并给出他看到的**四个仍在推进的 scaling law**：

#	scaling law	讲者的说法（转写要点）
1	**预训练**	用全互联网、全语言、全领域的数据训练，模型更大、数据更多、并用 AI 筛数据提纯；讲者强调"**数据变聪明，让预训练（同等硬件）快了 4 倍**"，并把预训练数据集公开以加速他人
2	**后训练**	"这是一个**巨大的加速计算问题**"：要让智能体在**百万级 sandbox** 与各类环境中训练，其中一些环境**需要内置模拟器**，模型才能更好地理解世界
3	**测试时计算**	"模型算 token 很快，但回答一个问题用掉太多 token 就是浪费"——**回答一个问题需要多少 token，本身就是加速计算问题的一部分**；他还提到受 **GLM 5.2 → 5.3** 的进展启发
4	**（跨段中断）**	第四条的完整表述落在两段素材之间，据前后文疑与**数值格式 / 稀疏性带来的效率**有关（"大部分训练已经用 8 位算术，等于 4×"），标 `(?)`

其余要点： - **AI 不是"一刀切"的问题**，需要不同规模与能力的**模型系统**，还需要**专门化（specialized）的 AI**——"把某件事做到极好，本身也是一种效率"。 - 因此**必须开放 AI 技术**。NVIDIA 的 **Nemotron** 不止给权重，还给**数据、环境、构建用的软件与配方（recipes）**——"不只是 weights，而是 weights + data + environments + recipes"（与上午另一场"open = 权重 + 数据 + recipes"的说法一致）。 - **Nemotron 有两个工作**：① 帮 NVIDIA 自己想清楚"加速计算 + AI 的端到端问题"；② 让整个生态更高效。 - 一个具体因果：**Blackwell 的 NVLink-72（72 卡全速互通、低延迟）之所以这样设计，是因为他们判断 MoE 会改变 AI 的构建与部署方式**——"我们走过这个过程、做出了 NVL-72，所以今天全世界构建与部署最好的模型容易得多"。 - 数值与稀疏性："我 20 年前开始做深度学习时几乎全是 FP32，现在大部分训练是 8 位算术，这就是 4 倍"。 - 视频帧 OCR 可辨认的幻灯片标题：`The Biggest Computational Challenge of Our Time`、`Accelerated Computing`（其余帧是讲者画面与现场中文字幕）。

> 说明：本节内容来自视频音轨的转写 + 抽帧 OCR（每 20 秒一帧，共 64 帧）；这场演讲是英文，现场有中文

> 字幕，因此转写是中英夹杂的。

A.2 上午两段录音的纪要（原稿）

> 依据：`转写结果/录音机-20261016-0945.md`（时长 53:47）与 `转写结果/录音机-20261016-1103.md`（时长 67:28）的讯飞语音转写文本，辅以 `_build/audio/speaker_scan.txt` 的关键句时间戳。**本纪要仅据语音转写，未经投影片核对**；所有时间戳为录音内相对时间 `[分:秒]`，两段录音分别对应绝对时间 09:45 起与 11:03 起。

1. 场次与讲者

录音一：09:45–10:39（时长 53:47，起始绝对时间 09:45）

时间（相对）	绝对时间	讲者	身份 / 主题
[00:00]–[01:30]	09:45–09:46	**Maggie**（麦琪(?)，音译）	开场致辞的收尾段（录音从中途进入）。单位未在转写中出现，只提到"nvidia 对中国开发者的承诺"，推断为 NVIDIA 侧主持 / 高管，姓名仅由串场词"感谢 maggie 温暖又激励人心的开场致辞"确认。主题：DGX 类平台、对中国开发者的支持、DLI 培训与开发者日
[01:29]–[01:37]	09:46	主持人（**姓名未识别**）	串场，感谢 Maggie，引出下一位讲者
[01:38]–[11:48]	09:46–09:56	**An Patel**（转写 `an patel` / `anie`，串场词称"**安迪**"，NVIDIA 开发者产品副总裁）	主旨演讲（英文为主）：CUDA 与 tile programming、开源生态、当日议程。三种写法不一致，见第 4 节 (?)
[11:49]–[13:06]	09:56–09:58	主持人（未识别）	串场："加速计算已经不仅仅是硬件的故事，更是一个全栈软件生态的故事"，引出世界模型主题演讲
[13:06]–[52:48]	09:58–10:37	**刘明玉 / 刘明宇**（转写不一致；NVIDIA **Cosmos Lab(?)** 副总裁，领导 Cosmos 平台）	主题演讲：世界基础模型 Cosmos 与物理 AI（physical AI）。串场词写作"刘明玉 nvi、cs ms la 副总裁"，结束语称"谢谢明玉"，scan 清单记作"刘明宇"；头衔原文"cs ms la"疑为 "Cosmos Lab" (?)
[52:57]–[53:47]	10:38–10:39	主持人（未识别）	串场：从"数据仿真到生成式世界模型的三位一体"转向开源 AI 生态，引出 **Brian**（转写"胡瑞" / "brain" / "b brand"，NVIDIA 应用深度学习研究 ADLR 副总裁，团队构建 cuDNN 与 Megatron）

录音二：11:03–12:10（时长 67:28，起始绝对时间 11:03）

时间（相对）	绝对时间	讲者	身份 / 主题
[00:00]–[18:15]	11:03–11:21	**Brian**（转写"胡瑞" / "brain" / "please welcome b brand"，NVIDIA 应用深度学习研究 **ADLR** 副总裁，团队构建 cuDNN 与 Megatron）	承接录音一结尾的介绍，英文演讲：Nemotron 3 模型族的数据飞轮、端侧模型、定制化案例，以及 Megatron Core 新特性。录音从中段进入（首句为"That data center..."），姓氏未出现 (?)
[18:19]–[01:06:56]	11:21–12:10	**圆桌讨论"从开放到可控"**，6 位嘉宾 + 主持人（未识别，疑为 **Julian**(?)，据录音一中 An Patel 的介绍"julian has a panel with the speakers..."）	见下方嘉宾表
[01:07:10]–[01:07:28]	12:10	主持人	收尾："从开放到可控不是一条直线，而是一个需要社区平台工程师共同参与的生态协作过程"

圆桌 6 位嘉宾（人名/单位均以串场词与自我介绍为准，转写前后写法不一致者标 (?)）：

1. **赖俊杰** — NVIDIA 工程和解决方案副总裁（仅见于圆桌开场串场词，姓名可信度较低 (?)）。
2. **韩晓**（自我介绍"我叫哈瑞韩晓"，亦被称"哈瑞"）— 来自 **Dify**（转写"低费/d费"，据其自称"他的名字也是 d费，就是 do it for you"，判断为 Dify (?)，公司名被 ASR 转成中文谐音），全栈工程师。负责用户侧/应用侧视角。
3. **吴自华**（亦作"吴子华"）— NVIDIA **GPU 计算专家**（自称"来自于 n威利亚泰克的吴子华"）。
4. **张家驹**（亦作"张家军"）— **vLLM 社区**大使，在国内组织多次社区活动，专长是"更快更好地产生 token"的基础设施。
5. **朱立根**（亦作"朱立登"、"李根"、"林哥"）— **NVIDIA Research** 研究员，带队做 agent 策略与 tokenomics 相关开发（自称"过去三个月 token 使用最多的人"）。
6. **姚勉**（亦作"杨敏"、"姚敏"、"杨远"）— **阶跃星辰（StepFun）** 开放平台 Harness 产品经理。

2. 各场主要观点

09:45–10:39 录音一

A. Maggie 开场（[00:00]–[01:30]）

- [00:00] 录音从致辞中段开始：把（加速计算）平台开放给开发者，"可以轻松部署到各种应用场景"，强调 NVIDIA 提供的是"全球最高产出、最持久耐用、最通用灵活的 AI 平台"（"pdf 平台"疑为 **DGX** 平台之误 (?)）。
- [00:10] 对中国开发者的承诺：开源模型、被称为"秘密武器"的 **DGX Spark**（转写"djx spark"），以及 **DLI** 培训（转写"doi/doy 的培训"）；"前天"在**苏州**开幕的（AI PC(?)/"o pc 融创中心"）把培训教室设在那里。
- [00:20] 结语：请开发者"坚持你们的创新以及梦想"，一起开创 AI 的美好世界。

B. An Patel / "安迪"：CUDA、开源与议程（[01:38]–[11:48]）

- [01:38] 这是他**第三次**在中国参加 developer day，第一场在一个大学里办、规模较小，很高兴看到"一些同样的面孔"。
- [≈02:20] 观察一：**用 AI 写代码已成默认**——"anybody I talk to... everyone's using AI to code, that's kind of a default now"。
- [≈03:00] 观察二：**推理（inference）需求爆炸**。原因不只是单个 agent，而是"多个 agent、多个 tool/lab 互相通信、每个都在消耗 token"，再加上视频生成等新应用。
- [≈03:40] 观察三：**每 token 成本在下降**（更多优化、更多 kernel），但需求同时爆炸——成本与需求赛跑。
- [≈04:20] 今天讲三件事：**CUDA、开源**，以及当天议程与"你会学到什么"。
- [≈05:20] CUDA 传统路径：C++/Python → 编译到 **PTX** → runtime 把 PTX 变成 GPU 上的字节码；并用现场举手调查"谁在做底层 kernel 优化"。
- [≈05:50] **CUDA 最大的变化：tile programming**。表达 AI 算法的方式本质上是"分块（tile）"，因此 CUDA 现在把 tile 算法作为一等能力，且可以用 **Python / C++ / Rust** 写 tile 程序。
- [≈06:40] 特别提到 **Rust**：agent 写的代码应当"可验证（verifiable）"，Rust 编译器和内存模型极严格——人写起来痛苦，但 agent 写出来"一编译通过就是安全的"，因此 Rust 在 agent 时代越来越受欢迎。
- [≈07:30] **RTX Spark**（上周发布）被定位为 CUDA 开发与部署的**一等平台（first class）**；最新 CUDA 版本为 **13.4**，新增对更多操作系统与编译器的支持。两个 takeaway：tile programming 成为一等公民；RTX 是很好的 CUDA 开发平台。
- [≈08:20] 澄清"做 CUDA 就不开源"的误解：**CUDA Python、CUDA Rust、CUTLASS**（转写"cut list"，GEMM 高性能库，即 AI 用的矩阵乘）、**NCCL**、**TileLang(?)**（转写"tylerre"）都是开源的。
- [≈09:00] GitHub 上 **1400+ 开源项目**，均为 MIT/Apache 许可、有 contributor 流程；NVIDIA 还付薪雇**专职维护者**每天为 **Kubernetes、Linux kernel、OpenUSD、PyTorch** 等上游项目贡献。
- [≈09:40] 模型层面：**Hugging Face 上 650+ 个模型**，按 family 组织；NVIDIA 的"open"是**权重 + 数据 + recipes（配方）**都开放，"pretty much reproduce most of what we've done"。
- [≈10:10] 举例：**Nemotron**（语言与 agentic）、**Cosmos**（物理 AI / 机器人）、**BioNeMo**（医疗与生物，提到"virtual cell"初创公司）、**Earth-2**（天气/海啸/飓风预测，开源以便与实验室合作）、最新的**量子计算**（"quantum computing dericing" (?)）模型族，另有 ASR/TTS 模型。
- [≈10:40] 议程：**Brian** 讲 Nemotron 与 Megatron 生态；**Julian** 主持的 panel（嘉宾来自 NVIDIA、StepFun、Dify 等，主题为 agentic 与"模型与控制"）；午餐后**四个分会场**。
- [≈11:20] 四个分会场之一是 **LLM training and inference**：讲 serving 模型、**MoE kernel**、inference workload 与模型 runtime；其余为 physical AI / 机器人、AI infrastructure（"如何拿到最低的每 token 成本"）等。口号："intelligence per token / lowest cost per token"。

C. 刘明玉 / 刘明宇：Cosmos 与物理 AI（[13:22]–[52:48]）

- [13:22] NVIDIA 是"**AI-ready computing company**"，最重要的事是造出最好的计算机/计算平台，"成为你强大的靠山"。列举要攻克的应用：enterprise AI、agentic AI、用 AI 写代码、digital twin、biology/life science、material、气候、robotics、manufacturing、robotaxi/自动驾驶——"我们想识破的远远超出这些"。

- [14:05] 做法是**从最底层往上建**：GPU/CPU/DPU/network device 并把它们高效连起来；再往上做 **library**（如 cuDNN 一类），"让你有效榨取这个算力平台上的每一分算力"；再往上是大量 AI 模型（含自研与世界模型）。
- [≈15:00] 除开源模型外还做 **reference 应用**，把自家 library 与别人的开源模型连起来跑通，以"切身体验做出来到底需要什么"。这么做的**两个原因**：(1) 更好地服务开发者——很多人有好 idea 但缺资源与团队，可直接用已有成果快速验证（"大部分一开始的 idea 都是错的"）；(2) 通过各垂直领域的经验与反馈，确保**未来的 GPU**（"未来的架构/未来选型"）更贴合需求。
- [17:08] 用一个比喻回答"为什么讲这个"：大家都相信 **AGI**（一个模型解决所有问题），他借**热力学与引擎**的历史给出**否定答案**——人类掌握热力学后造出引擎，但并没有"一个引擎搬所有东西"，摩托车要轻小、赛车要快响应、卡车要强拉力与耐久、公车要可靠并**支持轮椅（wheelchair）上下**。
- [≈18:50] 推论：AI 也会**按场景分化（specialization）**，"未来的软件就是会有很多 specialization"，同时一定会有很强的通用智能技术并存。
- [≈19:30] 他从事 AI 研究近 **20 年**、研究过上百个模型，最后业界普遍在用的只剩**两类**：**GPT 式自回归**（一次一个 token）与**扩散模型 diffusion**（先出噪声再多次重建）。共同点是**把困难问题拆成许多小问题**；一次性全生成的模型往往效果不好。
- [≈20:20] **Agentic AI 是新范式**：不再期待一个大 LM 把所有事做完，而是产生一份 **task list**，靠**工具验证**并逐步迭代；区别在于每个节点做的事完全不同（一个节点做 matmul/kernel、一个做 generation、一个只做简单 LLM 推理），因此"每一小步用不同方式解"。
- [≈20:50] 模型与软件架构会**持续 evolving**：过去人开发模型，现在 **agent 也能帮你开发模型**，迭代会越来越快、模型越来越多；因此坚持"做**可编程的 GPU**（programmable GPU）"是正确决策——换模型、换赛道时不必换卡。
- [≈21:00] **GPU 保值论**：因为持续投入软件维护、保证向后兼容，V100/A100/H100 至今可用，"过去的 GPU 现在反而比当时卖得还贵"，这在电子产品界极罕见。自嘲 NVIDIA 是"卖铲子的"，但"我们卖铲子还会自己先挖一挖，确保它真的能挖"。
- [21:13] 引用 **No Free Lunch（天下没有白吃的午餐）定理**："average of all distributions, all models are equal"——模型在不同分布上各有取舍，一个模型打天下就像"用卡车当公车、当赛车、当摩托车"，能拉但不高效。
- [≈23:00] 另一个极端（每个分布一个模型）也不合适，因为**维护与开发成本**过高；所以答案是在通用与专用之间取平衡。
- [≈24:00] 数字世界 vs 物理世界的差别：数字世界数据好收集（有 internet）；物理世界很多 **corner case** 恰恰收集不到。且云端可以把**多个用户的请求 batch 起来**把算力打满，而机器人/自动驾驶看到信号就要在**实时（"30Hz/3 万分之一秒"级别，转写作"h30 万" (?)）**内反应，无法 batch，因此大模型在这里"不是特别有效率"，成本与效率成为落地关键。
- [≈25:00] 物理 AI 的机会规模："全世界大概有 **1000 万个工厂、400 万个工业机器人、5.7 亿个农场、12000 个城市、15 亿辆汽车、20 亿个摄像头**"，这些机器大部分还没有智能；AI 从数字世界走到物理世界"将是一次生产力的大爆发"，影响农业、运输、建造、能源、healthcare、life science、logistics、manufacturing、robotics。

- [29:53] **Cosmos 想提供的三种帮助**：**better data**、**better environment**、**better study**（据上下文疑为 **better simulation** (?)，讲的是替代实车测试）。建模型、开源模型、开源建模型的工具与数据。
- [≈30:30] **better data**：物理数据难收集（例：希望自动驾驶在"三崩/三车相撞"这类状态也正常运作，但这种状态难以在真实世界采集；又如机器人抓水壶，换水壶、换场景就要重新采一遍）；Cosmos 生成真实世界难以获得的数据，并让已采集数据"产生更多价值（augmentation）"；还能**切分视频并逐段解释发生了什么**，帮你在训练时挑出**关键数据**而非全部数据。
- [≈32:00] **better environment**：机器人/自动驾驶与真实世界互动成本极高（造机器人、造场景、造触发都花钱），若学习必须依赖真实交互，领域发展就快不起来；Cosmos 作为 **world model** 提供"训练环境"，让机器人、自动驾驶与 world model 互动快速学会技能。
- [≈33:00] "用**算力换时间**"的比喻：像电视剧里主角进入"异次元世界"，在那里 10 年相当于人间 1 分钟——把物理世界投进这个虚拟世界做海量训练，"十年磨一剑"再回来落地；因为在真实世界"你永远被时间速度限制，不可能跑得比时间快"，而有竞争力就要求你比时间快。
- [≈34:00] **better study/simulation**：自动驾驶新版本要选哪个 checkpoint 落地，传统做法是装车、由驾驶员带车跑各种场景对比——成本高且**有些场景无法重建**；用世界模型替代测试过程就能**迭代更快**，"在开发者的世界里面迭代速度就是一切"。
- [35:19] **World（foundation）model 的定义**：机器人教科书上的定义是"**given your current observation and action taken, what the future state looks like**"；他给出更宽的定义——建模是为了**解决问题**，不同人要解 perception、state estimation、manipulation 等不同问题，就需要不同的 world model，所需数据也不同。
- [≈36:30] **Foundation model** 的思路是"纳百川"：把各种数据放在一起，让模型去解释所有数据，从而学到核心概念并**泛化**到数据中未包含的东西。
- [37:43] 关键论点：虽然各领域数据都不一样，但"**他们的数据都来自同一个世界**"，聚合后可学到"你还没想到的 world model 所需要的能力"——即泛化能力，这对物理世界的开发团队是"很大的工具"。
- [≈38:20] 这些能力主要有几类：**信号理解**（视频里发生了什么）、**预测未来**、**simulation**（不同 action 会怎样）、**逆推 action**（从结果推断动作）。
- [≈39:00] 开源范围：**模型 + 数据（data factory，含如何生成数据、如何标注/挖掘自动驾驶与异构数据的方法）+ 框架**（他们自用的训练框架，验证可行后开源，含 training、post-training、一些 sim 与加速组件）。自嘲像"NVIDIA 的慈善机构"。
- [≈40:00] 新一代模型是 **omni model**：输入 **text / image / video / audio / action**，输出同样这些模态的各种组合；应用包括视觉理解、生成新训练数据、仿真、以及"pass 行/规划"。
- [≈41:00] **Cosmos 3 架构**：混合（hybrid）架构，**两个损失/两塔**——左边 **autoregressive**（世界理解，用语言去理解看到的视觉信号），右边 **diffusion**（生成，每个 token 可以 attend 到所有 token）；支持多视角（multi-view），因为具身本体普遍有多个摄像头。
- [≈42:20] 三个尺寸：nano（最小）、super（16B）、ultra（60B，追求最高 accuracy）(?)；另有面向机器人的 **Edge** 版本，目标跑在 **Jetson Orin / "S、Orin" 这类平台**上；中间尺寸适合做实验。
- [43:21] **Visual reasoning 的应用**：数据整理与标注（切开视频、抽取重要片段）、**robot planning**（把"做个披萨"拆成开冰箱、取出、放进烤箱等步骤）、工厂场景里判断工人操作是否合规；生成侧可做 image-to-video 预测；且**支持 multi-view**。

- [≈45:00] 把 **action 提升为 first-class citizen**：一般 foundation model 把 language 或 video 当一等公民，但解物理 AI 必须把 action 也带进来——因为世界的变化都是本体 action 造成的。难点是"每个本体都说不同的语言"（自动驾驶、机械臂、灵巧手各不相同）。
- [≈45:40] 为此定义了**几何化的 action vocabulary**：**ego pose**（现在在哪、往哪看）、**end-effector pose**（手末端在哪、往哪走）、**1D gripper**（夹爪开合）、**手指关节位置**；任何本体都是这几个元素的**排列组合**（摄像头=一个 ego；自动驾驶=ego+pose；固定机械臂=end-effector+gripper；双臂=两个 end-effector+两个 gripper；人形机器人再加全身自由度）。
- [47:18] 有了 action 之后可把三个要素放进同一框架互相促进：**observation**（你看到什么）、**dynamics**（哪些 action 导致状态变化，对生成训练数据很有价值）、**video + action generation**（给 instruction 与场景，推断要做什么、会发生什么、会看到什么）。
- [≈48:00] 相关模型与应用：将开源 **Cosmos 3 Transfer(?)**（实时 transfer 类模型，转写"NC transfer"）、专为**双臂机器人**设计的模型，以及两个与自动驾驶相关的——一个**自回归 driving model**（像开车游戏：左转右转加速，生成对应视频）、一个支持**多视角（11 个视角）+ 激光雷达**生成的模型，用于仿真或训练。
- [≈49:00] Cosmos 从 **2025 年 1 月**开始，已近两年；**Cosmos 3.5** 在建，规划两个尺寸 **ultra** 与 **light**，将增加 **multi-view LiDAR**、支持更多分辨率，并强化 instruction/推理能力，同时融入 agent。
- [50:38] 模型全部托管在 **Hugging Face**（三个主要 family），软件在 **GitHub**，数据在 **Hugging Face**；有 **Cosmos Dataset** 与 **Cosmos samples**，示范如何用 base/super 模型做后训练。云平台方面提到中国的**阿里云、字节**，美国的**微软**以及"covi / 利BI尔斯"（疑为 CoreWeave、Oracle 等 (?)）。
- [≈51:20] 一个重要的产品转向：过去 framework 服务"人类开发者"，现在"**每个开发者都自带一个智能体**"，真正使用框架的往往是 agent，因此要给 framework 加 **skills**，让它更适合 agent 使用；并**从只支持 Cosmos 自家模型扩展到支持数十个模型**（含 Qwen、GLM(?)、以及一些"哈大模型"等开源模型，转写不清 (?)），这些组合都经海量 GPU 验证并做过推理优化。
- [≈52:20] 总结三点：(1) NVIDIA 卖 GPU、卖铲子；(2) 希望成为开发者"强大的靠山"；(3) Cosmos 以开源数据、框架、模型服务物理世界开发者，并透露下一代规划（Cosmos 3.5）。

D. 交接（[52:57]–[53:47]）

- [52:57] 主持人感谢"明玉"，串词点题："从数据仿真到生成式世界模型的三位一体，物理 AI 的开发范式正在被重新定义"。
- [53:17] 转向"开源 AI 生态"，请出 **ADLR 副总裁 Brian**（转写"胡瑞"）：介绍他将"完整拆解 Nemotron 开源模型的全链路——从架构设计、训练数据、权重开放、后训练配方到评测体系"；[53:47] 录音结束。

11:03–12:10 录音二

A. Brian：Nemotron 3 与 Megatron（[00:00]–[18:15]，绝对 11:03–11:21）

- [00:00] 训练是**闭环飞轮**：data curation → **pre-training**（数千张 GPU 数周同步跑）→ **post-training RL**（rollout 与 reward 全规模）→ **evaluation**（判断模型强弱并指导开发）→ **inference**（部署并生成下一批数据），"这是连续循环，每一环都是巨大的算力挑战"。

- [≈00:40] **Nemotron Ultra** 的制作流程图：从 **20 万亿 token** 预训练开始，数据先分类、再经大量实验确定混合比例（blend）；接着做**长上下文扩展到 1M token**，得到 base checkpoint。
- [≈01:20] **SFT** 阶段用"teacher 生成"：从大量不同模型（如 **DeepSeek、GPT-OSS、Qwen(?)、MiniMax、GLM**）生成数据，再用 **Data Designer** 造更多**合成数据**，合成 SFT 数据集，得到 RSFT checkpoint。
- [≈02:00] 然后做**统一的 RLVR（verifiable rewards）**，包含 **15 个环境**（terminal、软件工程、search、tool use 等），产出 student checkpoint。
- [≈02:30] 再把这个 checkpoint **投影到 10 个不同领域**做成各自的 teacher 模型（每个 teacher 只擅长问题的一部分），好处是**可并行化**——不同团队各自把自己的 teacher 做到极致。
- [≈03:00] 最后用 **multi-domain on-policy distillation** 把这些 teacher 的能力合到一个最终模型；部署前再做量化（低比特算术）与 **speculative decoding** 等优化以加速推理。
- [≈03:40] **Nemotron 3 家族**共四个模型（转写含糊，"Nano/Super/Ultra/Bold" (?)）；模型分为两类目标：**数据中心模型**（在 GB200/NVL72 这类大平台上最大化"每一美元的智能"，可用 **expert parallelism** 降低部署成本）与**端侧模型**（把最多的智能塞进小设备）。
- [≈04:20] 端侧案例：把 Super 从 **120B 总参 / 12B 激活**压缩到 **74B / 9B 激活**，再量化到 **2-bit**，塞进 **<24GB 内存**跑在 **RTX Spark 笔记本**上；结论是 2-bit 是"能塞进 23GB 内存里最聪明的模型"的取舍。
- [≈05:00] 新发布的 **120B（12B 激活）** 模型继承了 Nemotron 3 家族架构优势，主要提升来自更强的**后训练**；在 **Artificial Analysis** 指数上（带视觉能力）得分 **26**，超过更大的 Nemotron 3（原文 "full truck" (?)），但"还有很多优秀模型在我们前面"。
- [≈05:40] 核心主张：**"intelligence is efficiency"**——同样智能水平下速度可达 **5 倍**；优势来自架构、软件与**投机解码（speculative decoding）**；单卡（"one Blackwell" (?)）即可获得不错的智能与速度。
- [≈06:20] 外部评测（**LocalAI** 公司）在 RTX Spark 上做的 frontier plot（任务准确率 × 耗时）显示 Nemotron 3.5 Super 位于前沿。
- [≈06:50] **可定制性**是 Nemotron 的定位：不是"拿来就用"，而是让企业用自己的**私有数据**微调、在自己可控的环境部署，数据留在组织内部。
- [≈07:20] 五个定制案例：**Scale**（PDF 文档智能，测试集准确率 **+15%**，超过"quad opus five" (?)）；**AI21** 与 Alice（后训练后在**安全攻击韧性**上超过 "GPT-6.1 s soul" (?)）；**Ramp**（超过"bgp six lunar"，转写不清 (?)）；**Lila**（科学实验，驱动机器人实验室，后训练后在自身任务上超过"GPT-6 astra" (?)）；另一家做**医疗保险理赔**达到同类最佳，远超"Claude/Opus"系列 (?)。
- [≈08:20] 效率再举例：**SWE-bench Pro**（agentic 软件工程任务）上，完成 **10000 个任务**比许多同类模型**快 4 倍**且准确率优秀。
- [≈09:39] **Megatron**：多年积累的 AI 开发与部署软件、开源在 GitHub；本次讲 **Megatron Core** 的新特性。回顾并行方式：data parallel、tensor parallel、pipeline parallel、**FSDP**。
- [≈10:20] 新特性 **Generalized Tensor Parallelism（GTP）**：类似 FSDP 但更高效，参数 sharding 的方式"更贴合硬件"，从而系统整体更高效；对比 modular FSDP 的 gather/reduce-scatter 气泡，GTP 通过不同的 materialization 方式减少等待。

- [≈11:00] GTP 的灵活性：**MoE 层与 dense 层可各自独立设置 expert sharding 大小**、支持 **prefetch recomputation**（专用 recompute 预取链）、weight sharding policy（fc1/fc2 始终 shard，小张量可 opt-in 不 shard）；因为权重直接 shard，**与 DP、optimizer、checkpoint 解耦**，更容易集成 **Muon** 等矩阵级优化器与 **NVFP4 / MXFP8(?)** 等 block-wise 量化。
- [≈12:00] 在 **Nemotron 3 Ultra** 上与标准 3D 并行对比，modular FSDP 有加速，而 GTP 更快；代码在 GitHub 上开发中。
- [≈12:40] **Megatron Inference**（实验性研究项目）：现在多数 RL 系统是"训练循环（Megatron）+ 推理循环（vLLM/SGLang 做 rollout）"耦合，他们问"是否需要这么复杂"，结论是**直接用 Megatron 做推理同样高效**；收益：不必在推理框架与训练框架之间反复转换 checkpoint、新架构"day-zero"即可推理、训练与推理保持一致（避免 **logprob mismatch** 拖累 RL）。强调它不是通用推理服务器，只服务 RL rollout。
- [≈14:20] **位级可复现训练（bitwise reproducible training）**：Megatron Core 现在**确定性**，在约 **2000 张 Blackwell、万亿参数**规模上验证，确定性带来的减速只有 **2%**；可重放计算让训练更稳定，也便于诊断**硬件故障**，"resilience 是大规模训练极重要的一部分"；计划默认开启确定性模式。
- [≈15:40] **更快重启**：checkpoint 存/取从 **300 秒**降到（原文数字含糊 (?)）；初始化从"2 到 5"(?)、首次迭代从 **225 秒降到约 100 秒**；手段包括 caching、checkpoint plans、跳过冗余文件系统检查、延迟索引、pooled kernel、缓存 autotune 结果。
- [≈17:00] 弃用 **GPTModel** 类，转向 **hybrid model** 架构：可自由组合不同 attention（linear attention、dense attention 等），Nemotron、**Qwen3-Next**、**DeepSeek** 等都能用它直接表达；后续会精炼 API，让人更容易实验模型架构，从而"与硬件协同设计（co-design），更多效率就是更多智能"。
- [≈18:00] 结语：感谢中国开发者，称"这里的创新令人敬佩"，"我们都在为这个星球创造更多智能"。

B. 圆桌"从开放到可控"（[18:19]–[01:06:56]，绝对 11:21–12:10）

开场与自我介绍

- [18:19] 主持人串场承接：我们了解了物理 AI 和开源大模型，"但真正让技术走进生产的是工程师们的实战选择"；[18:28] 介绍 6 位嘉宾，[19:01] 请上台。
- [19:33] 主持人开场引用狄更斯《双城记》：**"it was the best of times, it was the worst of times"**——现在是"红利与阵痛并存"的最好时代，在座开发者"非常幸运"处在 AI 飞速发展的阶段。
- [≈20:20] 主题阐释：今天聊"**从开放到可控**"，并再加一个"**可用**"（从开、到可用、到可控）；背景是大模型与 agent 在生活工作中渗透越来越深。
- [21:32] 请每位嘉宾自我介绍并回答"harness 在你岗位上意味着什么"。
- [21:50] **韩晓（Dify）**：harness 的意义就是"到底**用什么**去完成任务"——让 AI 盖一座城堡，是"从沙子揉出来"还是"用积木拼出来"？后者明显更快、更高效、更可控、更不容易让 AI 犯错。自称在 AI 时代大家都是"**全栈工程师**"。
- [22:34] **吴自华（NVIDIA GPU 计算专家）**：harness 的本质可以归纳为一个词——**interaction**，含三层交互：**人与 agent**、**agent 与 agent**、**agent 与外部世界**（主要是整个计算系统，并通过 **computer use** 接触现实世界）。

- [23:18] **张家驹（vLLM 社区）**：过去两年在国内组织多次 vLLM 社区活动；harness 是"大模型真正落地、给企业或普通用户真正产生价值的一个非常有效的方式"；社区专长是"更快更好地产生 **token**"的底层基础设施（AI infra），为上层 harness/agent 应用打好基础。
- [24:07] **朱立根（NVIDIA Research）**：带队做 agent 策略与 **tokenomics** 相关开发；他们的理解是 harness≈**agent management**——人类历史上第一次拥有这么多"智力"（呼应 Brian 所说"把电批量变成智力的手段"），问题变成"如何有效地管理它"：现在多数 agent 都相当于有"清北本科生到硕博"的能力，若一家公司突然有 100、1000 甚至 1 万个"硕博生"，怎么把它们协同调动到同一个方向上？这就是 harness，做好调度才能把 agent 策略与 token 高效调动起来，把"以前要几年几十年的事缩到几个月几周"。
- [25:37] **姚勉（阶跃星辰）**：引用业界公式"**agent = harness + model**"，并指出 model 与 harness 的**边界在不断被打破**，harness 层的很多东西会随模型发展被**内化**到模型内部；因此作为基础大模型厂商，关注前沿开源 harness 非常重要。
- [26:45] 主持人点出嘉宾的代表性：覆盖"芯片层—平台层—模型框架—应用"的完整技术栈生态位，随后进入分问题环节。

问题一：开源模型发布时的上下游设计与开放边界（答：姚勉）

- [≈27:10] 补完 harness 理解后，引用一家"比较极端"的前沿模型公司观点：**harness 层终将消失**，会被逐渐内化进模型。例证：**Claude Code** 在某次重大更新中大幅删减系统提示词、拆掉外围脚手架，实测执行任务性能**不降反升**。
- [≈28:10] 工程史印证：两年前还没有"harness"概念，那时叫**提示词工程/上下文工程**；经典范式 **ReAct**、plan-thought 起初只是提示词编排与脚手架，当模型把这类范式**训练进数据**之后，脚手架就被拆掉了。
- [≈29:00] 除开放权重外，阶跃星辰的做法是开放"**model-native 的 harness**"——发布新模型（转写"step e co o" (?)）时同时开源自家的 harness，目的是让 harness 层最大限度发挥自家模型能力。
- [≈29:40] 三种交付形态：(1) **Agent SDK**，供企业集成到已有软件系统或自有 agent 中（最灵活）；(2) **场景化开发工具**（如 coding 场景的 CLI 与桌面端），让开发者更轻松驾驭新模型；(3) **Manager agent**——把 harness 层托管到模型厂商云端基础设施，企业不必自己维护 harness 基础设施，只需把 harness 连到内部系统与数据库；并预测模型厂商会从 **MaaS** 走向"**Agent as a Service**"，提供开箱即用的 agent 套件。

问题二：harness 评测最常犯的错误（答：朱立根）

- [31:26] 主持人把问题给到"李工/李根"（即朱立根 (?)）：开发者设计大模型 agent harness 评测时最常犯的错误。他先强调"harness 之间比较，评测是最重要的一步"。
- [≈31:50] **错误一：只做主观炫酷 demo，缺少可量化的客观 benchmark**。现在 GitHub 上 harness 脚手架框架极多，"正儿八经跑过 SWE-bench、跑过真实任务验收的非常少"；很多 harness 更像"role play"（他举"三省六部制"式玩法：一堆 agent 扮演大臣，让用户体验"赛博皇帝"）。研究视角真正想知道的是"有/没有这个 harness，在具体任务上性能差几个点"。
- [≈33:20] **错误二：忽略 endpoint 的非确定性带来的抖动**。多数大模型公司提供的 endpoint 不确定——同样 prompt 在不同时间生成的 token 不同，variance 可能比框架本身差异更大；他们组的做法是**每组实验跑 5 次**，确保结果 solid 可复现。

- [≈34:20] **错误三：对前沿模型能力认知不到位导致评测失灵**。例如直接拿通用 benchmark 去评 harness，某些 harness（如 DeepSeek(?)、Codex 类工具）会去**读取根目录下其他 session 的文件**，"做着做着偷看前面答案"，评测完全失真；更不用说模型厂商报告过多次 **jailbreak（越狱）**事件。
- [≈35:00] 对策：做**强隔离**——起容器，把所有流量与链接监控起来，防止越狱与偷看；在"5 次跑 + 隔离 + 可量化指标"之上才能建出可信、可复现、可公平比较的评测系统。主持人插话共鸣：他们团队也遇到过 agent"找了别人的好结果、声称是自己做的"。

问题三：多轮长上下文、记忆一致性与上下文存储（答：张家驹）

- [≈35:40] 提问：从单轮到多轮超长对话、可开始存储上下文之后，这会成为很大的系统瓶颈；在记忆一致性、状态管理、上下文检索上，开源方案如何解决？harness 又如何评测这些能力？
- [≈36:20] 回答（先介绍定位）：推理引擎的核心是"让模型更快、更好地产生大量 token"，并在生产环境解决随之而来的系统级问题。
- [≈36:50] 一方面，引擎要跟上模型能力的变化——随着时间推移，越来越多能力被加进模型，引擎侧要为这些变化提供支撑/视野。
- [≈37:30] 另一方面，针对上下文一致性等问题，通过**外接存储**的方式、并与**安全系统**良好配合来解决。

问题四：区分"有效探索"与"昂贵空转"，以及把失败沉淀为资产（答：吴自华）

- [38:11] 回答第一问的关键是**奖励函数（reward function）的设计**：奖励函数设计偏了就会进入"昂贵空转"。
- [≈38:50] 例一：若把"软件系统好"定义为"测试多么全面完善"，最终会得到一个用各种组合覆盖现实中根本不会出现的极端 case 的**庞大测试库**，而真正需要的功能反而被拖累——典型的昂贵空转。
- [≈39:50] 例二：用 agent 做数学定理证明，若一开始不在框架上把目标规定好，最后可能得到**几万个各自成立、但加起来也无法向主结论推进一步的引理**。
- [≈40:30] 第二问（如何把失败变成资产）：**人类时代成立的东西在 AI/agent 时代仍然成立，只是形式与基础设施需要升级**。例：Anthropic 说要在 Git 之上重造版本管理，不是 AI 时代不需要代码版本管理，而是 Git 的设计/架构/呈现形式需要改造升级。
- [≈41:10] 知识积累同理：人类靠文字与书籍实现几千年传承；AI 时代只要把成功经验与失败教训像"LMDB(?)/lmvp"那样**系统地归纳总结**，形成**可追溯、可检索的结构化文档**，就能在新任务与探索性任务中复用。

问题五：把推理框架、上下文存储串成可用产品（答：韩晓，Dify）

- [41:41] 主持人把问题给到"离用户最近"的 Dify（转写"DK/低费"）。
- [≈42:10] 韩晓回顾：2023 年大模型很火，团队想做自己的 AI app，过程中发现"从 LLM API 到最终 APP 中间还差很多"——需要**可观测性**（知道模型跑了哪些数据、钱花在哪里）、需要连接企业已有的业务系统与数据库/知识库。
- [≈42:50] 把这些补齐之后就成了 Dify，并且**开源**（"我们觉得这个东西不是只有我们需要，整个世界可能都需要"）；名字含义就是"**do it for you**"。

问题六：demo 很惊艳、准备上线时最核心的建议（答：张家驹、吴自华、韩晓）

- [43:03] 提问面向"离卡最近的三位嘉宾"：用开源模型做一个应用，demo 效果惊艳，现在要上线，各给一条最核心建议、要避免哪些坑。

- [≈43:20] **张家驹**：demo 阶段的 benchmark 与真实流量有差别；上真实流量后，一看功能性，二要考虑**潮汐/流量暴增**，确认当前设计能否满足线上环境与用户需求；提到他们在大规模生产实践上有大量经验，可线下交流。
- [≈44:40] **吴自华**：要对 demo 有**清醒认识**——demo 有点像在榜单上刷榜（模型榜首、harness/agent 刷榜），分数惊艳但离工程师与办公用户的日常使用还有很大距离；从 demo 到真正推向生产之间有一个非常关键的环节：**内部高强度、大量的使用**，只有当内部用户在内部生产中表现出足够可靠稳定，才具备对外上线的基础。
- [45:56] **韩晓**：同意前两位；Dify 今年年初做了桌面 agent "**ThinkNKS**(?)"（转写"think nks"），demo 非常好，但**go-to-market 遇到挫折**。反思结论：早期要**先找好场景**，确保该场景用户"用得爽"；做 **benchmark 与数据记录**（也是后续训练数据的积累）；然后**逐步**推到各领域，不要一上来大规模推广，每一步都要有详细评估；早期还**手动挑选用户**、限制新用户加入。
- [≈48:00] 主持人追问"怎么选用户"，韩晓答：**先自己用爽**——团队都是 coding 背景，先把日常开发流程（PRD → 研发实现 → 验收）用自己的产品跑通，做到"用自有产品迭代自有产品"的**自举**流程，确保没有明显 bug 达到可用标准；然后推给公司内部，例如先找**法务**（每天审核大量合同、有 SOP），中间层 agent 类似 workflow（每个人像一个 inbox，上游任务来了 agent 自动处理、再由人审批），团队**陪跑**观察问题并持续优化，跑通后再推广到其他部门。
- [≈49:30] 主持人小结：需要"**由内到外**"逐步推广。

问题七：harness 是否会成为"模型 + 推理框架 + 记忆上下文 + 工具"的标准控制面（答：朱立根、姚勉）

- [≈50:00] 提问给右侧两位：harness 是否有可能成为把模型、推理框架、记忆上下文、工具全部串起来的**标准控制面/标准组件**？
- [≈50:30] **朱立根**：这个时间点没人有 100% 确定答案——今年模型发展速度远超预期，年初的"激进派/保守派"之辩，"现在回过头来看保守派不算激进"。模型能力还在变强、scaling 没停，**任何标准化的东西都会很快被基模能力吃进去**，社区里活跃期可能只有 **2–3 个月**：例如早期 **ReAct** 范式、编码里的 **TDD（test-driven development）** 都一度成为 prompt engineering/harness 的"standard"，现在基本被模型内化了。
- [≈52:30] 但他不悲观：模型越强，**需求也会变得越多越奇怪**。类比 **2G→3G→4G** 的流量史：从翻盖机到大屏再到折叠屏，流量越来越多，并不是"用不完"，而是从纯文字到图片、视频、云游戏——标准化之后会在其之上开出新需求。模型同理：标准不停被吃进去，同时人们会在更强的模型上开出更多想法。
- [≈53:30] **姚勉**：平台化（涵盖模型、agent 运行时、对话管理、上下文）**会出现**，但"这两年还没达到那个程度"；国外前沿公司已在探索并想定义标准。但他同意朱立根：变化太快，很多能力被内化，没有人能提出被大多数人接受的标准。
- [≈55:00] 更认同"**同心圆**"比喻：agent 在外、model 在内，共同构成 agent；随着基模推理能力增强，整个 agent 能做的事越来越多，同心圆**半径会不断变大**——内部能力被吸收进模型，但边界同时外扩，最终"通往真正的人工智能"。主持人小结：两位意见一致，暂时看不到成为通用标准。

收尾展望：未来 1–2 年各层边界会怎么变（六位嘉宾依次）

- [56:02] **韩晓**：模型层负责"内核"，用户需求交给模型，**模型自己改造自己的 harness**——即现在很多模型厂在讲的 **RSI（递归自我改进）**；因此 harness 层应做的是"提供好让模型 DIY 自己 harness 的能

力"并开放出来；Dify 提供 **Dify CLI / Dify Core(?)** 这类东西就是让模型自己拖拽、自己编排 agent，这是一个趋势。

- [≈57:20] 呼应吴自华开场："harness 本质上是交互"；并观察到 **DSH（原文"dsh"）引领了插件化趋势**，Pi(?)/"派"是插件化的 agent 体系，**Claude Code** 最新改版也做了插件化改造并引入 **Claude Hooks**（转写"cloud 帽子" (?)）。
- [≈57:50] 他给出更激进的判断：harness 的终局可能不止于插件化系统，而是 **OS**——把操作系统的所有东西都包括进去。因为现在"瓶颈不是模型的吞吐，瓶颈是各种系统工具：存储系统、CPU 侧的任务执行能力跟不上模型发展的速度，它们占用了更多时间"。在 RSI 背景下，希望 agent 能把整个外围（即整个操作系统）完整改造与全面加速；未来完整的操作系统可能是"**内核层 + 用户层**"，内核由 AI agent 驱动，外围设施与程序间交互都由内核 agent 驱动并持续迭代。
- [58:38] **张家驹**：1–2 年尺度"可能很长了"，变化太快不太敢预测。可观察到的是：最近几个月以 vLLM 为例，很多同事**基本不再手写代码**，给指令、连系统问题都交给 agent 去做，而这在几个月前大部分代码还是人手写。趋势是**人给 agent 的指令从细节变得越来越高阶、抽象、覆盖更大范围**；再过一两年随着模型能力增强，现在需要关心上下文等细节的范围会像"通讯半径"一样扩大，去做更高阶的事。
- [≈60:30] 另一层提醒：今天在这个场合能看到，底层不论 GPU 还是 CPU，**性能能不能打满、GPU 与 CPU 能否很好结合**仍然非常重要。他表达一个担忧：当人用更高层的界面操纵 agent 做事，对底层的了解会越来越少；"再过两年，大家都已经摆脱底层 low-level 的实践，是不是我们还能继续判断系统性能是否达标"。
- [01:01:14] **朱立根**：同意前两位——AI 时代会改变很多生产方式，harness 是其中非常重要的一环；他在 NVIDIA Research 更看前沿与 3–5 年趋势，给出一个"MBA 式"的预测：到 **2029 或 2030 年**会看到三个"1"——接近 **100T（万亿级千倍）参数的模型**、**1000 TPS**、**100 万个 agents**，最终走向 RSI/scale-out。
- [≈01:02:20] 在那个量级做 scale 时，现在很多基建都会成为瓶颈，尤其**软件工程的护城河会转移**。举例：他们组最近用 agent 非常大规模地重写"cuBLAS 类/CUDA 相关"代码（转写"qwl"，(?)），并坦言自己上次一行行手写 CUDA 已是 2025 年 3 月、约一年半没手写过；借助 agent + 自研 harness，可快速把性能优化到 **human SOTA** 级别甚至更好。
- [≈01:03:30] 另一例：与**康奈尔**合作把**几何版的庞加莱猜想**做了形式化，用一整套 harness 系统在**三天内写完 270 万行 Lean 形式化代码**，完成整个工作。由此推论：当模型足够聪明、agent 足够多，"无限量"地做时，关键是有一个**有效的 harness 把 agents 调动起来**；那时很多软件功能、生产级 demo 可能就是一个 prompt 下去、几个 agent 军团替你完成。
- [≈01:03:50] 他开玩笑说那时"最大的瓶颈最后不是人的瓶颈，而会回到**卡和电**"——大部分生产可能是两件事：**电多加卡、卡多加电**，把"4 个 10 打满（利用率打满）"就是趋势；并强调已有许多成功信号证明"把电和卡堆上去，比堆能力更高效"。
- [≈01:04:25] **主持人最后猜想**：现在的基础设施（操作系统）是给人用的，训练数据也大多是**人类使用操作系统、编程的 trace**；有没有可能以后训练数据不再是人类产生的数据？那会对 harness 形态产生很大冲击——可能出现**专门给 agent 用的操作系统**，agent 在"更贴合基模理解"的 OS 上完成自闭环训练，"不是收集自然人使用编程的 traces，而是模型自闭环生成数据"。
- [01:06:44] 主持人收尾总结主题：**开放开源解决"企业用户用什么"的问题，harness 解决"用户敢不敢用、敢不敢上生产"的问题**；感谢 6 位嘉宾与幕后准备问题的同事。

- [01:07:10] 结束串场："从开放到可控不是一条直线，而是一个需要社区、平台、工程师共同参与的生态协作过程"；[01:07:28] 上午主论坛结束。

3. 与下午"LLM 训练与推理"主题的关联

上午场几乎为下午的 LLM 训练与推理分会场铺好了全部概念与动机：

- **推理成本与"intelligence per token / 最低每 token 成本"**：An Patel 直接把"推理需求爆炸 + 每 token 成本下降"作为全天主线，并在介绍下午分会场时明确把 AI infrastructure 分会场定义为"如何拿到最低的每 token 成本"；Brian 用"intelligence is efficiency"（同智能下快 5 倍、SWE-bench Pro 上万任务快 4 倍）与端侧 2-bit 量化落地了这句话的具体形态。下午的 serving/推理加速（如转写中的 KV cache 压缩、Dynamo 推理基础设施等）正是这条线的工程展开。
- **CUDA 与 tile programming**：An Patel 说 CUDA 最大的变化是把 **tile 算法**做成一等公民，并支持 Python/C++/Rust 写 tile 程序，这对下午的 **MoE kernel / 算子优化**类分享是直接的前置知识；他提到 CUTLASS、NCCL、TileLang(?) 等开源库，也是下午算子与通信优化的话题基础。
- **Megatron 与大规模训练实践**：Brian 讲的 **GTP（generalized tensor parallelism）、FSDP 气泡、位级可复现训练（确定性税仅 2%）、checkpoint 300s→更快重启、Megatron Inference 服务 RL rollout** 等，正好对应下午"结合 Megatron 库的大规模训练实践"与"完整训练系统"两个环节；他反复强调的"训练—后训练 RL—评测—推理"闭环也解释了为什么训练与推理两个话题被放在同一个 track。
- **开源模型与可复现**：An Patel 强调"open = 权重 + 数据 + recipes"，Brian 给出 Nemotron 的 SFT/RLVR/多 teacher 蒸馏全流程，圆桌里姚勉讨论"除权重外还开放 harness/SDK"，与下午"开源模型落地、后训练配方"的议题呼应。
- **MoE 与并行**：GTP 允许 **MoE 层与 dense 层使用不同的 expert sharding 大小**、fc1/fc2 始终 shard 的 weight policy，是下午 MoE kernel 与并行策略讨论的直接线索。
- **推理/服务层的系统瓶颈**：圆桌里张家驹（vLLM）与吴自华反复回到"更快更好地产出 token""上下文存储与一致性""存储/CPU 跟不上模型吞吐"，这些正是下午 KV cache 压缩、上下文管理、推理引擎优化的动机。
- **智能体（agent）与 harness**：上午把"agent = harness + model"作为核心公式（姚勉），并讨论 harness 评测、长上下文记忆、插件化与"harness 终局是 OS"（韩晓、吴自华）。下午若涉及 **Dynamo 为智能体推理基础设施加速**等内容，正是"harness/agent 侧需求倒逼推理基础设施"的落点——圆桌里"评测要跑 5 次、容器隔离防偷看答案"等实践也与推理服务的确定性、可复现性直接相关。
- **物理 AI 与推理负载的多样性**：刘明玉的 Cosmos（多视角、action token、world model 仿真）提醒推理负载不止文本——多模态、长视频、实时（他说的"30Hz 级"约束）会带来完全不同的推理优化目标。
- 交叉参考（来自 `speaker_scan.txt` 的下午交接句，仅作对照，非本纪要依据）：下午依次有算子/OP 优化、结合 Megatron 库的 DeepSeek(?) V4 大规模训练实践、面向高效 LLM 服务的 **KV cache 压缩**、以及"Dynamo 为智能体推理基础设施加速"等议题，与上述线索一一对应。

4. 存疑与说明

1. **本纪要仅据语音转写，未经投影片核对。** 两段录音均无对应投影片；数字、模型名、公司名、人名均照转写文本记录，明显的 ASR 错字在正文标 `(?)` 并在下方给出推测。

2. **人名不确定处**： - 录音一开场致辞者 **Maggie / 麦琪(?)**：姓名仅来自串场词"感谢 maggie"，中文写法未出现。 - **An Patel / "安迪" / "anie"**：同一人在转写中有三种写法，中文串场词称"安迪"，英文自称处转写为"An Patel"，头衔为"NVIDIA 开发者产品副总裁"。是否同一人无法从转写确认 (?)。 - **刘明玉 / 刘明宇**：串场词写作"刘明玉"，结束语称"谢谢明玉"，`speaker_scan.txt` 记作"刘明宇"；头衔原文"nvi、cs ms la 副总裁"，推测为"NVIDIA **Cosmos Lab** 副总裁"(?)。 - **Brian / "胡瑞" / "brain" / "b brand"**：录音一末尾介绍为"NVIDIA 应用深度学习研究方向 ADLR 副总裁胡瑞，他带领团队构建了 cuDNN 和 Megatron"，录音二为英文演讲，英文姓氏未在转写中出现（推测为业内知名的 ADLR 负责人，但**无法据转写确证**，故不写具体英文姓氏）。 - 圆桌主持人**姓名未识别**：录音一 An Patel 提到"julian has a panel with the speakers..."，推测主持人可能叫 Julian (?)，但转写中未出现其自我介绍，亦未出现"Julian"的重复出现。 - **赖俊杰**（NVIDIA 工程和解决方案副总裁）：仅出现在圆桌开场的嘉宾名单串词中，其自我介绍段落未被清晰捕捉，姓名可信度较低 (?)。`speaker_scan.txt` 中该处转写为"赖俊杰...韩晓 d 全站工程师吴自华"等，标点与姓名切分明显有误。 - **韩晓 / "哈瑞"**：自我介绍为"我叫哈瑞韩晓"，其中"哈瑞"疑为英文名 Harry/Harrison 的音译 (?)。其公司转写为"低费 / d费 / DK"，据其自述"他的名字也是 d费，就是 **do it for you**"判断应为 **Dify**（(?)，公司名被 ASR 音译成中文）。 - **吴自华 / 吴子华**、**张家驹 / 张家军**、**朱立根 / 朱立登（亦被称"李工""李根""林哥"）**、**姚勉 / 杨敏 / 姚敏 / 杨远**：同一人多种写法并存，正文采用首次出现的写法，其余在括号中注明。
3. **明显的 ASR 错字与推测**： - "pdf 平台"→疑为 **DGX** 平台；"djx spark"→**DGX Spark**；"doy/doi 的培训"→疑为 **DLI**（深度学习培训）；"o pc 融创中心"→疑为某"AI PC/融创中心"名称 (?)，地点为**苏州**。 - "cut list"→**CUTLASS**；"库大 library"→**CUDA library**；"ccl 和 tylerre"→疑为 **NCCL** 与 **TileLang/Tile IR**(?)；"mega 创 / e 某创"→**Megatron**；"酷 dn"→**cuDNN**；"你檬床马"→疑为 **Nemotron**(?)；"奥巴马又"→疑为 **Omni/OmniX**(?) 或某开源模型名，转写过于破碎，无法判定 (?) - "cs ms la"→**Cosmos Lab**(?)；"俄比利亚 Cosmos"→**NVIDIA Cosmos**；"贝特贝塔 / better data"、"better environment"、"better study"——第三项按上下文（替代实车测试、迭代验证）疑为 **better simulation**，但讲者两次都说"study"，故照录并标 (?)。 - Cosmos 尺寸：转写"最小的 hob、中间的 16b、最大的 super 60b"，疑为 **nano / super(16B) / ultra(60B)** 的错乱 (?)，正文已按"三尺寸 + Edge 版"谨慎表述；Edge 版平台名转写为"s 或 orange 这种 HD 拜"，疑为 **Jetson Orin/Nova** 一类平台 (?)。 - 中国区云平台"包含阿里跟字节的云"，美国区"微软，还有 covi 跟利BI尔斯"→疑为 **CoreWeave、Oracle** 等 (?)。 - 物理 AI 场景规模中的"5 亿 7,000 万个农场""1,000 万个工厂""400 万个工业机器人""12,000 个城市""15 亿的汽车""20 亿个摄像头"照转写记录，未经核对；"h30 万"（机器人需在极短时延内反应）疑为"**30Hz** 级"或"30 万分之一秒"(?)。 - Brian 段：模型名与榜单名多处破碎——"quad opus five""gpt 6.1 s soul""bgp six lunar""gpt six astra""cloud of the spire"疑分别指 **Claude/GPT 系列竞品与 Claude**；(?) 均无法确证。"12 million active parameters to nine"从上下文看应为 **12B→9B 激活**(?)。"one black hole"疑为 **一张 Blackwell**(?)。checkpoint 存取"从 300 秒"之后的目标数字转写缺失 (?)。模型家族四个名字（Nano/Super/Ultra/Bold）中"Bold"与"data center models"的对应关系不清 (?)。Nemotron Inference 用于 RL rollout 的既有框架"vlm 或 sg length"疑为 **vLLM 与 SGLang**(?)。 - 圆桌中"ThinkNKS"（Dify 的桌面 agent 名）、"LMDB/lmvp"（系统性文档化方法）、"qwl"（被 agent 大规模重写的库，疑为 **cuBLAS/CUDA** 相关）、"另的形式化代码"（疑为 **Lean** 形式化证明）、"DSH/dsh"（引领插件化的项目名，疑与本次开发者日相关的开源 harness）、"cloud 帽子"（疑为 **Claude Hooks**）、"派"（疑为某插件化 agent 体系名）均标 (?)。 -

朱立根提到的"100T 模型、1000 TPS、100 万 agents"照转写记录（"100t""1000 的 TPS""一 million 的 agents"），其确切含义未核实。

4. **时间戳精度**：`[分:秒]` 中带 `≈` 的为按段落内文字比例**估算**的时间（转写只在段落级给出起止时间，如 `[13:22 - 29:49]`），误差可达数十秒；不带 `≈` 的时间来自 `speaker_scan.txt` 的关键句时间，或转写段落边界。
5. **录音不完整**：录音一从开场致辞中途开始（首句为断句"您基于打开发一次..."），因此 Maggie 致辞前半段缺失；录音二从 Brian 演讲中段开始（首句 "That data center. Then we have um pre training..."），且录音一结束于 [53:47]（10:39）而录音二始于 11:03，**中间约 24 分钟的 Brian 演讲开头未被任何一段录音覆盖**。
6. 圆桌的问答对应关系（哪个问题由哪位嘉宾回答）依据问答间的串场句与回答风格推断，个别处（如"李工/李根"确指哪位嘉宾）存在不确定性，已在正文标 (?)。

附录 B：ASR 术语对照与纠错表

讯飞转写对**中文技术口语里的英文术语**识别偏差很大，几乎每个术语都有一个稳定的"错误音译形态"。下表是整理 96 页窗口时反复出现的对应关系，可用于**核对原始 `.srt` / `.md` 转写**，也说明了本文术语是怎么纠正的。

转写里的样子	实际术语	主要出现场次
罗go my / 俄罗my / 我 m	local / GMEM（本地全局显存）	第 1 场
科能 / 开过 / 开 s / 挑赛	kernel / tile	全场
IP / IPA / ipl / iplog / IPO / IP 露我	epilogue	第 1 场
既然的 aw / 开测后 / 探后 / WMA	WGMMA / MMA	第 1 场
圈了 / 托 / 头 / 通数量	token（数量）	第 1 场
沃克 / 沃铺 / 卧铺 / 例子 / 网格	warp	第 1 场
我顾客 / 我本部	warpgroup	第 1 场
不探色 / 不探索	per-tensor（量化）	第 1 场
d 个 GS / QQ 的个版本	DeepGEMM	第 1 场
通通 / 平台 / 靠微信 / 拼棒	ping-pong、cooperative	第 1 场
毛巾 ps / 猫 t 卡 / ca	（TMA）multicast	第 1 场
拍卖的结构	pipeline	第 1 场
ST 机 / ST 及 128 / stg16	`STG` / `STG.128` / `STG.16`	第 1 场
销售	shuffle	第 1 场
县城	线程（thread）	第 1 场
计术器 / 技术器	寄存器 / 计数器	第 1 场
合模型 / 广模式 / promot 型 / 和模型	promotion（FP32 promote）	第 1 场
捐入 / 此微转入 / 活动	SwiGLU	第 1 场
例子和 AP / get 和 up p	gate / up（门控与上投影）	第 1 场
客户 k / 抗办 / 抗班 / 抗败	combine	第 1、2 场
卖票 / 后客	MegaMoE	第 1 场
卡拉斯的	CUTLASS	第 1 场
e 蔻的	NVSHMEM（NVIDIA 通信库，`(?)`）	第 1 场
MV / m moe	MoE	第 1 场

转写里的样子	实际术语	主要出现场次
麦克窗 / 麦窗 / Mt 克	Megatron	第 2 场
tckv4 / dpc 个 v4 / DC 个 v4 / 维斯	DeepSeek-V4（-Pro）	第 2 场
葡萄 / 葡	CUDA Graphs	第 2 场
丢人 / 流人	fusion	第 2 场
一兆	1M（上下文长度）	第 2 场
s8so / ss	CSA（稀疏注意力路径）	第 2 场
e 该	Indexer	第 2 场
group o / group 口 o	Group-O Projection	第 2 场
应应该是	attention	第 2 场
脱矿 / 拖拉机 / 通矿	TurboQuant	第 3 场
kvtc / 开始管理	KVTC / KV cache 管理	第 3 场
pmc / pvpc / 微看	nvCOMP（-ANS）	第 3 场
Stv / spv	FlexKV	第 3、4 场
地纳米 / dynamo	Dynamo	第 4 场
老 / rooter	router	第 4 场
action ma	hash map	第 4 场
c 卡人的 r tree	concurrent radix tree	第 4 场
啤酒零	P90	第 4 场
拜托的曲线	Pareto（曲线）	第 4 场
垂直的	trajectory（轨迹级 SLA）	第 4 场
阿里斯 / 哈斯 space 斯	harness	第 4 场、上午场
三类阵 / 三级	KV 三级存储（GPU / CPU / SSD）	第 4 场
p 死	OOM / 打爆容量	第 4 场
免费的午餐	无损压缩（讲者的比喻）	第 3、4 场

使用建议：读原始转写时，先按上表替换术语，再结合本文第五章的逐页要点理解；若某句话仍不通，多半是被截断的短句（转写按停顿切句），可回到对应页的窗口文件 `_build\audio\pages\NNN.txt` 看上下文。

附录 C：产物与方法清单

文件	说明
`大模型训练与推理_讲稿整理（录音对照）.pdf`	**本文件**：录音转写 × 投影片对照册
`大模型训练与推理_内容整理与总结分析.pdf`	投影片内容整理与总结分析（主文档，已按第六章修订）
`大模型训练与推理_图片影印本.pdf`	96 页原图影印本，含隐形 OCR 文本层
`转写结果\*.srt / *.md / *.json`	讯飞转写原始产物（四段录音）
`_build\audio\alignment.json`	照片时刻 ↔ 音频偏移 ↔ 场次/讲者 的对齐表
`_build\audio\pages\NNN.txt`	每一页对应的转写窗口（含时间戳）
`_build\audio\qa\qa1-4.txt`	四场问答段的转写（原文档未覆盖部分）
`_build\audio\notes\seg01-06.md`	逐页口头要点的中间稿
`_build\audio_align.py`、`audio_talks.py`、`audio_qa.py`、`assemble2.py`	对齐、打标签、问答切段、文档装配脚本

复现流程：`python _build\audio_align.py`（解析 srt 并按照片时刻切窗口）→ `python _build\audio_talks.py`（按交接点标注场次讲者）→ `python _build\audio_qa.py`（切出问答段）→ `python _build\assemble2.py` → `python _build\build_doc2.py`。